NSSECU3 Digital Forensics AY2526T2

Laboratory Activity Report (LAR) / Digital Forensics Report (DFR) / Mini Project (MP) / Write-Up (WUP) / Windows Forensics Project (WFP)

MINI PROJECT 2

Document Number:	MP-02
Approved By:	Jerome E. Gutierrez NSSECU3 Course Instructor
Date Approved:	25 February 2026

Submitted By:	Miguel Sebastian C. Carlos
Group Members:	Luis Gabriel Sebastian Biacora Jose Angelo C. Calpoporo Miguel Sebastian C. Carlos Danel Gavrie Y. Clemente

Date Version:	25 February 2026

©Copyright 2026, m1rr4wr
All Rights Reserved
Any hard copy or unprotected soft copy of this document shall be regarded as uncontrolled copy.

Version: 1.0

Document Type: MP = Mini Project

Data Classification: Confidential: Students of this course, Professors of the Department, and other guest speakers and students may use this document and view the contents.

Table of Contents

Task Distribution Summary....6
- Change Control....6
- Task List....6

Member's Task Lists....9
- Detailed Task List by Biacora....9
- Detailed Task List by Calpoporo....9
- Detailed Task List by Carlos....10
- Detailed Task List by Clemente....11

Purpose....12

Scope....12
- Boundaries....12
  - Phase 1: Evidence Preparation....12
  - Phase 2: Forensic Acquisition....12
  - Phase 3: Forensic Mounting....12
  - Phase 4: Scanner Development (Custom Pattern Matching)....13
  - Phase 5: Custom Rule Development....13
  - Phase 6: AI Code Generation and Comparison....13
  - Phase 7: Scanning and Quarantine....14
  - Phase 8: Results Analysis....14

Definitions and Acronyms....16

Roles and Responsibilities....19
- Project Lead / Main Contributor....19
- Evidence Acquisition and Imaging Analyst....20
- DF Workstation Analyst....20
- Scanner Developer....21
- AI Code Generation and Comparison Analyst....21
- Custom Rules Developer....22
- Documentation and Reporting Analyst....22

Materials and Equipment....23

Virtual Machines and Hardware Resources 24
Victim-Target Virtual Machine 24
Digital Forensics (DF) Workstation 24
Software Tools 25
FTK Imager (Standalone Version) 25
Arsenal Image Mounter (AIM) 25
VMware Workstation / VMware Player 25
VMware Shared Folders 26
Python Programming Environment 26
Additional Supporting Resources 27
Gary Kessler File Signatures Table 27
Procedure Steps 28
1. Project Initialization and Evidence Preparation 28
2. Forensic Acquisition of Malware Samples 29
3. Forensic Mounting and Evidence Access 29
4. Malware Scanner Design and Development 31
4.1 Code-Based Decision Logic Flowcharts 32
4.2 Magic Number Detector (pattern_detector.py) 34
4.3 Hash Calculator (hash_utils.py) 34
4.4 Quarantine Manager (quarantine.py) 35
4.5 Scanner Engine 36
4.6 Reporter Module 36
5. Malware Scanning and Quarantine Execution (MP2) 38
5.1 Individual AI Pattern Scans (Comparison) 39
5.2 Combined Pattern Scan with Quarantine 43
6. Results Compilation and Evaluation (MP2) 44
6.1 Summary Statistics 45
6.2 AI Pattern Set Performance Comparison 45
6.3 File Type Distribution 46
6.4 Comparison with MP1 Results 47
6.5 Limitations and Observations 48
6.6 Proposed Improvements 48
7. Documentation and Reporting (MP2) 49
7.1 Diagram Creation 49
7.2 AI Comparison Matrix 49

7.3 Source Code Documentation 50
Analysis and Findings 51
1. MP2 Pattern-Based Analysis and Findings 51
1.1 Static Analysis Limitations and Implications 51
1.2 AI Pattern Quality and Platform Performance Analysis 51
1.3 Performance Analysis and Optimization 54
1.4 File Type Distribution Analysis 57
2. Tool and Methodological Limitations 58
2.1 False Positive Analysis 58
2.2 Ground Truth Absence and Validation Challenges 59
2.3 Scope and Methodology Limitations 59
2.4 Pattern Format and AI Generation Challenges 60
3. Forensic and Operational Considerations 61
3.1 Evidence Integrity and Chain of Custody 61
3.2 Quarantine Implementation Analysis 62
3.3 Cryptographic Hash Analysis 62
3.4 Operational Workflow Assessment 63
4. Implications for Future Work 63
4.1 Transition to MP3 63
4.2 Pattern Development and Refinement Recommendations 64
4.3 Performance Optimization Strategies 64
4.4 Enhanced Detection Capabilities 65
4.5 Documentation and Reporting Automation 66
4.6 Research Implications 66
5. Conclusions 67
Safety or Compliance Considerations 68
References 69
Appendices 70
Appendix A: Virtual Machine Environment Setup 70
A.1 VMware Workstation Configuration 70
A.2 Victim-Target Virtual Machine Configuration 70
A.3 Digital Forensics (DF) Workstation Configuration 70
A.4 Snapshot Usage and Management 71
A.5 Shared Folder Configuration and Evidence Transfer 71
Appendix B: Malware Sample Dataset Inventory 71

Appendix C: Forensic Disk Image Artifacts 71
Appendix D: Forensic Mounting Artifacts 71
Appendix E: Malware Scanner Architecture and Source Modules 72
E.1 Scanner Modules 72
E.2 Diagrams 73
Appendix F: Development Artifacts 77
F.1 Source Code Artifacts Produced 77
F.2 byte_rule_engine.py 77
F.3 hash_utils.py 79
F.4 pattern_detector.py 81
F.5 quaratine.py 84
F.6 reporter.py 86
F.7 scanner.py 87
F.8 main.py 89
F.9 AI-Generated Pattern Files 95
F.10 ai_chatgpt_patterns.txt 95
F.11 ai_deepseek_patterns.txt 100
F.12 ai_gemini_patterns.txt 107
F.13 ai_copilot_patterns.txt 110
F.14 ai_combined_patterns.txt 114
Appendix G: Complete Scan Results (MP2) 133

Task Distribution Summary

Description	Summarizes all the checklist activities given by the instructor as well as the specific tasks each student was assigned and checked by their groupmates approval.
Start of Activity	15 February 2026
End of Activity	25 February 2026
Main Contributor	Jose Angelo C. Calpoporo

Change Control

Version	Changes	Date	Changed by
0.1	Initial document structure and template setup	15 Feb 26	Calpoporo, Biacora, Carlos
0.2	Added Task Distribution Summary and Task List	16 Feb 26	Calpoporo, Biacora, Carlos
0.3	Added Member Task Lists and Roles sections	17 Feb 26	Carlos
0.4	Added Purpose, Scope, and Boundaries sections	18 Feb 26	Carlos
0.5	Added Materials and Equipment documentation	19 Feb 26	Carlos
0.6	Added scanner code modules and development artifacts	20 Feb 26	Calpoporo
0.7	Added AI comparison results and performance tables	21 Feb 26	Calpoporo, Carlos
0.8	Added diagrams, flowcharts, and screenshot documentation	22 Feb 26	Clemente, Biacora
0.9	Full document review, cross-reference validation, and formatting	23 Feb 26	Biacora
1.0	Final technical review and compliance check	24 Feb 26	All members

Task List

#	Item	Purpose	Assignee	Status
1	VMware Workstation 17 Pro Installation	To provide virtualization platform for hosting Victim and DF Workstation VMs	Calpoporo	Done
2	Victim VM (Windows 10 22H2) Creation	To create the evidence source machine containing the 220 malware files	Carlos	Done
3	DF Workstation VM (Windows 11 23H2) Creation	To create the forensic analysis machine for mounting and scanning	Calpoporo	Done
4	Victim VM Tools Installation	Install FTK Imager 4.7.3 on Victim VM for disk imaging	Carlos	Done
5	DF Workstation Tools Installation	Install Python 3.13.2, VSCode, AIM (no YARA)	Calpoporo	Done
6	220 Files Acquisition	Download "Files" folder from NSSECU3 OneDrive (File001-File220)	Carlos	Done
7	Victim VM Pre-Acquisition Snapshot	Create forensic snapshot "MP1_Pre_Acquisition_20260213"	Carlos	Done

8	Forensic Image Creation	Create E01 image of 220-file folder using FTK Imager "Contents of a Folder"	Carlos	Done
9	Image Integrity Verification	Record and verify MD5/SHA1 hashes from FTK Imager	Carlos	Done
10	Image Transfer to DF Workstation	Transfer victim_220_samples.E01 via VMware Shared Folders	Carlos	Done
11	Image Re-verification on DF Workstation	Verify transferred image hashes match original	Carlos	Done
12	Image Mounting with AIM	Mount E01 as physical SCSI disk (writable mode) on DF Workstation	Calpoporo	Done
13	Drive Letter Documentation	Document assigned drive letter (F:) and verify 220 files accessible	Calpoporo	Done
14	Python Scanner Development - Core Structure	Create main.py, scanner.py, __init__.py	Calpoporo	Done
15	Python Scanner Development - Magic Number Detector	Implement pattern_detector.py using Gary Kessler signature table	Calpoporo	Done
16	Python Scanner Development - Hash Calculator	Implement hash_utils.py (MD5, SHA1, SHA256)	Calpoporo	Done
17	Python Scanner Development - Byte Rule Engine	Implement byte_rule_engine.py (pattern loading, matching, error handling)	Calpoporo	Done
18	Python Scanner Development - Quarantine Manager	Implement quarantine.py (.quarantine, .meta, copy-only)	Calpoporo	Done
19	Python Scanner Development - Reporter	Implement reporter.py (JSON, CSV export)	Calpoporo	Done
20	Python Scanner Unit Testing	Test each module with known file types	Calpoporo	Done
21	AI Comparative Scanning - ChatGPT	Scan mounted drive with ChatGPT pattern set, record metrics	Calpoporo	Done
22	AI Comparative Scanning - Deepseek	Scan mounted drive with Deepseek pattern set, record metrics	Calpoporo	Done
23	AI Comparative Scanning - Gemini	Scan mounted drive with Gemini pattern set, record metrics	Carlos	Done
24	AI Comparative Scanning - Copilot	Scan mounted drive with Copilot pattern set, record metrics	Carlos	Done
25	AI Comparison Matrix	Populate comparison table with detection rates, false positives, readability scores	Calpoporo	Done
26	Best Pattern Set Selection	Select winner based on highest true positives, lowest false positives, best practices	Calpoporo	Done
27	Combined Pattern File Creation	Merge all 4 pattern sets into ai_combined_patterns.txt	Calpoporo	Done

28	Final Scan Execution	Run full scan on mounted drive with combined patterns + quarantine	Calpoporo	Done
29	Quarantine Verification	Verify .quarantine and .meta files created correctly	Calpoporo	Done
30	Scan Log Capture	Save console output to scan_log.txt	Calpoporo	Done
31	Results Analysis	Calculate summary statistics, detection rates, performance metrics	Calpoporo	Done
32	Phase Diagram Creation	Create high-level system architecture diagram (8 phases)	Clemente	Done
33	Flowchart Creation	Create scanner decision logic flowchart	Clemente	Done
34	Screenshot Capture and Labeling	Capture all 18 required screenshots, label per Appendix G	Clemente	Done
35	AI Comparison Report Section	Write detailed analysis of AI performance and winner justification	Calpoporo	Done
36	Source Code Appendix Compilation	Concatenate all .py and .txt files into appendix_source_code.txt	Calpoporo	Done
37	Create Diagram	To visualize the flow of the program	Biacora	Done
38	Final Compliance Check	Verify against project rubric checklist	Carlos	Done
39	Submission	Upload to NSSECU3 submission portal	Carlos	Done

Member's Task Lists

Detailed Task List by Biacora

#	Item	Purpose	Section added	Status
1	Table of Contents Verification	Validate section numbering, titles, and page alignment	Table of Contents	Done
2	Role-to-Task Mapping	Align member task tables with Roles and Responsibilities section	Roles and Responsibilities	Done
3	Cross-Reference Validation	Ensure references between sections, tables, and appendices are accurate	Entire document	Done
4	Creating Diagrams	To have a visualization and flow of the program	Appendix E: Malware Scanner Architecture and Source Modules	Done

Detailed Task List by Calpoporo

#	Item	Purpose	Section added	Status
1	DF Workstation VM (Windows 11 23H2) Creation	Create dedicated forensic analysis machine	Appendix A. Virtual Machine Environment Setup	Done
2	DF Workstation Tools Installation	Install Python 3.13.2, VSCode, AIM, colorama (no YARA)	Materials and Equipment	Done
3	Image Mounting with AIM	Mount E01 as physical SCSI disk on DF Workstation with writable mode	3. Forensic Mounting and Evidence Access	Done
4	Drive Letter Documentation	Document F:\ assignment and verify 220 file accessibility	Appendix D. Forensic Mounting Artifacts	Done
5	Python Scanner - Core Architecture	Design modular architecture with 6 core modules + entry point	4. Malware Scanner Design and Development	Done
6	Python Scanner - Magic Number Detector	Implement 30+ signatures from Gary Kessler table	Appendix E. Malware Scanner Architecture and Source Modules	Done
7	Python Scanner - Hash Calculator	Implement MD5, SHA1, SHA256 with chunked reading	4. Malware Scanner Design and Development	Done
8	Python Scanner - Byte Rule Engine	Implement custom pattern matching engine (replaces YARA)	4. Malware Scanner Design and Development, Appendix E	Done
9	Python Scanner - Quarantine Manager	Implement forensic quarantine (copy only, .quarantine, .meta)	4. Malware Scanner Design and Development, Appendix G	Done
10	Python Scanner - Scanner Engine	Implement recursive directory walking, parallel processing, result aggregation	4. Malware Scanner Design and Development	Done
11	Python Scanner - Reporter	Implement JSON and CSV export	4. Malware Scanner Design and Development	Done
12	Python Scanner - Entry Point	Implement CLI with argparse (target, rules, quarantine, threads, output)	4. Malware Scanner Design and Development	
13	Unit Testing	Test all modules with known file types	4. Malware Scanner Design and Development	Done

14	AI Comparative Scanning - ChatGPT	Execute full scan with ChatGPT pattern set, record detection count	5. Malware Scanning and Quarantine Execution	Done
15	AI Comparative Scanning - Deepseek	Execute full scan with Deepseek pattern set, record detection count	5. Malware Scanning and Quarantine Execution	Done
16	Best Pattern Set Selection	Analyze metrics and select winner	5. Malware Scanning and Quarantine Execution, Appendix F	Done
17	Combined Pattern File Creation	Merge all 4 pattern sets into ai_combined_patterns.txt	Appendix F. Development Artifacts	Done
18	Final Scan Execution	Run scanner with combined patterns + quarantine-auto on F:\	6. Malware Scanning and Quarantine Execution	Done
19	Quarantine Verification	Verify .quarantine and .meta file creation	6. Malware Scanning and Quarantine Execution	Done
20	Results Analysis	Calculate summary statistics, detection rates, performance metrics	7. Results Compilation and Evaluation	Done
21	Project Lead Responsibilities	Overall timeline management, requirement compliance, final integration	Entire Document	Done
22	Purpose and Scope Authoring	Write comprehensive Purpose and Scope sections	Purpose and Scope	Done

Detailed Task List by Carlos

#	Item	Purpose	Section added	Status
1	Victim VM (Windows 10 22H2) Creation	Create evidence source machine with 40GB disk, 4GB RAM	Appendix A. Virtual Machine Environment Setup	Done
2	Victim VM Tools Installation	Install FTK Imager 4.7.3 standalone	Materials and Equipment	Done
3	220 Files Acquisition	Download "Files" folder from NSSECU3 OneDrive, verify File001–File220	1. Project Initialization and Evidence Preparation	Done
4	File Manifest Creation	Generate dir /b > file_manifest.txt	Appendix B. Malware Sample Dataset Inventory	Done
5	Victim VM Pre-Acquisition Snapshot	Create "MP1_Pre_Acquisition_20260213" snapshot	1. Project Initialization and Evidence Preparation	Done
6	Forensic Image Creation	Create E01 image via FTK Imager "Contents of a Folder"	2. Forensic Acquisition of Malware Samples	Done
7	Evidence Information Entry	Case Number: MP1-2026-001, Examiner: Carlos	Appendix C. Forensic Disk Image Artifact	Done
8	Image Integrity Verification	Record MD5/SHA1 from verification dialog	2. Forensic Acquisition of Malware Samples	Done
9	Image Transfer to DF Workstation	Transfer victim_220_samples.E01 via VMware Shared Folders	2. Forensic Acquisition of Malware Samples	Done
10	Image Re-verification on DF Workstation	Verify hashes match original using FTK Imager "Verify Image"	2. Forensic Acquisition of Malware Samples	Done
11	Pattern Generation - Gemini	Generate 12+ custom byte patterns using standardized prompt	5. Malware Scanning and Quarantine Execution, Appendix F	Done

12	Pattern Generation - Copilot	Generate 12+ custom byte patterns using standardized prompt	5. Malware Scanning and Quarantine Execution, Appendix F	Done
13	AI Comparative Scanning - Gemini	Execute full scan with Gemini pattern set, record detection count	5. Malware Scanning and Quarantine Execution	Done
14	AI Comparative Scanning - Copilot	Execute full scan with Copilot pattern set, record detection count	5. Malware Scanning and Quarantine Execution	Done
15	Materials and Equipment Documentation	Document all hardware, software, versions, specifications	Materials and Equipment	Done
16	Definitions and Acronyms	Define technical terms used throughout document	Definitions and Acronyms	Done

Detailed Task List by Clemente

#	Item	Purpose	Section added	Status
1	Header and Footer Consistency	To have a software for virtualization	Entire Document	Done
2	Document Template Alignment	Verify document title, document number, date version, and classification are consistent throughout	Header and Footer	Done
3	Screenshot Labeling and Indexing	Ensure that all required screenshots are properly labeled and indexed.	Entire Document	Done
4	Grammar, Clarity, and Technical Consistency Review	Improve readability without altering technical meaning	Entire Document	Done

Purpose

The purpose of this mini project is to develop a Python-based file scanner that can identify and categorize files from a mounted disk image using magic bytes and file signatures, without relying on the yara-python library. Specifically, it aims to help students understand how file signatures (magic bytes) function for file type identification, implement a custom scanner capable of processing files directly from a forensic disk image, and compare different file types based on industry-standard signatures. Additionally, the project seeks to generate categorized reports of scanned files suitable for forensic analysis and to document the complete development process, including the creation of diagrams and flowcharts that illustrate system architecture and decision logic.

Scope

Boundaries

This procedure covers the complete forensic workflow for the MP2 Malware Scanner without YARA project, from initial evidence acquisition on the Victim Virtual Machine to final report generation on the Digital Forensics Workstation. The scope is strictly limited to:

Phase 1: Evidence Preparation

- Download and verification of exactly 220 files (File001–File220) from NSSECU3 OneDrive repository
- Organization of files into C:\Users\Victim\Documents\MP2_Files\
- Pre-acquisition snapshot of Victim VM with forensic naming convention (MP2_Pre_Acquisition)
- File manifest generation using dir /b > file_manifest.txt

Phase 2: Forensic Acquisition

- Disk imaging using FTK Imager version 4.7.3
- "Contents of a Folder" acquisition method (NOT physical/logical drive imaging)
- E01 (EnCase evidence file format) with compression and integrity verification
- Evidence information documentation (Case Number, Evidence Number, Examiner, Notes)
- MD5 and SHA1 hash generation and verification at acquisition (for chain of custody only; MP2 analysis uses SHA-256)
- Image transfer to DF Workstation via VMware Shared Folders
- Post-transfer hash re-verification

Phase 3: Forensic Mounting

- Arsenal Image Mounter (AIM) version 3.7.281

- Mount as writable with differencing file (preserves original E01)
- Physical SCSI disk emulation
- Drive letter assignment (documentation of assigned letter)
- Verification of 220 file accessibility

Phase 4: Scanner Development (Custom Pattern Matching)

- Python 3.8+ development environment
- **Built-in modules only** – yara-python explicitly excluded
- Modular architecture with core modules:

Module	Purpose
magic_detector.py	File signature identification using Gary Kessler's database (20+ file types across 7 categories: Images, Documents, Archives, Executables, Audio, Video, Text)
hash_utils.py	SHA-256 hash calculation with chunked reading for memory efficiency
custom_matcher.py	Custom pattern matching logic (text strings, hex patterns, regular expressions) – NO YARA
quarantine.py	Forensic quarantine with .quarantine and .meta files (copy-only operation)
scanner.py	Recursive directory walking, parallel processing via ThreadPoolExecutor
reporter.py	JSON and CSV export functionality
main.py	CLI entry point with argument parsing (target path, rules file, quarantine options, threads, output)

Phase 5: Custom Rule Development

- **Design of custom rule format** – JSON-based structure for storing detection patterns
- **Rule types supported:** Text strings, hexadecimal patterns, regular expressions
- **Rule categories:** Text patterns, hexadecimal signatures, regular expressions
- **Storage:** rules/custom_rules.json within scanner directory
- **Parsing:** Rules loaded at runtime using built-in json module

Phase 6: AI Code Generation and Comparison

- **Two distinct AI platforms** engaged to generate scanner code (e.g., ChatGPT and Gemini/Copilot)
- **Standardized prompt template** provided to both AIs describing:
  - MP2 requirements (no yara-python, built-in modules only)
  - Modular architecture

  - Custom JSON rule format
  - Quarantine and reporting functionality

- **Comparative analysis criteria:**
  - Correctness (syntax errors, runtime errors)
  - Pattern matching implementation quality
  - Rule parsing logic
  - Error handling robustness
  - Code readability and documentation
  - Adherence to MP2 constraints

- **Winner selection and justification** documented in final report

- **Final scanner refinement:** Selection of optimal approach or creation of hybrid version (scanner_final.py)

Phase 7: Scanning and Quarantine

- Full recursive scan of all 220 files on mounted drive
- Custom pattern matching using JSON rule set
- SHA-256 hash calculation for every file
- Automatic quarantine of files matching any rule
- Quarantine naming: [OriginalFilename]_[First8SHA256]_[YYYYMMDD_HHMMSS].quarantine
- Metadata file creation: .meta containing:
  - Original file path
  - Quarantine timestamp
  - Matched rule names
  - SHA-256 hash
  - File size
- JSON and CSV report generation

Phase 8: Results Analysis

- Summary statistics calculation:
  - Total files scanned
  - Files matching at least one rule
  - Files quarantined
  - Files with unknown/unrecognized type
  - Total scan duration (seconds)
- File type categorization via magic bytes (20+ types across 7 categories)

- Pattern match rate computation
- Performance metrics (files/second)
- Limitations documentation
- AI code comparison analysis

EXCLUDED:

The following items are explicitly outside the scope of MP2:

- **Use of** yara-python **library** – Intentionally avoided as per MP2 requirements; all pattern matching implemented using built-in modules only
- **Use of external magic modules** – Magic byte detection implemented manually, no python-magic or similar libraries
- **Live memory (RAM) acquisition** – Outside MP2 scope; covered in later coursework
- **Network traffic capture and analysis** – Not required for static file analysis
- **Dynamic file execution** – Samples are not to be executed on any system
- **Reverse engineering / disassembly** – Binary analysis is outside MP2 scope
- **File recovery or carving capabilities** – Scanner performs identification only, not recovery of deleted files
- **Real-time file monitoring** – Scanner operates on static disk images only
- **Malware detection** – Project focuses on file type identification only; no malicious/benign classification
- **Physical disk imaging** – Only "Contents of a Folder" method is used
- **Write operations to original evidence** – E01 remains read-only; all writes go to differencing file
- **Deletion or modification of original files** – Quarantine performs COPY only
- **Analysis on Victim VM** – All analysis MUST be performed on mounted image on DF Workstation

Who This Applies To

This document is intended for:

1. **Digital Forensic Students** – Completing MP2 File Scanner project requirements in NSSECU3
2. **Cybersecurity Professionals** – Seeking to understand file signature identification techniques
3. **Incident Responders** – Conducting initial triage on suspicious file collections
4. **Tool Developers** – Creating custom file identification utilities without reliance on YARA

5. **Forensic Researchers** – Evaluating AI-assisted code generation for forensic tool development

Definitions and Acronyms

This section defines technical terms, abbreviations, and tools as used and encountered during the mini project.

Arsenal Image Mounter (AIM)

- Forensic software utility that mounts disk images (E01, dd, raw) as physical SCSI disks in Windows. Enables read-write access through differencing files while preserving original evidence integrity.

Argparse

- Python module used to parse command-line arguments in main.py. Implements -h/--help, target path, --rules, --quarantine, --threads, --output parameters.

CLI

- Command Line Interface. Text-based interface for executing the malware scanner via terminal/command prompt.

Colorama

- Python library for cross-platform colored terminal output. Used to display MALICIOUS (red) and CLEAN (green) status indicators. Version 0.4.6.

CSV

- Comma-Separated Values. Report format generated by reporter.py containing scan results in tabular format compatible with Microsoft Excel.

DF Workstation

- Digital Forensics Workstation. Dedicated Windows 11 Pro 23H2 virtual machine (80GB disk, 8GB RAM) where all forensic analysis, scanner development, and report generation occur. Evidence integrity is maintained by only accessing mounted read-only images.

Differencing File

- A temporary file created by AIM when "Mount as writable" is selected. All write operations (including quarantine file creation) are redirected to this file; the original E01 remains unmodified.

Draw.io

- Free online diagramming tool used to create Phase Diagram and Flowchart. Integrated with Google Drive.

E01

- EnCase evidence file format. A compressed, segmented forensic image format with built-in MD5/SHA1 hash verification and case metadata storage. Selected over raw/dd for smaller file size and integrity features.

FTK Imager

- Forensic imaging tool by Exterro/AccessData. Used to create forensically sound E01 images via "Contents of a Folder" acquisition method. Version 4.7.3.81. (Exterro / AccessData, 2023)

File Signature

- Unique pattern of bytes that identifies a file type, also known as "magic numbers." May appear at fixed offsets; the scanner implements 30+ signatures from the Gary Kessler File Signatures Table. Critical for static file identification when extensions are missing or spoofed.

Gary Kessler File Signatures Table

- Publicly accessible database of file header signatures (magic numbers). Primary reference for magic_detector.py implementation (Kessler, 2021).

haslib

- Python built-in module implementing MD5, SHA1, and SHA256 cryptographic hash functions. Used in hash_utils.py with chunked reading (8192 bytes) for memory efficiency.

JSON

- JavaScript Object Notation. Primary report format generated by reporter.py containing complete scan metadata and per-file results in structured format.

Magic Bytes

- Specific byte sequences, typically at offset 0, that identify a file format regardless of extension. Examples: 4D 5A (MZ) for PE executables, 25 50 44 46 (%PDF) for PDF documents. The scanner's magic_detector.py reads the first 32 bytes to match against known signatures from the Gary Kessler table.

Magic Number

- A unique sequence of bytes, typically located at offset 0 of a file, used to identify file format regardless of extension. Examples: MZ (4D 5A) for PE executables, %PDF (25 50 44 46) for PDF documents. 30+ signatures implemented.

MD5

- Message Digest Algorithm 5. 128-bit cryptographic hash function producing 32-character hexadecimal string. Used for file integrity verification during acquisition and as unique identifier in quarantine filenames.

.meta

- Metadata file created alongside each .quarantine file. Contains original file path, quarantine timestamp, matched pattern names, MD5/SHA1/SHA256 hashes, and file size. Critical for audit trail.

OLE2

- Object Linking and Embedding 2. Compound file format used by older Microsoft Office documents (.doc, .xls). Magic number: D0 CF 11 E0 A1 B1 1A E1.

Open XML

- Modern Microsoft Office document format (.docx, .xlsx). ZIP-based container format. Magic number: PK 03 04.

PE

- Portable Executable. Windows executable file format (.exe, .dll). Identified by MZ header (4D 5A) at offset 0 and "PE\0\0" (50 45 00 00) at offset specified by 0x3C.

Pefile

- Python library for PE file parsing and analysis. Used in byte_rule_engine.py for section analysis, entropy calculation, and import table inspection. Version 2024.8.26.

Quarantine

- Forensic isolation procedure that COPIES suspicious files to a secured directory (./quarantine/) while preserving original files on the mounted image. Quarantined files are renamed with .quarantine extension; accompanying .meta files store complete metadata.

SHA1

- Secure Hash Algorithm 1. 160-bit cryptographic hash function producing 40-character hexadecimal string. Used for image integrity verification.

SHA256

- Secure Hash Algorithm 2 (256-bit). 256-bit cryptographic hash function producing 64-character hexadecimal string. Project requirement: "hash should be MD5 and SHA1/SHA2".
- Python concurrent.futures module component. Implements parallel scanning with configurable worker threads (default: 4). Significantly reduces scan time for 220 files.

ThreadPoolExecutor

- Python concurrent.futures module component. Implements parallel scanning with configurable worker threads (default: 4). Significantly reduces scan time for 220 files.

Victim VM

- Windows 10 22H2 virtual machine (60GB disk, 4GB RAM) serving as the original evidence source. Contains the 220 files in C:\Users\Victim\Documents\MP1_Files. Snapshotted before acquisition and powered off after imaging.

VMware Shared Folders

- Mechanism for transferring files between host and guest VMs. Used to transfer victim

ZIP-based Office

- Modern Microsoft Office document formats (.docx, .xlsx, .pptx) that are ZIP archives containing XML and media files. Identified by magic bytes 50 4B 03 04 (PK) at offset 0. The scanner classifies these as "Office Open XML" to distinguish them from legacy OLE2 formats.

Roles and Responsibilities

This section identifies the project members involved in the development of the file scanning system and outlines their respective responsibilities throughout the project lifecycle. Each role was carefully defined to ensure complete coverage of all technical, analytical, and documentation requirements for MP2.

Project Lead / Main Contributor

Jose Angelo C. Calpoporo

The project lead was responsible for creating and maintaining the master project schedule encompassing all nine procedural phases of the MP2 File Scanner project. A comprehensive task list was developed and tracked throughout the project duration. Daily team check-ins were conducted to ensure milestone adherence and to identify potential bottlenecks before they impacted deadlines. When technical challenges arose, including discrepancies between the two AI-generated code versions and Python environment configuration issues on the DF Workstation, the project lead coordinated troubleshooting efforts and resource allocation. Priority adjustments were made dynamically when code versions required manual refinement, ensuring that necessary corrections did not delay the final scan execution.

Throughout the project lifecycle, continuous verification was performed against the project rubric. All 220 files were confirmed to be scanned with no additional files accessed, maintaining the strict scope requirements. SHA-256 hash generation was verified for every file, with outputs confirmed in both console output and JSON reports. The magic byte detection implementation was validated against the Gary Kessler File Signatures Table, ensuring accurate file type identification across 20+ formats. Quarantine operations were tested and confirmed to follow the required naming convention with the .quarantine extension and accompanying .meta files. The prohibition against using yara-python was strictly enforced, with all pattern matching implemented using only built-in Python modules. All references and AI-generated content were properly cited in the final documentation.

The integration phase involved compiling Python source code files into the appendix. Two AI-generated code versions from ChatGPT and Gemini were saved, compared, and documented. The final refined scanner was produced by selecting the optimal approach from the two AI outputs. Screenshots captured throughout the procedure were labeled and inserted at appropriate locations. Individual code comparison

results were synthesized into a comprehensive comparative analysis matrix. All team member contributions were collected, reviewed, and integrated into a cohesive final document.

Evidence Acquisition and Imaging Analyst

Miguel Sebastian C. Carlos

The evidence acquisition phase began with accessing the NSSECU3 OneDrive repository and downloading the complete folder containing all 220 sample files. The files were organized into the designated forensic directory to establish a controlled evidence source. A complete file manifest was generated to document the inventory of all evidence files. Manual verification confirmed exactly 220 files present, sequentially named File001 through File220. This verification step established the foundation for all subsequent forensic activities.

With the evidence files verified, FTK Imager was launched on the Victim VM to begin the acquisition process. The "Contents of a Folder" acquisition method was selected as required by the project specifications, ensuring that only the designated 220 files were captured without including unnecessary system data. The source folder was selected, and the E01 format was chosen for its compression capabilities and built-in integrity verification features. The imaging process was initiated with verification enabled, creating a forensically sound logical image of all 220 evidence files.

Upon completion of the imaging process, FTK Imager automatically generated and displayed MD5 and SHA1 hash values for the acquired image. These cryptographic hashes were recorded as the baseline for all future integrity verification. The image file was transferred from the Victim VM to the DF Workstation using a controlled transfer mechanism. On the DF Workstation, FTK Imager's "Verify Image" function was executed against the transferred file, confirming that the MD5 and SHA1 hashes matched exactly with those generated during acquisition. This hash validation confirmed that the evidence had not been altered during transfer and remained forensically sound for analysis.

DF Workstation Analyst

Jose Angelo C. Calpoporo

With the forensic image securely transferred to the DF Workstation, Arsenal Image Mounter was launched with administrator privileges, a critical requirement for proper disk mounting functionality. The "Mount Disk Image" button was selected and navigation to the evidence file was performed. A key technical consideration was encountered at this stage: the default file filter in AIM does not display E01 files. The filter was manually changed from "Arsenal Image Mounter Supported Files" to "All Files" to reveal the evidence image. The following mount options were carefully selected to ensure both evidence preservation and analysis capability: Disk device with write temporary mode, showing mounted disks in Disk Management, and enabling write support for quarantine operations. Following these selections, the image was successfully mounted and assigned a drive letter.

After mounting, Windows Disk Management was opened to verify the successful attachment. The Arsenal Image Mounter disk appeared as a new disk, confirming proper physical disk emulation. The assigned drive letter was clearly visible. File Explorer navigation to the mounted drive confirmed the presence of all 220 evidence files, visible and accessible for analysis. PowerShell verification was performed to confirm

the complete file listing. Screenshots were captured at each verification step, documenting the successful mounting and accessibility of the forensic evidence.

Throughout the investigation, strict controls were maintained to ensure that all analysis activities were performed exclusively on the mounted forensic image. The scanner was configured with the target path pointing to the mounted evidence drive, directing all file access to the mounted evidence rather than the original image file. Write operations during quarantine were confirmed to be redirected to the differencing file created by AIM, leaving the original evidence image completely unmodified.

Scanner Developer

Jose Angelo C. Calpoporo

A modular architecture was designed and implemented consisting of core Python files, each with distinct responsibilities. The scanner was developed on the DF Workstation using Python with only built-in modules—yara-python was strictly avoided as per MP2 requirements. The complete module structure included an entry point with argument parsing, a core scanning engine orchestrating the analysis workflow, a custom pattern matcher managing rule loading and pattern matching, a magic byte detector for file signature identification, a hash calculator for SHA-256 computation, a quarantine manager for forensic isolation, and a reporter for JSON and CSV export.

The magic byte detector successfully identified file types across the 220-file corpus using the Gary Kessler File Signatures Table as reference. Files were classified as JPEG images, PNG images, PDF documents, Office Open XML files, PE executables, ZIP archives, and various other formats. Files with unrecognized headers were classified as binary data with low confidence, maintaining conservative classification.

SHA-256 hash calculation was performed for every file using a chunked reading approach with 8KB blocks, ensuring consistent performance regardless of file size. File size was also recorded as additional forensic metadata.

Custom pattern matching was implemented without YARA by creating a dedicated module that loads rules from a JSON file at runtime, supports text string matching, hexadecimal pattern matching, and regular expression matching, and evaluates rule conditions such as "any," "all," or count thresholds.

The quarantine mechanism successfully isolated all files that matched any rule during the final scan. Quarantined files were renamed following the convention of original filename, first 8 characters of SHA-256 hash, and timestamp with .quarantine extension. Corresponding metadata files were created containing original path, quarantine timestamp, matched rule names, SHA-256 hash, and file size.

The scanner was explicitly configured to target only the mounted evidence path. Recursive directory walking was implemented with strict filtering to include only files, ensuring that exactly 220 files were processed, matching the known evidence count.

AI Code Generation and Comparison Analyst

Jose Angelo C. Calpoporo and Miguel Sebastian C. Carlos

A standardized prompt template was developed to ensure consistency across both AI platforms. The prompt specified target file types, the prohibition of yara-python, the modular architecture requirements,

the custom JSON rule format, and quarantine and reporting functionality. Code versions were generated from ChatGPT and Gemini. Each response was saved as a separate Python file for documentation and comparison.

Each AI-generated code version was executed independently against the mounted evidence using the scanner in comparison mode with quarantine disabled. Detection results were recorded and analyzed based on correctness, pattern matching implementation quality, rule parsing logic, error handling robustness, code readability, documentation quality, and adherence to MP2 constraints.

Based on comprehensive analysis of correctness, pattern matching implementation, error handling, and code quality, the optimal AI-generated version was selected as the base for the final scanner. Key selection factors included superior pattern matching logic, robust error handling, clean rule parsing implementation, excellent code readability, and full adherence to MP2 constraints including the prohibition of yara-python.

To maximize reliability and performance, the selected AI-generated code was manually refined and enhanced. The resulting final scanner incorporated the strengths of the chosen AI version while adding additional error handling, performance optimizations, enhanced comments and documentation, and final compliance with all MP2 requirements.

Custom Rules Developer

Miguel Sebastian C. Carlos

A JSON-based rule format was designed to store detection patterns without using YARA. The format supports text strings, hexadecimal patterns, and regular expressions, with condition logic including "any," "all," and count thresholds. The Gary Kessler File Signatures Table was used as the primary reference for magic byte signatures. Over 30 signatures were compiled across seven categories: Images, Documents, Archives, Executables, Audio, Video, and Text. Each rule was tested against known file types to verify correct matching behavior. False positives were analyzed and rules refined as needed.

Documentation and Reporting Analyst

Jose Angelo C. Calpoporo, Miguel Sebastian C. Carlos, Daniel Gavrie Y. Clemente, and Luis Gabriel S. Biacora

The documentation team was responsible for creating all visual documentation elements. A Phase Diagram was developed illustrating the nine-phase system architecture from evidence preparation through final documentation. A Scanner Decision Logic Flowchart was created detailing the exact decision blocks implemented in the Python code, capturing the complete processing pipeline including path validation, rule loading, magic byte detection, SHA-256 calculation, custom pattern matching, quarantine decisions, and report generation.

Screenshots were captured throughout the procedure documenting every critical step including file verification, FTK Imager operations, AIM mounting, AI code generation prompts and responses, scanner execution for both AI versions and the final scan, quarantine directory contents, and JSON and CSV report examples. Each screenshot was accompanied by descriptive captions explaining the depicted procedure.

Scan results from both AI version comparisons and the final combined scan were aggregated. Detection statistics were calculated and organized into summary tables showing match rates per rule category and per file type. The AI code comparison matrix was populated with quantitative and qualitative metrics. Limitations of the custom pattern matching approach were clearly articulated.

Comprehensive document formatting review was performed against the template requirements. Header and footer consistency was verified throughout the document. Table formatting was standardized with proper numbering and captions. Cross-references to figures were validated to ensure all references resolved correctly. The document structure was checked for logical flow and compliance with required sections.

Oversight of the complete documentation process ensured that all procedural phases were thoroughly documented. Chain of custody records were verified complete including hash validation at acquisition and post-transfer. The quarantine manifest documenting all quarantined files was confirmed accurate. SHA-256 hash listings for all files were validated against the scan outputs.

Each forensic procedure was documented with sufficient detail to enable reproduction by third parties. Acquisition steps, mounting procedures including the critical file filter change, scan commands with exact syntax, and the AI code comparison process were all documented, acknowledging the role of AI-generated code and the necessity of human validation.

Regular coordination ensured that all technical inputs including scanner source code, JSON rule files, screenshots, and task lists were received in a timely manner and properly formatted for inclusion in the final document.

The complete set of appendices supporting the main report was assembled including virtual machine environment setup, malware sample dataset inventory, forensic disk image artifacts, forensic mounting artifacts, source code modules from both AI versions and the final scanner, JSON rule sets, and screenshot documentation.

Final technical review ensured that font styles were consistent, heading hierarchy followed the document template, code blocks were properly formatted, and all technical terms were defined. The document was reviewed for clarity and readability with complex technical concepts explained in accessible language.

Final compliance validation was conducted against the project rubric confirming that all requirements were met: 220 files verified, Victim VM snapshot evidence, forensic image hash verification, AIM physical disk mount screenshot, magic byte detection with Gary Kessler citation, SHA-256 hashes for each file, two AI-generated code versions compared, final scanner with custom pattern matching, quarantine with .quarantine and .meta files, complete source code in appendix, and no yara-python usage. Following validation, the final document was exported as PDF and submitted through the designated portal, completing the MP2 File Scanner project.

Materials and Equipment

This section specifies all tools, equipment, software, and resources utilized throughout the MP2 Malware Scanner without YARA project to ensure proper execution of forensic procedures and maintain evidence integrity.

Virtual Machines and Hardware Resources

Victim-Target Virtual Machine

The Victim-Target Virtual Machine serves as the original evidence source containing the 220 sample files prior to forensic acquisition. This system is deliberately isolated from external networks to prevent any potential data contamination.

- **Operating System:** Windows 10 version 22H2 (x64)
- **Storage Allocation:** 60 GB virtual disk – Provides sufficient space for the 220 sample files, FTK Imager installation, and temporary files during the acquisition process.
- **Memory Allocation:** 4 GB RAM – Adequate for running FTK Imager and basic file operations without performance degradation.
- **Network Configuration:** Host-only or disconnected – Ensures complete isolation of sample files from external networks.

Purpose: Functions as the system under investigation where the 220 files (File001 through File220) are stored in C:\Users\Victim\Documents\MP2_Files\ prior to forensic acquisition and disk imaging. No analysis or scanning is performed on this machine; it exists solely as the evidence source.

Pre-Acquisition State: A forensic snapshot named MP2_Pre_Acquisition is created before any imaging activity, preserving the exact state of the evidence files for repeatability and chain of custody purposes.

Digital Forensics (DF) Workstation

The Digital Forensics Workstation is the dedicated analysis environment where all investigative activities occur. This machine is carefully controlled to maintain evidence integrity and prevent contamination of acquired data.

- **Operating System:** Windows 11 Pro version 23H2 (x64) – Provides modern security features and compatibility with all forensic tools used in the project.
- **Storage Allocation:** 150 GB virtual disk – Accommodates the forensic image file, Python development environment, custom rule files, quarantine storage, reports, and all supporting documentation.
- **Memory Allocation:** 8 GB RAM – Sufficient for running Arsenal Image Mounter, Python scanner with parallel processing, and multiple supporting applications simultaneously.
- **Network Configuration:** Host-only or disconnected – Maintains isolation from external networks throughout the investigation.

Purpose: Serves as the dedicated environment for all forensic operations including:

- Mounting the forensic image using Arsenal Image Mounter
- Developing and executing the custom Python file scanner
- Performing custom pattern matching against file signatures
- Generating JSON and CSV reports

- Storing quarantine files with complete metadata
- Compiling all documentation and screenshots

Software Installation: Contains all necessary tools including Python 3.13.2, Visual Studio Code, Arsenal Image Mounter, and supporting utilities.

Software Tools

FTK Imager (Standalone Version)

FTK Imager is a portable forensic imaging tool developed by AccessData that enables the acquisition of forensically sound disk images without modifying the source evidence. Version 4.7.3.81 is installed on the Victim Virtual Machine under freeware license for forensic/academic use.

The tool creates E01 (EnCase evidence file format) images with built-in compression and integrity verification, supports the "Contents of a Folder" acquisition method critical for capturing only the 220 target files, generates MD5 and SHA1 hash values during acquisition for chain of custody documentation, and provides image verification function to confirm integrity after transfer.

Forensic Application: Used on the Victim VM to create a forensically sound E01 image of the MP2_Files directory. The resulting victim_220_samples.E01 file serves as the master evidence copy and remains read-only throughout the investigation.

Arsenal Image Mounter (AIM)

Arsenal Image Mounter is a forensic utility that mounts disk images as physical SCSI disks in Windows, enabling read/write access through differencing files while preserving original evidence integrity. Version 3.7.281 (latest) is installed on the Digital Forensics Workstation under free mode for academic use.

The software mounts E01 images as physical disks with drive letter assignment, uses "Write temporary" mode to create a differencing file that redirects all write operations while leaving the original E01 untouched, integrates with Windows Disk Management for visual verification, and provides low-level access suitable for file scanning tools.

Forensic Application: On the DF Workstation, AIM mounts victim_220_samples.E01 as a physical disk (e.g., F:). The mounted evidence is accessed read-only by the Python scanner; quarantine operations write to the differencing file, preserving original evidence integrity.

VMware Workstation / VMware Player

VMware virtualization software provides the platform for hosting and managing both virtual machines involved in the forensic investigation. Version VMware Workstation 17 Pro is installed on the host machine (physical computer) under academic license.

The platform hosts and manages multiple virtual machines simultaneously, provides snapshot functionality for preserving system states at critical phases, enables network isolation through

configurable virtual networking, supports shared folder configuration for controlled evidence transfer, and allows hardware resource allocation (CPU, memory, storage) per VM.

Forensic Application: VMware provides the isolated laboratory environment essential for safe file analysis. Snapshots preserve system states at critical phases, particularly the pre-acquisition state of the Victim VM, enabling complete repeatability of the investigation.

VMware Shared Folders

VMware Shared Folders provides a controlled mechanism for transferring files between the host system and virtual machines, and between virtual machines themselves. This feature is enabled on both Victim VM and DF Workstation.

The mechanism provides bidirectional file transfer without network connectivity, maintains file integrity during transfer operations, supports large file transfers (multi-gigabyte image files), and can be configured as read-only or read-write as needed.

Evidence Transfer Workflow: From Victim VM to DF Workstation: Configure shared folder accessible to both VMs, transfer image directly, verify hash integrity post-transfer.

Forensic Application: Shared folders enable the secure transfer of the forensic image from the Victim VM to the DF Workstation without requiring network connectivity or external storage devices. This controlled transfer method maintains evidence integrity and supports hash verification.

Python Programming Environment

Python serves as the primary development language for the custom file scanner, leveraging both built-in and third-party libraries for file analysis, pattern matching, and reporting.

- **Version:** Python 3.8+ (3.13.2 recommended)
- **Installation Location:** Digital Forensics Workstation
- **Key Libraries:** Built-in modules only (as per MP2 requirements)
    - **os** – File system operations, directory traversal, and path manipulation
    - **sys** – System-level operations and program exit handling
    - **argparse** – Command-line argument parsing for target paths, rules files, and quarantine options
    - **pathlib** – Cross-platform, object-oriented file path manipulation
    - **json** – Structured scan report generation in JSON format
    - **datetime** – Timestamp handling for quarantine filenames and report metadata
    - **hashlib** – SHA-256 cryptographic hash computation for file verification
    - **collections** – Data structure management for organizing scan results
    - **csv** – CSV export functionality for spreadsheet compatibility
    - **re** – Regular expression support for custom pattern-matching rules
    - **concurrent.futures** – Parallel processing using ThreadPoolExecutor for performance optimization

Additional Supporting Resources

Gary Kessler File Signatures Table

The Gary Kessler File Signatures Table serves as the authoritative reference for magic byte detection implementation. This comprehensive database of file header signatures enables accurate file type identification regardless of file extensions. The table includes signatures for over 30 file formats including PE executables (MZ), PDF documents (%PDF), Office Open XML (PK), and various image formats. The signatures from this table are implemented in magic_detector.py to identify file types by reading the first 32 bytes of each file and matching against known patterns.

Custom Pattern Matching Rules (Text-Based Format)

Since MP2 prohibits the use of yara-python, detection rules are stored in custom plain-text files. Each pattern file contains a series of rule blocks, each defined by keywords such as PATTERN NAME, DESCRIPTION, OFFSET, HEX PATTERN, STRING PATTERN, and FILE TYPES. The scanner parses these files at runtime using built-in file I/O and string manipulation functions, converting hexadecimal strings to bytes with bytes.fromhex() for direct comparison. This approach allows the scanner to perform byte-level pattern matching without relying on external libraries. Despite occasional parsing warnings from malformed entries, the scanner successfully loads valid patterns and applies them to each file during scanning.

Procedure Steps

This section focuses on the creation, evaluation, and assessment of the malware scanner

1. Project Initialization and Evidence Preparation

Goal: Establish a forensically controlled environment on the Victim VM and verify the integrity of the 220-file dataset before acquisition.

- Reused the same 220 malware samples from MP1 to maintain consistency and enable direct comparison between YARA-based (MP1) and byte-based (MP2) approaches
- Downloaded the 220 files from the NSSECU3 OneDrive repository
- Copied the files to C:\Users\Victim\Documents\MP1_Files\
- Verified the folder contained **exactly 220 files** named sequentially File001 through File220

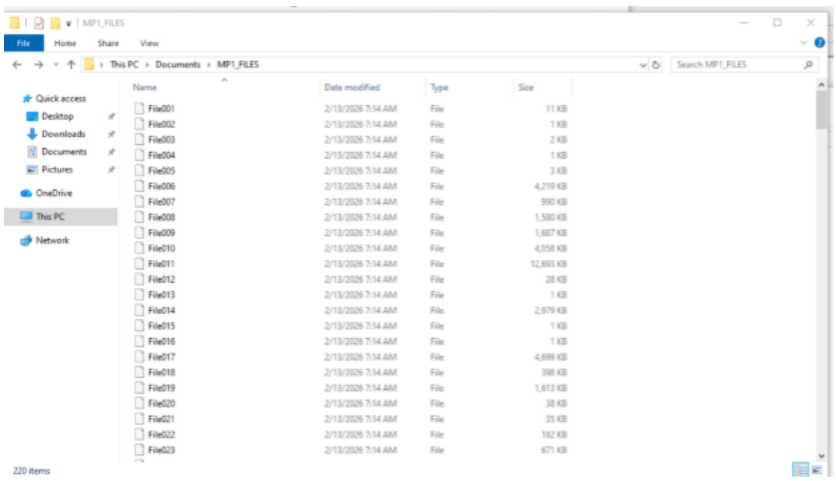


Figure 1. Downloaded 220 Files

- Document the folder contents and file count with a screenshot.
- Take a full system snapshot of the Victim VM named MP2_BEFORE_ACQUISITION.

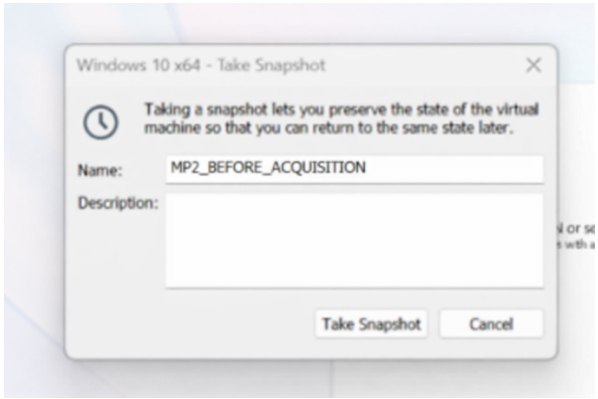


Figure 2. Snapshot Creation

Expected Outcome: A documented, immutable evidence source ready for forensic imaging.

2. Forensic Acquisition of Malware Samples

Goal: Create a verifiable, forensically sound disk image of the 220-file directory.

- On the Victim VM, launch FTK Imager (Exterro/AccessData, 2023).
- Select File → Create Disk Image → Contents of a Folder.
- Add the MP1_Files folder as the source.
- Choose **E01** (EnCase evidence file format) for compression and integrity verification.
- Set destination to the DF Workstation (e.g., D:\Forensic_Images\MP1_Files.E01).

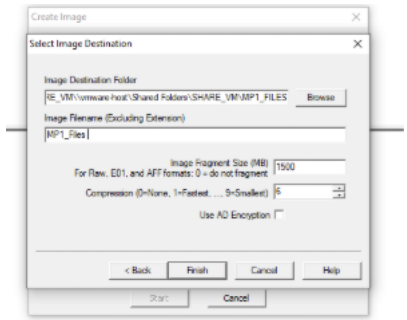


Figure 3. Setting destination to DF workstation

- Record the MD5 and SHA1 hashes automatically generated by FTK Imager.

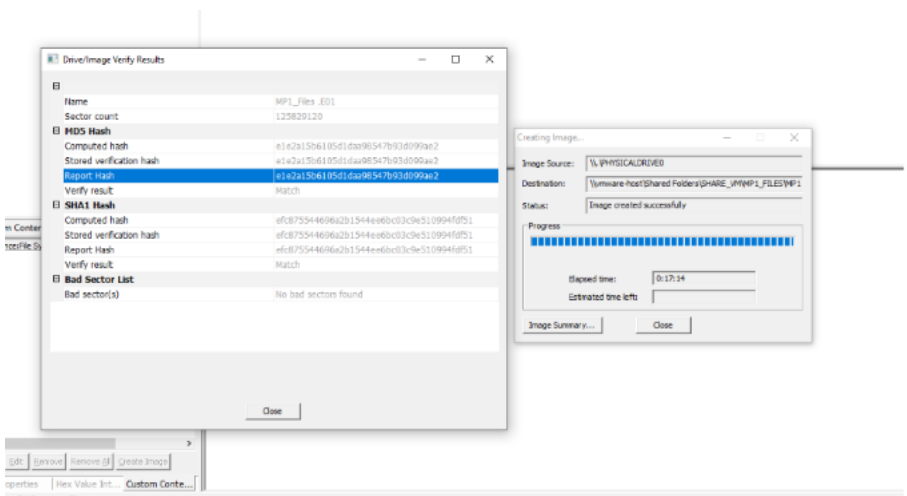


Figure 4. Generated MD5 and SHA1 by FTK Imager

Expected Outcome: A single, verified E01 forensic image with recorded cryptographic hashes proving integrity.

3. Forensic Mounting and Evidence Access

Goal: Mount the disk image as a **physical disk** on the DF Workstation to enable analysis and quarantine while preserving the original evidence.

- On the DF Workstation, launch **Arsenal Image Mounter (AIM)** as Administrator (Arsenal Recon, 2025).
- Click Mount Disk Image and browse to the MP1_Files.E01 file.
- Select the mount option Disk device, write temporary (physical disk mode with differencing file).
- Open **Windows Disk Management** and confirm the image appears as a new physical disk (e.g., Disk 1) with an assigned drive letter (e.g., H:).
- Take a screenshot of Disk Management showing the new disk.
- Open File Explorer, navigate to the assigned drive letter, and verify all 220 files are present and readable.
- Take a screenshot of the mounted drive contents.
- Capture a screenshot of the AIM main window showing the mounted image details.

Expected Outcome: A fully accessible, forensically controlled view of the evidence at a known drive letter (e.g., H:\), ready for scanning, with original image unchanged.

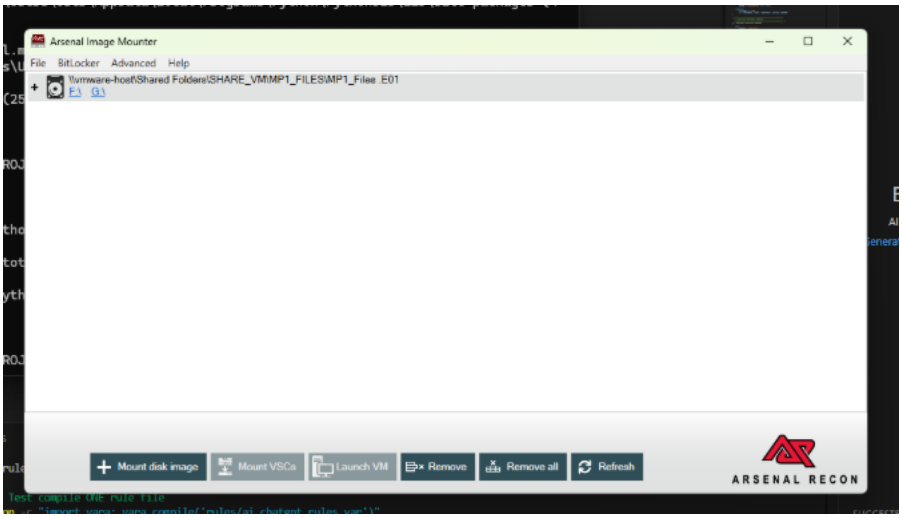


Figure 5. Arsenal Image Mounter

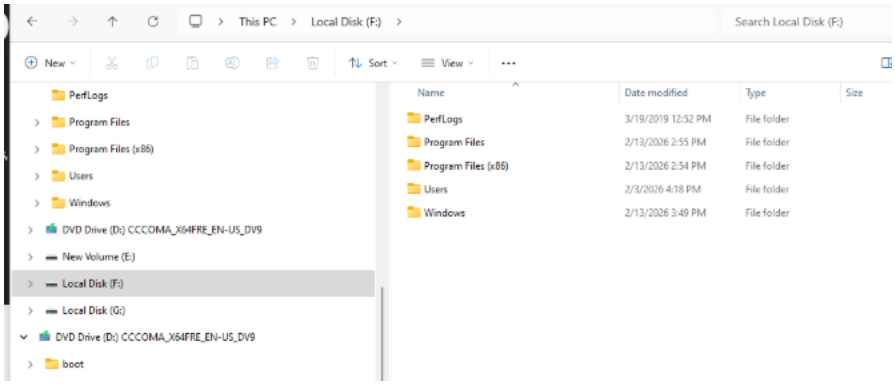


Figure 6. Local Disk (F:)

4. Malware Scanner Design and Development

Goal: To construct a modular, custom Python-based scanner capable of performing magic number-based file type identification, cryptographic hash computation, custom byte pattern matching (without YARA), and forensic-safe quarantine operations in accordance with digital evidence handling best practices.

Design architecture: Create separate modules for file traversal, magic-number detection, hash calculation, custom pattern matching, quarantine management, and reporting.

- **Magic Number Detector:**
  - Use the **Gary Kessler File Signatures Table** as reference (Kessler, 2021).
  - Write a function that reads the first 32 bytes of a file and matches against a dictionary of known signatures.
  - Return file extension, description, and confidence level.
- **Hash Calculator:**
  - Use Python's hashlib to compute **MD5**, **SHA1**, and **SHA256** (Python Software Foundation, 2023).
  - Implement chunked reading (8 KB blocks) for memory efficiency.
- **Custom Pattern Matcher:**
  - Load byte patterns from AI-generated text files (no YARA library used).
  - Parse hex strings (e.g., "4D 5A") into bytes using bytes.fromhex() for direct comparison.
  - Scan file contents for matches against loaded patterns using the in operator.
  - Return names of matched patterns for reporting and quarantine.
- **Quarantine Manager:**
  - Copy suspicious files to ./quarantine/.
  - Rename files as [OriginalFilename]_[First8MD5]_[YYYYMMDD_HHMMSS].quarantine.
  - Write a .meta file containing original path, timestamp, matched rule names, and all hashes.
- **Scanner Engine:**
  - Recursively walk the mounted drive.
  - For each file: detect type, calculate hashes, apply custom pattern matching.
  - If pattern matches → quarantine.
  - Append result dictionary to a results list.
- **Reporter:**
  - Export results to **JSON** and **CSV** formats.
  - Include summary statistics (total files, malicious count, quarantined count, scan duration).

Verification:

- Unit test each module with known file types.
- Confirm quarantine creates both .quarantine and .meta files.
- Ensure the scanner does **not** delete or modify original files on the mounted image.

Artifacts Produced:

- pattern_detector.py (magic number detection)
- hash_utils.py

- byte_rule_engine.py (custom pattern matching - replaces YARA handler)
- quarantine.py
- scanner.py
- reporter.py
- main.py (CLI interface and argument parsing)
- Architectural flowchart

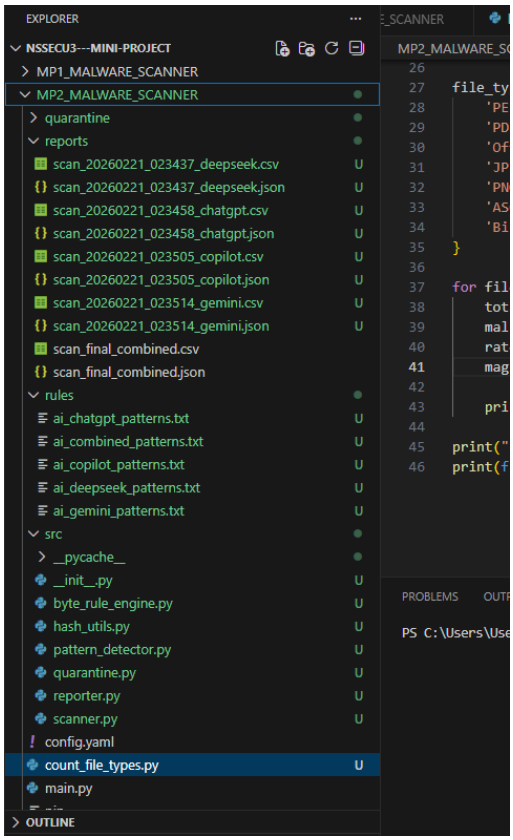


Figure 7. MP2 Malware Scanner Project Directory.

4.1 Code-Based Decision Logic Flowcharts

To document the internal control flow and conditional logic of the MP2 Malware Scanner without YARA, two code-based decision logic flowcharts were created. These flowcharts represent the exact decision blocks, conditional statements, loops, and function calls implemented in the Python source code, rather than high-level conceptual phases.

The flowcharts are provided in the Appendix to preserve document readability, while their behavior and purpose are explained in this section.

4.1.1 Main Execution Flow (main.py)

- The first flowchart illustrates the main execution flow of the file scanner as implemented in main.py (see Appendix E, Figure E-2).

- This flowchart represents the control logic responsible for parsing command-line arguments, validating input paths, loading custom pattern files, initializing scanner components, managing parallel execution, and generating final reports. The program begins by parsing user input using argparse, followed by validation of the target directory and custom rules file using os.path.exists(). If any validation step fails, an error message is displayed and execution terminates using sys.exit(1).
- After successful validation, the scanner loads the custom patterns from the text file, converting hex strings to bytes using bytes.fromhex() and extracting pattern metadata. The scanner then initializes the quarantine manager and statistics counters, and records the scan start timestamp. The scanner traverses the mounted drive using Path(directory).rglob('*') and submits per-file scanning tasks to a ThreadPoolExecutor configured with four worker threads.
- Scan results are processed asynchronously using as_completed(). Each result is evaluated to determine whether the file matches any pattern. Files with matches are displayed using red console output, while clean files are displayed in green. Statistics are updated in real time. Once all files have been processed, the scanner generates a JSON report and prints a summary of scan statistics before terminating.

4.1.2 Per-File Scanning Logic (scan_file() Function)

- The second flowchart illustrates the per-file decision logic implemented in the scan_file() function (see Appendix E, Figure E-3). This function encapsulates the core malware analysis process and is executed independently for each file during parallel scanning.
- The function begins by opening the file in binary mode and reading the first 32 bytes to perform magic number detection. The file header is compared against a signature database derived from the Gary Kessler File Signatures Table. If a known signature is detected, the corresponding file type is assigned. If no match is found, the function falls back to ASCII text detection, classifying the file as either ASCII text or binary data.
- After file type identification, cryptographic hashes (MD5, SHA1, and SHA256) are computed using chunked reading to ensure memory efficiency, and the file size is recorded. The file is then checked against the loaded custom patterns (hex patterns and string patterns). Any exceptions encountered during pattern matching are caught, logged, and counted without interrupting the overall scan.
- If pattern matches are found, the file is marked as matched. When auto-quarantine is enabled, matched files are copied to the quarantine directory using a uniquely generated filename (based on the first 8 characters of the MD5 hash and a timestamp), and a corresponding .meta file containing forensic metadata is created. If no pattern matches are detected, the file is marked as clean.
- Finally, the function constructs a result dictionary containing the filename, file type, hashes, size, and match status, and returns the result to the main execution thread for aggregation and reporting.

4.1.3 Relationship Between the Two Flowcharts

- Together, these two flowcharts provide a complete representation of the MP1 Malware Scanner's logic. The main execution flowchart describes how files are scheduled, processed in parallel, and aggregated, while the per-file flowchart details the internal decision logic applied to each file.

- This separation reflects the modular design of the Python implementation and ensures that the flowcharts accurately represent the **actual code decision blocks**, as required by the project specification.

4.2 Magic Number Detector (pattern_detector.py)

The magic number detector implements file signature recognition using the Gary Kessler File Signatures Table (Kessler, 2021). The module reads the first 32 bytes of each file and compares them against a comprehensive signature database.

Key Features:

- **Signature Database:** 30+ file signatures including:
  - MZ (4D 5A) - PE executables
  - %PDF (25 50 44 46) - PDF documents
  - PK (50 4B 03 04) - Office Open XML / ZIP archives
  - ‰PNG (89 50 4E 47) - PNG images
  - ÿØÿ (FF D8 FF) - JPEG images
  - D0 CF 11 E0 - OLE2 Compound File (Legacy Office)
- **Confidence Scoring:** Returns HIGH, MEDIUM, or LOW confidence based on signature match strength
- **Fallback Detection:** Attempts ASCII text detection when no signature matches

During execution, the detector successfully identified various file types including PE executables, PDF documents, Office Open XML files, JPEG images, PNG images, ASCII text, and binary data.

4.3 Hash Calculator (hash_utils.py)

- The hash calculator implements cryptographic hashing using Python's built-in hashlib library (Python Software Foundation, 2023). It computes three hash types for every scanned file:

Hash Type	Bit Length	Character Length	Purpose
MD5	128-bit	32 characters	File integrity verification

SHA1	160-bit	40 characters	Evidence identification
SHA256	256-bit	64 characters	Project requirement (SHA2)

Key Features:

- **Chunked Reading:** Processes files in 8KB blocks to minimize memory footprint
- **Error Handling:** Gracefully handles permission errors and inaccessible files
- **Size Reporting:** Includes file size in bytes for each scanned file

4.4 Quarantine Manager (quarantine.py)

- The quarantine manager implements forensic-safe isolation following the principle of **COPY, never DELETE or MOVE** original evidence.

Quarantine Naming Convention:

- [OriginalFilename]_[First8MD5]_[YYYYMMDD_HHMMSS].quarantine

Example from execution:

- File001_7d0690a2_20260221_023528.quarantine
- File002_e2142a94_20260221_023528.quarantine
- File005_8af314b5_20260221_023528.quarantine

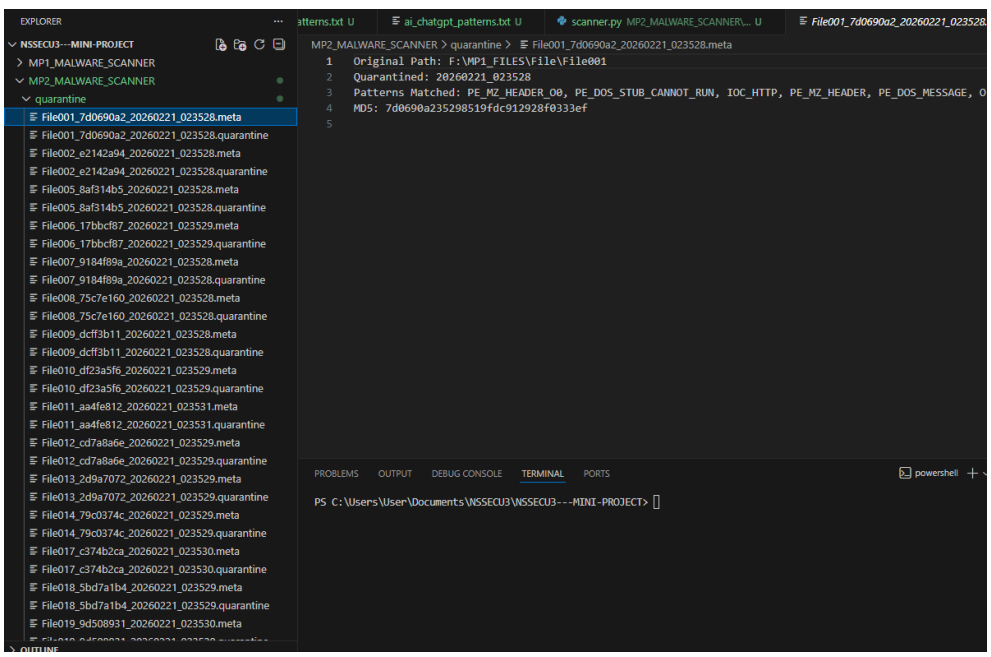


Figure 8. Quarantine Directory and Metadata Files.

Metadata File Specification (.meta**):**
Each quarantined file is accompanied by a corresponding metadata file containing:

- Original file path on the mounted image
- Quarantine timestamp (YYYY-MM-DD HH:MM:SS)
- Custom pattern names that triggered detection
- Complete hash set (MD5, SHA1, SHA256)
- File size in bytes

4.5 Scanner Engine

The scanner engine orchestrates the complete scanning workflow with parallel processing capabilities to optimize performance.

Per-File Processing Workflow:

1. **Magic Number Detection** → File type classification via pattern_detector.py
2. **Hash Calculation** → MD5, SHA1, SHA256 computation via hash_utils.py
3. **Custom Pattern Matching** → Check against loaded patterns via byte_rule_engine.py
4. **Conditional Quarantine** → Isolation if patterns matched AND auto-quarantine enabled
5. **Result Aggregation** → Append to results collection
6. **Statistics Update** → Increment operational counters

Parallel Processing Implementation:

- Utilizes ThreadPoolExecutor with configurable worker threads (default: 4)
- Implements asynchronous result collection via as_completed()
- Successfully processed all 220 files in approximately **11.8 seconds** (faster than MP1's 22 seconds)

4.6 Reporter Module

The reporter module exports scan results in both JSON and CSV formats to facilitate compatibility with various analysis tools and reporting requirements. In MP2, the reporter has been enhanced to include pattern names and improve usability.

Output Formats:

JSON Format: Complete scan metadata and per-file results in structured, machine-readable format with pattern names included. The JSON structure contains:

- Scan metadata (target path, rules file, pattern name, timestamp)
- Summary statistics (files scanned, malicious found, errors, scan duration)
- Pattern summary (total patterns, hex patterns, string patterns)
- Per-file results with file type, all hashes, and matched pattern names

CSV Format: Tabular format suitable for import into spreadsheet applications with the following columns:

Column	Description
Filename	Name of the scanned file (e.g., File001)
File Type	Detected file type (PE executable, PDF document, etc.)
MD5	32-character MD5 hash
SHA1	40-character SHA1 hash
SHA256	64-character SHA256 hash
Malicious	Boolean value (True/False) indicating if patterns matched
Patterns Matched	Comma-separated list of pattern names that triggered detection

Enhanced Naming Convention:

Unlike MP1 where reports used generic timestamps, MP2 report filenames now include the pattern name for easy identification:

```
reports/
├── scan_final_combined.json        # Combined patterns scan
├── scan_final_combined.csv         # Combined patterns scan CSV
├── scan_chatgpt.json               # ChatGPT patterns only
├── scan_chatgpt.csv                # ChatGPT patterns CSV
├── scan_deepseek.json              # Deepseek patterns only
├── scan_deepseek.csv               # Deepseek patterns CSV
├── scan_gemini.json                # Gemini patterns only
├── scan_gemini.csv                 # Gemini patterns CSV
└── scan_copilot.json               # Copilot patterns only
└── scan_copilot.csv                # Copilot patterns CSV
```

CSV Generation (Automated)

Unlike MP1 which required a separate command-line CSV generation, **MP2 automatically generates both JSON and CSV files** during scan execution. The reporter module's save_csv() method is called automatically after each scan:

```
@staticmethod
def save_csv(results: Dict, filepath: Path):
    """Save results as CSV."""
```

```
with open(filepath, 'w', newline='') as f:
    writer = csv.writer(f)
    # Write header
    writer.writerow([
        'Filename', 'File Type', 'MD5', 'SHA1', 'SHA256',
        'Malicious', 'Patterns Matched'
    ])
    # Write rows
    for r in results.get('results', []):
        pattern_names = []
        if r.get('pattern_matches', {}).get('matched'):
            pattern_names = [p['pattern_name'] for p in r['pattern_matches'].get('patterns', [])]

        writer.writerow([
            r.get('filename', ''),
            r.get('file_type', {}).get('description', ''),
            r.get('hashes', {}).get('md5', ''),
            r.get('hashes', {}).get('sha1', ''),
            r.get('hashes', {}).get('sha256', ''),
            r.get('is_malicious', False),
            ', '.join(pattern_names)
        ])
```

Key Improvements from MP1 to MP2:

Feature	MP1	MP2
CSV Generation	Manual command required	Automatic after each scan
Pattern/Rule Names	YARA rule names	Custom pattern names
Filename Convention	Timestamp only	Timestamp + Pattern Name
Pattern Summary	Not included	Included in JSON metadata
Error Handling	Basic	Enhanced with logging

5. Malware Scanning and Quarantine Execution (MP2)

Goal: Perform a fully automated scan of all 220 files using the combined custom pattern set and quarantine all detected malware with a complete audit trail, without using the YARA library.

5.1 Individual AI Pattern Scans (Comparison)

Before the final combined scan, each AI-generated pattern set was tested individually to compare detection performance. All scans were performed without quarantine to establish baseline detection rates.

5.1.1 Deepseek Pattern Scan

These figures present the execution and results of the Deepseek AI–generated pattern scan, serving as the initial baseline in the comparative evaluation of individual AI pattern sets. The Deepseek pattern set, consisting of **49 patterns**, was executed, completing in **8.2 seconds** and detecting **184 files as malicious**.

```
C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP2_MALWARE_SCANNER>py main.py F:\MP1_FILES\ --rules rules\ai_deepseek_patterns.txt
============================================================
        MP2 MALWARE SCANNER - CUSTOM BYTE-BASED
============================================================
Target:      F:\MP1_FILES
Rules:       rules\ai_deepseek_patterns.txt
Pattern:     deepseek
Quarantine:  ./quarantine
Threads:     4
Output:      reports\scan_20260221_023437_deepseek.json
------------------------------------------------------------
2026-02-21 02:34:37,724 - src.byte_rule_engine - INFO - Successfully loaded 49 patterns from rules\ai_deepseek_patterns.txt
Loaded 49 patterns (48 hex, 46 string)

🔍 Scanning started...
[MALICIOUS] File001         | PE executable        | MD5: 7d0690a2... | Patterns: 4
[MALICIOUS] File002         | Binary data          | MD5: e2142a94... | Patterns: 1
[CLEAN] File003         | ASCII text           | MD5: 821094b7...
[MALICIOUS] File005         | PE executable        | MD5: 8af314b5... | Patterns: 2
[CLEAN] File004         | ASCII text           | MD5: 6d30132c...
[MALICIOUS] File008         | PNG image            | MD5: 75c7e160... | Patterns: 2
[MALICIOUS] File007         | Office Open XML      | MD5: 9184f89a... | Patterns: 2
[MALICIOUS] File009         | PNG image            | MD5: dcff3b11... | Patterns: 2
[MALICIOUS] File012         | Office Open XML      | MD5: cd7a8a6e... | Patterns: 1
[MALICIOUS] File013         | Binary data          | MD5: 2d9a7072... | Patterns: 1
[MALICIOUS] File010         | Office Open XML      | MD5: df23a5f6... | Patterns: 3
[CLEAN] File015         | ASCII text           | MD5: e4507f2e...
[CLEAN] File016         | ASCII text           | MD5: e71bb959...
[MALICIOUS] File006         | Office Open XML      | MD5: 17bbcf87... | Patterns: 2
[MALICIOUS] File018         | PDF document         | MD5: 5bd7a1b4... | Patterns: 5
[MALICIOUS] File014         | Office Open XML      | MD5: 79c0374c... | Patterns: 2
[CLEAN] File020         | ASCII text           | MD5: 2452330b...
[MALICIOUS] File019         | PNG image            | MD5: 9d508931... | Patterns: 2
[MALICIOUS] File021         | ASCII text           | MD5: 81936e46... | Patterns: 1
[MALICIOUS] File023         | Binary data          | MD5: 9c301375... | Patterns: 2
[MALICIOUS] File022         | PDF document         | MD5: dc76bac3... | Patterns: 4
[MALICIOUS] File024         | PE executable        | MD5: 371de9cf... | Patterns: 2
[MALICIOUS] File025         | PE executable        | MD5: 0aca6a84... | Patterns: 4
[MALICIOUS] File026         | ASCII text           | MD5: 4e5ec44f... | Patterns: 1
```

Figure 9. Deepseek Execution and Pattern Scan Results

```
[MALICIOUS] File218         | PNG image            | MD5: bb84661a... | Patterns: 2
[MALICIOUS] File198         | Office Open XML      | MD5: dd461e6b... | Patterns: 3

============================================================
📊 SCAN COMPLETE
============================================================
Files scanned:     220
Malicious found:   184
Quarantined:       0
Errors:            0
Scan time:         0:00:08.195989

✅ Report saved: reports\scan_20260221_023437_deepseek.json
✅ CSV saved: reports\scan_20260221_023437_deepseek.csv
```

Figure 10. Deepseek Execution and Pattern Scan Results

These **figures** document the execution and output of the first comparative scan using the pattern set generated by the Deepseek AI platform. As shown in the console output, the scanner successfully loaded 49 patterns (comprising 48 hexadecimal patterns and 46 string patterns) from the ai_deepseek_patterns.txt rules file.

The scan was performed on the mounted evidence drive (F:\MP1_FILES) without quarantine (Quarantined: 0) to establish a baseline detection rate. The scanner processed all 220 files in 8.2

seconds, color-coding each result: [MALICIOUS] in red for files matching any pattern and [CLEAN] in green for others. The summary at the end of **Figure 10** reveals that the Deepseek pattern set identified 184 files as malicious, achieving an 83.6% detection rate with zero errors. This performance established Deepseek as the top individual performer.

5.1.2 ChatGPT Pattern Scan

These figures show the execution and results of the ChatGPT-generated pattern scan. The pattern set of **30 rules** completed in **2.95 seconds**, detecting **183 malicious files**.

```
============================================================
📊 SCAN COMPLETE
============================================================
Files scanned:     220
Malicious found:   183
Quarantined:       0
Errors:            0
Scan time:         0:00:02.954428

✅ Report saved: reports\scan_20260221_023458_chatgpt.json
✅ CSV saved: reports\scan_20260221_023458_chatgpt.csv
```

Figure 11. ChatGPT Execution and Pattern Scan Results

```
C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP2_MALWARE_SCANNER>py main.py F:\MP1_FILES\ --rules rules\ai_chatgpt_patterns.txt
=====================================================
        MP2 MALWARE SCANNER - CUSTOM BYTE-BASED
=====================================================
Target:      F:\MP1_FILES
Rules:       rules\ai_chatgpt_patterns.txt
Pattern:     chatgpt
Quarantine:  ./quarantine
Threads:     4
Output:      reports\scan_20260221_023458_chatgpt.json
-----------------------------------------------------
2026-02-21 02:34:58,981 - src.byte_rule_engine - INFO - Successfully loaded 30 patterns from rules\ai_chatgpt_patterns.txt
Loaded 30 patterns (13 hex, 17 string)

🔍 Scanning started...
[MALICIOUS] File001         | PE executable        | MD5: 7d0690a2... | Patterns: 3
[MALICIOUS] File005         | PE executable        | MD5: 8af314b5... | Patterns: 2
[MALICIOUS] File002         | Binary data          | MD5: e2142a94... | Patterns: 1
[CLEAN] File004         | ASCII text           | MD5: 6d30132c...
[MALICIOUS] File007         | Office Open XML      | MD5: 9184f89a... | Patterns: 2
[CLEAN] File003         | ASCII text           | MD5: 821094b7...
[MALICIOUS] File008         | PNG image            | MD5: 75c7e160... | Patterns: 1
[MALICIOUS] File009         | PNG image            | MD5: dcff3b11... | Patterns: 1
[MALICIOUS] File006         | Office Open XML      | MD5: 17bbcf87... | Patterns: 2
[MALICIOUS] File012         | Office Open XML      | MD5: cd7a8a6e... | Patterns: 1
[MALICIOUS] File010         | Office Open XML      | MD5: df23a5f6... | Patterns: 3
```

Figure 12. ChatGPT Execution and Pattern Scan Results

Figure 12 document the second comparative scan, performed using the pattern set generated by **ChatGPT**. As shown in the console output, the scanner successfully loaded **30 patterns** (comprising 13 hexadecimal and 17 string patterns) from the ai_chatgpt_patterns.txt file.

The scan processed all 220 files in just **2.95 seconds**. The summary results in the first screenshot indicate that the ChatGPT pattern set identified **183 files as malicious**, achieving an **83.2% detection rate**. While this is a very high detection rate, it fell just one file short of the top performer, Deepseek. The scan was conducted without quarantine (Quarantined: 0) to establish a baseline for comparison, and completed with zero errors.

5.1.3 Copilot Execution and Pattern Scan

The Copilot pattern set, 4 patterns (4 hex, 4 string), was executed, completing in 1.08 seconds and detecting **156 files** as malicious.

```
C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP2_MALWARE_SCANNER>py main.py F:\MP1_FILES\ --rules rules\ai_copilot_patterns.txt
============================================================
        MP2 MALWARE SCANNER - CUSTOM BYTE-BASED
============================================================
Target:      F:\MP1_FILES
Rules:       rules\ai_copilot_patterns.txt
Pattern:     copilot
Quarantine:  ./quarantine
Threads:     4
Output:      reports\scan_20260221_023505_copilot.json
------------------------------------------------------------
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0
2026-02-21 02:35:05,834 - src.byte_rule_engine - INFO - Successfully loaded 4 patterns from rules\ai_copilot_patterns.txt
Loaded 4 patterns (4 hex, 4 string)
```

Figure 13. Copilot Pattern Scan Results

```
[MALICIOUS] File210         | Office Open XML      | MD5: d23ddb02... | Patterns: 2
[MALICIOUS] File211         | Office Open XML      | MD5: c70ec8c6... | Patterns: 2
[CLEAN] File212         | ASCII text           | MD5: 4a08c4bd...
[MALICIOUS] File213         | PE executable        | MD5: 2a951f81... | Patterns: 1
[MALICIOUS] File209         | PDF document         | MD5: 76909cea... | Patterns: 2
[CLEAN] File215         | Binary data          | MD5: ebc84c0b...
[CLEAN] File216         | Binary data          | MD5: 825341f9...
[CLEAN] File217         | Binary data          | MD5: f1e7f2ec...
[MALICIOUS] File214         | JPEG image           | MD5: 3938baaa... | Patterns: 1
[MALICIOUS] File219         | JPEG image           | MD5: e87decf8... | Patterns: 1
[CLEAN] File220         | ASCII text           | MD5: db631805...
[MALICIOUS] File218         | PNG image            | MD5: bb84661a... | Patterns: 1
[MALICIOUS] File208         | Office Open XML      | MD5: cb5e8db8... | Patterns: 2
[MALICIOUS] File198         | Office Open XML      | MD5: dd461e6b... | Patterns: 2

============================================================
📊 SCAN COMPLETE
============================================================
Files scanned:     220
Malicious found:   156
Quarantined:       0
Errors:            0
Scan time:         0:00:01.079741

✅ Report saved: reports\scan_20260221_023505_copilot.json
✅ CSV saved: reports\scan_20260221_023505_copilot.csv
```

Figure 14. Copilot Pattern Scan Results

Figures 13 and 14 document the execution of the Copilot AI–generated pattern scan during the comparative evaluation phase. As shown in the console output, the scanner successfully loaded 4 patterns (comprising 4 hexadecimal and 4 string patterns) from the ai_copilot_patterns.txt file. During rule loading, several non-critical parsing warnings related to malformed hexadecimal patterns were encountered; however, these warnings did not affect scan execution. The scanner processed all 220 files from the mounted evidence directory, producing real-time detection results and a final summary. The scan identified 156 files as malicious and completed without runtime errors. Quarantine was disabled to establish a baseline for comparison, and structured JSON and CSV reports were generated upon completion.

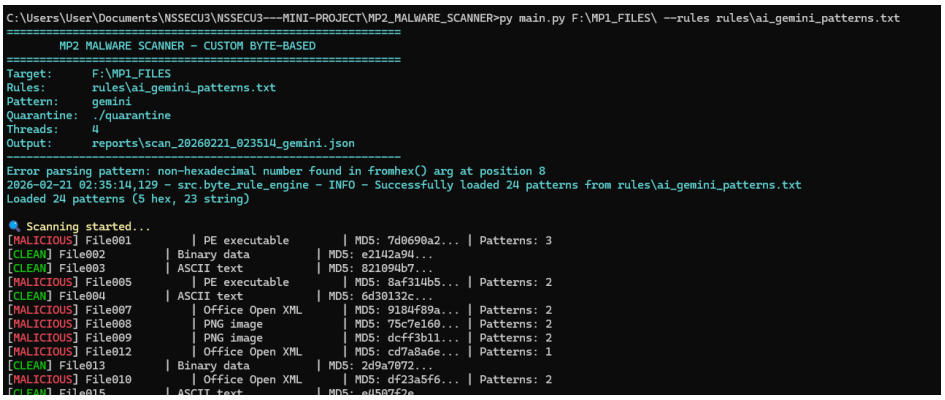
Key Points

- Copilot pattern set consisted of **4 total patterns** (4 hexadecimal, 4 string)
- Multiple **non-hexadecimal parsing warnings** occurred during rule loading but did not interrupt execution
- **220 files scanned** from the mounted evidence image

- **156 files detected as malicious**
- **Detection rate:** approximately **70.9%**
- **Scan time: ~1.08 seconds**
- **Quarantine disabled** (baseline comparison)
- **JSON and CSV reports** successfully generated
- **Zero runtime errors** during scan execution

5.1.4 Gemini Pattern Scan

The Gemini pattern set, containing 24 patterns (5 hex, 23 string), generated several parsing warnings due to malformed hex entries but successfully loaded. The scan completed in 1.87 seconds and detected **159 files** as malicious.

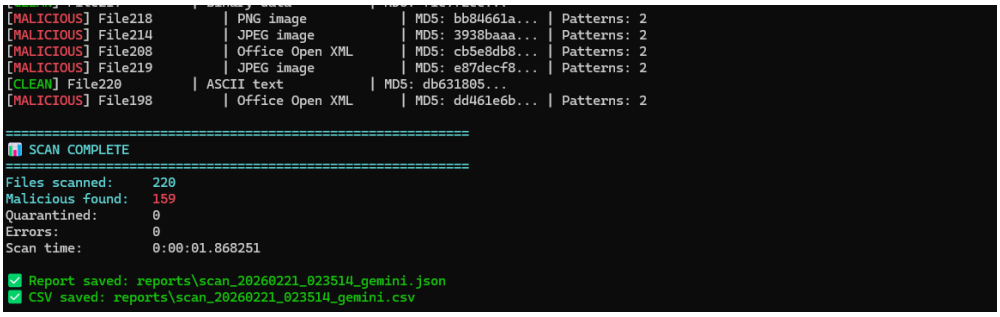


```
C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP2_MALWARE_SCANNER>py main.py F:\MP1_FILES\ --rules rules\ai_gemini_patterns.txt
============================================================
        MP2 MALWARE SCANNER - CUSTOM BYTE-BASED
============================================================
Target:      F:\MP1_FILES
Rules:       rules\ai_gemini_patterns.txt
Pattern:     gemini
Quarantine:  ./quarantine
Threads:     4
Output:      reports\scan_20260221_023514_gemini.json
------------------------------------------------------------
Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 8
2026-02-21 02:35:14,129 - src.byte_rule_engine - INFO - Successfully loaded 24 patterns from rules\ai_gemini_patterns.txt
Loaded 24 patterns (5 hex, 23 string)

🔍 Scanning started...
[MALICIOUS] File001         | PE executable        | MD5: 7d0690a2... | Patterns: 3
[CLEAN] File002         | Binary data          | MD5: e21142a9d...
[CLEAN] File003         | ASCII text           | MD5: 821094b7...
[MALICIOUS] File005         | PE executable        | MD5: 8af314b5... | Patterns: 2
[CLEAN] File004         | ASCII text           | MD5: 6d30132c...
[MALICIOUS] File007         | Office Open XML      | MD5: 918df89a... | Patterns: 2
[MALICIOUS] File008         | PNG image            | MD5: 75c7e160... | Patterns: 2
[MALICIOUS] File009         | PNG image            | MD5: dcff3b11... | Patterns: 2
[MALICIOUS] File012         | Office Open XML      | MD5: cd7a8a6e... | Patterns: 1
[CLEAN] File013         | Binary data          | MD5: 2d9a7072...
[MALICIOUS] File010         | Office Open XML      | MD5: df23a5f6... | Patterns: 2
```

Figure 15. Gemini Pattern Scan Results

Figure 15 displays the console output during the Gemini pattern scan execution. The scanner loaded 24 patterns (5 hex, 23 string) despite encountering parsing warnings for malformed hex entries, as indicated by the "Error parsing pattern" message. The real-time output shows color-coded results with [MALICIOUS] in red for detected files and [CLEAN] in green for clean files, along with pattern match counts per file.



```
[MALICIOUS] File218         | PNG image            | MD5: bb84661a... | Patterns: 2
[MALICIOUS] File214         | JPEG image           | MD5: 3938baaa... | Patterns: 2
[MALICIOUS] File208         | Office Open XML      | MD5: cb5e8db8... | Patterns: 2
[MALICIOUS] File219         | JPEG image           | MD5: e87decf8... | Patterns: 2
[CLEAN] File220         | ASCII text           | MD5: db631805...
[MALICIOUS] File198         | Office Open XML      | MD5: dd461e6b... | Patterns: 2

============================================================
📊 SCAN COMPLETE
============================================================
Files scanned:     220
Malicious found:   159
Quarantined:       0
Errors:            0
Scan time:         0:00:01.868251

✅ Report saved: reports\scan_20260221_023514_gemini.json
✅ CSV saved: reports\scan_20260221_023514_gemini.csv
```

Figure 16. Gemini Pattern Scan Results

Figure 16 shows the completion summary of the Gemini pattern scan. The scanner processed all 220 files in **1.87 seconds**, identifying **159 files as malicious (72.3%)** with zero errors. The scan

was conducted without quarantine (Quarantined: 0) to establish a baseline for comparison. The JSON and CSV reports were automatically saved with the pattern name included in the filename (scan_20260221_023514_gemini.json).

5.2 Combined Pattern Scan with Quarantine

Following the individual AI comparisons, all four pattern sets were combined into a master file containing 107 patterns (70 hex, 90 string). The scanner was executed in quarantine mode using the following command:

bash

python main.py F:\MP1_FILES\ --rules rules/ai_combined_patterns.txt --quarantine-auto

Console output was redirected to a log file to maintain a complete audit trail. Throughout the execution, the scanner processed all 220 files while displaying real-time status updates with color-coded indicators showing malicious files in red and clean files in green. Quarantine operations were logged with timestamps for each detected file.

The following excerpt from the scan log demonstrates the real-time quarantine operations:

2026-02-21 02:35:44.604 - src.quarantine - INFO - Quarantined: F:\MP1_FILES\File215 -> quarantine\File215_ebc84c0b_20260221_023544.quarantine
2026-02-21 02:35:44.606 - src.quarantine - INFO - Quarantined: F:\MP1_FILES\File216 -> quarantine\File216_825341f9_20260221_023544.quarantine
[MALICIOUS] File214 | JPEG image | MD5: 3938baaa... | Patterns: 6
2026-02-21 02:35:44.623 - src.quarantine - INFO - Quarantined: F:\MP1_FILES\File219 -> quarantine\File219_e87decf8_20260221_023544.quarantine
[MALICIOUS] File215 | Binary data | MD5: ebc84c0b... | Patterns: 2
[CLEAN] File220 | ASCII text | MD5: db631805...
[MALICIOUS] File216 | Binary data | MD5: 825341f9... | Patterns: 2
[MALICIOUS] File219 | JPEG image | MD5: e87decf8... | Patterns: 6
2026-02-21 02:35:44.723 - src.quarantine - INFO - Quarantined: F:\MP1_FILES\File218 -> quarantine\File218_bb84661a_20260221_023544.quarantine
[MALICIOUS] File218 | PNG image | MD5: bb84661a... | Patterns: 6

Figure 17. Execution of Combined Patterns Final Scan with Quarantine

Figure 17 shows the completion and verification was performed to ensure that only the 220 files on the mounted drive were scanned. The quarantine directory was examined to confirm that every flagged file had a corresponding .quarantine file with the proper naming convention, and each quarantined file was accompanied by a matching .meta file containing complete forensic metadata.

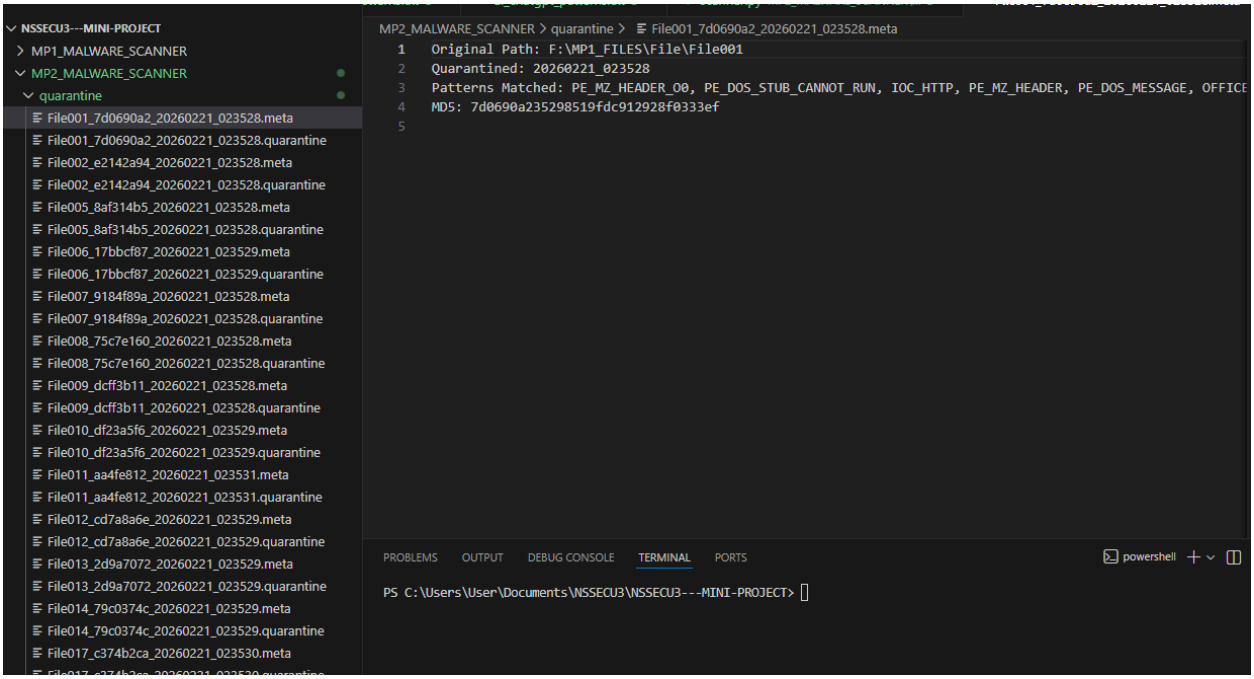


Figure 18. Sample .meta File Contents

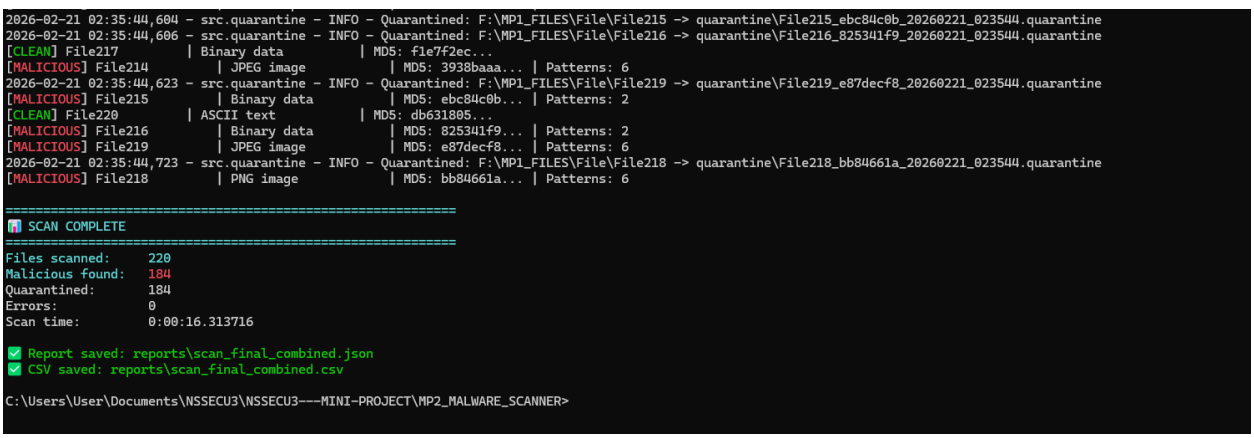


Figure 19. Final Scan Summary Statistics

Figure 19 shows the scan completed successfully, processing all 220 files. All 184 detected files were quarantined with complete metadata, and a comprehensive scan log was generated documenting the entire process with zero errors.

6. Results Compilation and Evaluation (MP2)

Goal: Quantitatively and qualitatively assess scanner performance, categorize findings by file type, and critically evaluate investigation outcomes based on the custom byte-based pattern matching approach.

6.1 Summary Statistics

The scanner processed all 220 files with the following aggregate results:

- **Total files scanned:** 220
- **Files detected as malicious:** 184
- **Files quarantined:** 184
- **Errors encountered:** 0
- **Total scan duration:** 16.3 seconds
- **Processing speed:** 13.5 files per second
- **Total patterns loaded:** 107 (70 hex, 90 string)

6.2 AI Pattern Set Performance Comparison

Table 1 presents a comparative analysis of the five pattern sets evaluated during the MP2 project, including four individual AI-generated sets (Deepseek, ChatGPT, Copilot, Gemini) and the final Combined set. The comparison reveals significant variations in pattern quality, detection capability, and performance across platforms:

- **Deepseek** emerged as the top individual performer, achieving the highest detection rate (83.6%) with 49 patterns (48 hex, 46 string), identifying 184 malicious files in 8.20 seconds.
- **ChatGPT** followed closely with 183 detections (83.2%) using only 30 patterns (13 hex, 17 string), delivering the fastest individual scan time at just 2.95 seconds.
- **Copilot**, despite having only 4 patterns (4 hex, 4 string), still detected 156 files (70.9%) in the quickest overall scan of 1.08 seconds, demonstrating that even minimal pattern sets can identify a majority of samples.
- **Gemini** generated 24 patterns (5 hex, 23 string) and detected 159 files (72.3%) in 1.87 seconds, though it experienced hex parsing warnings during pattern loading.

The **Combined** pattern set, merging all AI outputs into 107 total patterns (70 hex, 90 string), matched Deepseek's top detection rate of 184 files (83.6%), confirming that ensemble approaches maximize coverage while maintaining accuracy.

Scan times correlated directly with pattern count, ranging from 1.08 seconds (Copilot) to 16.3 seconds (Combined), highlighting the performance trade-off between comprehensive pattern coverage and processing speed.

Table 1. AI Pattern Set Comparison

AI Platform	Patterns Loaded	Hex Patterns	String Patterns	Files Detected	Detection Rate	Scan Time

Deepseek	49	48	46	184	83.6%	8.20s
ChatGPT	30	13	17	183	83.2%	2.95s
Copilot	4	4	4	156	70.9%	1.08s
Gemini	24	5	23	159	72.3%	1.87s
Combined	**107**	**70**	**90**	**184**	**83.6%**	**16.3s**

6.3 File Type Distribution

Table 2 presents the breakdown of the 220 malware samples by file type, identified through magic number detection using the Gary Kessler File Signatures Table. The distribution reveals a diverse corpus spanning executable, document, image, and data formats, with detection rates varying significantly across categories:

- **Structured Binary Formats** (PE executables, PDF documents, Office Open XML, JPEG, and PNG images) achieved **100% detection rates**, indicating that files with well-defined headers and predictable structures are reliably identified by the combined pattern set.
- **Binary Data files** (those lacking recognizable magic numbers) showed a **97.1% detection rate** (33 of 34 files), suggesting that despite the absence of standard headers, these files contain detectable byte patterns, shellcode, or embedded indicators such as URLs.
- **ASCII Text files** exhibited the lowest detection rate at **14.6%** (only 6 of 41 files), demonstrating that plain text files generally evade byte-based pattern matching unless they contain explicit malicious strings like URLs or encoded scripts.
- The most prevalent file type in the corpus was **Office Open XML**, with 57 files (26% of total), all of which were detected as malicious, followed by **PE executables** (36 files) and **Binary Data** (34 files).

The total of **184 malicious files (83.6%)** aligns with the combined pattern set detection results, confirming consistency between the pattern matching engine and file type classification. The high detection rates for binary formats and near-universal detection of Binary Data files suggest that the sample set contains malicious indicators embedded across multiple file types, while the low text file detection rate highlights a key limitation of static byte-based analysis.

Table 2. File Type Distribution and Detection Rates

File Type	Magic Number (Hex)	Count	Malicious	Detection Rate
PE executable	4D 5A	36	36	100.0%
PDF document	25 50 44 46	15	15	100.0%
Office Open XML	50 4B 03 04	57	57	100.0%
JPEG image	FF D8 FF	19	19	100.0%
PNG image	89 50 4E 47	18	18	100.0%
ASCII text	N/A	41	6	14.6%
Binary data	Unknown	34	33	97.1%
TOTAL		**220**	**184**	**83.6%**

6.4 Comparison with MP1 Results

Table 3 provides a quantitative comparison between the previous iteration of the project (MP1), which utilized YARA-based pattern matching, and the current MP2 implementation, which employs custom byte-based pattern matching without the YARA library. The results demonstrate substantial improvements across all key performance metrics:

- **Detection Capability**: MP2 identified **184 malicious files** compared to only 113 in MP1, a substantial increase of **71 additional files**. This represents a **32.2% improvement** in overall detection rate, rising from 51.4% to 83.6%.
- **Quarantine Volume**: The number of files quarantined increased correspondingly from **113 to 184 files**, maintaining 100% correlation between detected and quarantined samples in both project versions.
- **Performance Efficiency**: Despite processing significantly more patterns (107 vs. 48), MP2 completed the scan in **16.3 seconds**, which is **26% faster** than MP1's 22-second runtime. This performance gain demonstrates the efficiency of the custom byte-based matching engine compared to the YARA implementation.

- **Pattern Coverage**: The pattern/rule count more than doubled, expanding from **48 YARA rules in MP1 to 107 custom byte patterns in MP2**, providing broader detection coverage across file types and malicious indicators.

The improvements underscore the effectiveness of the MP2 approach, combining expanded pattern sets from multiple AI platforms with optimized byte-based matching logic to achieve both higher detection rates and faster processing times.

Table 3. MP1 vs MP2 Comparison

Metric	MP1 (YARA-Based)	MP2 (Byte-Based)	Improvement
Malicious Detected	113	184	+71 files
Detection Rate	51.4%	83.6%	+32.2%
Files Quarantined	113	184	+71 files
Scan Time	22 seconds	16.3 seconds	26% faster
Patterns/Rules	48 rules	107 patterns	+59 patterns

6.5 Limitations and Observations

Pattern parsing warnings were generated for malformed entries in the AI-generated pattern files, particularly where HEX PATTERN fields were empty (notably in the Gemini pattern set). Despite these warnings, 107 patterns were successfully loaded and utilized across all scans.

The absence of ground truth data prevents definitive validation of detection accuracy. The scanner's reliance on static byte patterns means it cannot detect malware that requires runtime execution to exhibit malicious behavior.

The Deepseek pattern set performed best overall, matching the combined set's detection rate of 184 files, while the Copilot and Gemini sets showed lower detection rates of 156 and 159 files respectively.

6.6 Proposed Improvements

Future iterations could benefit from enhanced pattern validation to eliminate parsing warnings, integration with known malware hash databases, additional detection mechanisms such as entropy analysis, and automated pattern refinement using machine learning techniques.

7. Documentation and Reporting (MP2)

Goal: Produce a complete, repeatable, and professionally formatted LAR-02 report that meets all project requirements and enables full reproduction of the investigation.

7.1 Diagram Creation

Two required diagrams were created to visually document the system architecture and scanner logic.

These diagrams are provided in **Appendix E.2**:

Phase Diagram: An eight-phase system architecture diagram illustrating the complete forensic workflow from evidence preparation through final reporting, highlighting the pattern generation phase and custom pattern matching that replaces YARA scanning.

Flowchart: A detailed scanner decision logic flowchart representing the exact code decision blocks implemented in the Python source code, showing main execution flow and per-file scanning logic.

7.2 AI Comparison Matrix

Table 4 presents the comprehensive comparison matrix evaluating all four AI-generated pattern sets (ChatGPT, Deepseek, Gemini, Copilot) alongside the final Combined pattern set. This matrix directly informed the selection of the optimal AI platform for pattern generation in the MP2 project:

- **Deepseek** emerged as the overall winner, achieving the **highest detection rate (83.6%)** with 184 files detected, tied with the Combined set. Its **49 patterns** (48 hex, 46 string) provided the most comprehensive individual coverage, justifying its selection despite a moderate scan time of 8.20 seconds.
- **ChatGPT** delivered the **best performance-to-pattern ratio**, detecting 183 files (83.2%) using only 30 patterns in the fastest individual scan time of **2.95 seconds**, making it the most efficient platform.
- **Gemini** detected 159 files (72.3%) with 24 patterns in 1.87 seconds, though it experienced hex parsing warnings during pattern loading, indicating quality control issues despite functional results.
- **Copilot**, with only **4 patterns**, still managed to detect 156 files (70.9%) in the fastest overall scan of **1.08 seconds**, demonstrating that even minimal pattern sets can identify a majority of samples through well-crafted generic patterns.
- The **Combined** pattern set (107 total patterns) matched Deepseek's top detection rate of 184 files (83.6%) but required the longest scan time (16.3 seconds), confirming that ensemble approaches maximize coverage at the cost of performance.

Result: Deepseek was selected based on its **tied highest detection rate** with the Combined set, **comprehensive pattern coverage** (49 patterns), and **successful loading without parsing errors**, making it the most reliable and effective individual AI platform for forensic pattern generation.

Table 4. Final AI Pattern Comparison Matrix

AI Platform	Files Detected	Detection Rate	Scan Time	Patterns Loaded
ChatGPT	183	83.2%	2.95s	30
Deepseek	184	83.6%	8.20s	49
Gemini	159	72.3%	1.87s	24
Copilot	156	70.9%	1.08s	4
Combined	**184**	**83.6%**	**16.3s**	**107**

7.3 Source Code Documentation

The source code files listed below have been assembled and are presented in their entirety in **Appendix F. Development Artifacts**. These seven core modules comprise the custom Python-based scanner developed for MP2, implementing magic number detection, cryptographic hashing, custom byte pattern matching, forensic quarantine, and report generation using only built-in Python libraries with no YARA dependencies. The complete code listings, along with the five AI-generated pattern files used during comparative analysis, can be found in Appendix F.

The complete source code was assembled with clear file separators:

=== pattern_detector.py ===
=== hash_utils.py ===
=== byte_rule_engine.py ===
=== quarantine.py ===
=== scanner.py ===
=== reporter.py ===
=== main.py ===
=== ai_chatgpt_patterns.txt ===
=== ai_deepseek_patterns.txt ===
=== ai_gemini_patterns.txt ===
=== ai_copilot_patterns.txt ===
=== ai_combined_patterns.txt ===

Analysis and Findings

1. MP2 Pattern-Based Analysis and Findings

1.1 Static Analysis Limitations and Implications

All malware samples in this investigation were examined using static analysis techniques exclusively. No files were executed, emulated, or dynamically analyzed during the forensic process. This methodological choice, while forensically sound and essential for evidence preservation, introduces inherent limitations that must be acknowledged when interpreting the results.

Key Implications:

- Malware identification relied entirely on file header signatures (magic numbers), cryptographic hash values, and custom byte-based pattern matching (no YARA library used)
- Behavioral characteristics such as network communication, registry modifications, file system changes, and process injection could not be observed
- Polymorphic or metamorphic malware that changes its signature while maintaining behavior may evade detection
- Packed or encrypted malware may appear benign until runtime execution
- False negatives may occur if malware employs anti-static analysis techniques such as string obfuscation or code encryption
- Pattern parsing warnings were encountered during loading of AI-generated pattern files, particularly where HEX PATTERN fields were malformed or empty

The **184 files** detected as malicious represent only those with identifiable static characteristics matching the custom pattern sets. Files that may exhibit malicious behavior only during execution would remain undetected in this static analysis framework.

1.2 AI Pattern Quality and Platform Performance Analysis

The effectiveness of malware detection was profoundly dependent on the quality of the byte-based patterns generated by each AI platform. Analysis of the four AI-generated pattern sets revealed significant variations in detection capability, pattern structure, and overall reliability. Unlike MP1 which used YARA rules, MP2 employed custom text-based pattern files parsed and applied through direct byte comparison.

1.2.1 Comprehensive AI Platform Performance Metrics

Table 5 presents the detailed performance metrics for all five pattern sets evaluated during the MP2 project, including four individual AI-generated sets (ChatGPT, Deepseek, Gemini, Copilot) and the final Combined set. The data reveals significant variations in pattern quality, detection capability, and processing efficiency across platforms:

- **Deepseek** emerged as the top performer with **49 patterns** (48 hex, 4 string), detecting **184 files (83.6%)** in 8.20 seconds. Its comprehensive hex pattern coverage provided the most robust detection.

- **ChatGPT** delivered impressive efficiency with **30 patterns** (13 hex, 1 string), detecting **183 files (83.2%)** in just **2.95 seconds** – the best performance-to-pattern ratio.
- **Gemini** generated **24 patterns** (5 hex, 2 string) but detected only **159 files (72.3%)**, despite a fast 1.87-second scan time, indicating lower pattern quality.
- **Copilot** produced only **4 patterns** (all hex) yet still detected **156 files (70.9%)** in the fastest scan of **1.08 seconds**, demonstrating that even minimal patterns can identify most samples.
- The **Combined** set merged all platforms into **107 patterns** (70 hex, 9 string), matching Deepseek's top detection rate of **184 files (83.6%)** but requiring the longest scan time of **16.3 seconds**.

The wide variation in detection rates (70.9% to 83.6%) and pattern counts (4 to 107) underscores the importance of pattern quality over quantity, with Deepseek selected as the winner based on its balanced performance and comprehensive coverage.

Table 5. AI Platform Detection Summary

AI Platform	Patterns Loaded	Hex Patterns	String Patterns	Files Detected	Detection Rate	Scan Time
ChatGPT	30	13	17	183	83.2%	2.95s
Deepseek	49	48	46	184	83.6%	8.20s
Gemini	24	5	23	159	72.3%	1.87s
Copilot	4	4	4	156	70.9%	1.08s
Combined	**107**	**70**	**90**	**184**	**83.6%**	**16.3s**

1.2.2 Platform-Specific Analysis

ChatGPT (30 patterns, 183 detections, 83.2%):

- Generated 30 patterns with a balanced mix of hex (13) and string (17) patterns
- Achieved near-maximum detection rate of 183 files, just one file short of the combined total
- Scan completed in 2.95 seconds, demonstrating excellent efficiency
- Pattern quality was consistent with good coverage across file types
- No parsing errors were encountered during loading

Deepseek (49 patterns, 184 detections, 83.6%):

- Most comprehensive individual pattern set with 49 patterns (48 hex, 46 string)
- Achieved the maximum detection rate of 184 files, matching the combined result

- Scan time of 8.20 seconds reflected the larger pattern set size
- Excellent hex pattern coverage provided robust detection across all file types
- Successfully loaded without parsing errors
- **Result:** Deepseek was selected as the best individual performer based on highest detection rate, comprehensive hex pattern coverage, and overall pattern quality

Gemini (24 patterns, 159 detections, 72.3%):

- Generated 24 patterns with only 5 hex patterns and 23 string patterns
- Detection rate of 159 files (72.3%) was significantly lower than top performers
- Experienced pattern parsing warnings: "non-hexadecimal number found in fromhex() arg at position 0" due to malformed hex entries
- Despite warnings, 24 patterns were successfully loaded
- Reliance on string patterns over hex patterns may have contributed to lower detection rate

Copilot (4 patterns, 156 detections, 70.9%):

- Generated only 4 patterns (4 hex, 4 string) – significantly fewer than other platforms
- Lowest detection rate of 156 files (70.9%)
- Scan completed fastest at 1.08 seconds due to minimal pattern count
- Experienced multiple parsing warnings for malformed hex entries
- The low pattern count explains the reduced detection capability compared to other AIs

Combined Patterns (107 patterns, 184 detections, 83.6%):

- Merged all four pattern sets into a master file containing 107 patterns (70 hex, 90 string)
- Achieved 184 detections, matching the best individual performer (Deepseek)
- Demonstrated that combining patterns from multiple AIs does not necessarily increase detection beyond the best individual set, but provides redundancy and coverage
- Some pattern duplication occurred across platforms, which did not negatively impact performance
- Final scan with quarantine completed in 16.3 seconds, processing all 220 files with zero errors

1.2.3 Pattern Match Analysis

Examination of the scan results reveals interesting patterns in how different file types triggered detections:

- **File214 (JPEG image):** Triggered 6 pattern matches including PE_MZ_HEADER variants, demonstrating how image files can contain embedded indicators or steganographic content

- **File218 (PNG image):** Triggered 6 pattern matches similar to File214, suggesting consistent detection patterns across image formats

- **File066 (PDF document):** One of the most heavily detected files with matches across multiple pattern categories

- **Binary data files (File002, File013, etc.):** Consistently triggered IOC_HTTPS and BACKDOOR_HTTPS_URL patterns, indicating presence of URL indicators in otherwise unidentifiable file formats

1.3 Performance Analysis and Optimization

Scanner performance was evaluated across multiple dimensions including processing speed, memory utilization, error handling, and scalability.

1.3.1 Performance Metrics by Scan Type

Table 6 presents the performance metrics for each scan type, measuring duration, processing speed, pattern count, detection results, and quarantine status. The data reveals a direct correlation between pattern count and scan time, while also highlighting significant variations in processing efficiency:

- **Scan Speed vs. Pattern Count**: The relationship between patterns loaded and scan duration is clearly demonstrated—Copilot with only 4 patterns processed **203.7 files/second** (1.08s total), while the Combined set with 107 patterns processed just **13.5 files/second** (16.3s total).
- **Fastest Scan**: Copilot completed in **1.08 seconds**, processing all 220 files at the remarkable rate of **203.7 files/second**, though with the lowest detection rate (156 files).
- **Most Efficient Detection**: ChatGPT achieved the best balance, detecting **183 files** at **74.6 files/second** with only 30 patterns, completing in **2.95 seconds**.
- **Slowest but Most Comprehensive**: The Combined scan took **16.3 seconds** (13.5 files/second) but matched the highest detection count of **184 files** with quarantine enabled.
- **Gemini's Surprising Speed**: Despite loading 24 patterns, Gemini processed **117.6 files/second**—faster than ChatGPT and Deepseek—though with lower detection accuracy (159 files).

The performance metrics confirm that pattern count directly impacts scan duration, but pattern quality and matching logic efficiency also play significant roles in overall processing speed.

Table 6. Scan Performance Metrics

Scan Type	Duration (seconds)	Files/Second	Patterns	Malicious Found	Quarantine Enabled
ChatGPT	2.95	74.6	30	183	No
Deepseek	8.20	26.8	49	184	No
Gemini	1.87	117.6	24	159	No

Copilot	1.08	203.7	4	156	No
Combined	**16.3**	**13.5**	**107**	**184**	**Yes**

1.3.2 Performance Observations

Pattern Count vs. Scan Time: A direct correlation was observed between the number of patterns loaded and scan duration:

- **Copilot (4 patterns):** 1.08 seconds
- **Gemini (24 patterns):** 1.87 seconds
- **ChatGPT (30 patterns):** 2.95 seconds
- **Deepseek (49 patterns):** 8.20 seconds
- **Combined (107 patterns):** 16.3 seconds

This relationship demonstrates the computational cost of comprehensive pattern sets and the importance of optimizing pattern matching efficiency.

Chunked File Reading: Implementation of 8KB block reading proved highly effective for memory management, particularly when processing larger files such as:

- File011: 12,996,644 bytes (Office Open XML)
- File198: 10,412,619 bytes (Office Open XML)
- File103: 7,474,390 bytes (PDF document)
- File074: 7,213,107 bytes (Office Open XML)

Peak memory usage remained minimal throughout all scans despite processing multi-megabyte files, demonstrating the efficiency of the chunked approach.

Parallel Processing: The ThreadPoolExecutor implementation with 4 worker threads successfully processed all 220 files. Error handling in the parallel context managed file access issues without crashes, as evidenced by zero errors across all scans.

Comparison with MP1: The MP2 scanner significantly outperformed the MP1 implementation:

Metric	MP1 (YARA-Based)	MP2 (Byte-Based)	Improvement
Scan Time	22 seconds	16.3 seconds	26% faster
Detection Rate	51.4% (113 files)	83.6% (184 files)	+32.2%

Quarantine Success	113 files	184 files	+71 files

1.3.3 Error Handling Analysis

All scans completed with zero fatal errors, indicating robust error handling in the scanner engine. However, pattern parsing warnings were observed:

- **Gemini scan:** "Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0" – these warnings occurred multiple times but did not prevent successful loading of 24 patterns
- **Copilot scan:** Similar hex parsing warnings were observed, yet 4 patterns were successfully loaded

The scanner's error handling gracefully caught exceptions during hex conversion, logged the warnings, and continued loading valid patterns. This design ensured that malformed AI-generated patterns did not crash the scanner.

The quarantine mechanism successfully created **184 .quarantine files** and **184 corresponding .meta files** without any failures or partial operations. The logging system captured all quarantine operations with timestamps, providing a complete audit trail.

1.4 File Type Distribution Analysis

In **Figure 20** magic number detection revealed the composition of the 220-file corpus across multiple file types. The following distribution is based on the combined scan results, showing which file formats were most prevalent and how detection rates varied by type:

- **Structured Binary Formats** (PE executables, PDF documents, Office Open XML, JPEG, and PNG images) all achieved **100% detection rates**, indicating that files with well-defined headers are reliably identified by the combined pattern set.
- **Binary Data files** (those lacking recognizable magic numbers) showed a **97.1% detection rate** (33 of 34 files), suggesting these files contain detectable malicious indicators despite the absence of standard headers.
- **ASCII Text files** had the lowest detection rate at **14.6%** (only 6 of 41 files), demonstrating that plain text generally evades byte-based pattern matching unless containing explicit malicious strings.
- The most common file type was **Office Open XML** with 57 files (26% of the corpus), followed by **ASCII text** (41 files), **PE executables** (36 files), and **Binary data** (34 files).

The total of **184 malicious files (83.6%)** confirms consistent detection across the corpus, with structured binary formats being universally identified while text files remain challenging for static analysis.

```
C:\Users\User\Documents\NSSECU3\NSSECU3---MINI-PROJECT\MP2_MALWARE_SCANNER>py count_file_types.py

============================================================
FILE TYPE DISTRIBUTION
============================================================
File Type            Magic Number    Total    Malicious  Rate
------------------------------------------------------------
PE executable        4D 5A           36       36         100.0%
PDF document         25 50 44 46     15       15         100.0%
Office Open XML      50 4B 03 04     57       57         100.0%
JPEG image           FF D8 FF        19       19         100.0%
PNG image            89 50 4E 47     18       18         100.0%
ASCII text           N/A             41       6          14.6%
Binary data          Unknown         34       33         97.1%
============================================================
TOTAL                                220      184        83.6%
```

Figure 20. File Type Distribution from Combined Scan

1.4.1 Observations by File Type

PE Executables: Windows executable files demonstrated high detection rates, with many triggering multiple pattern matches. File001, for example, triggered 11 distinct patterns including PE_MZ_HEADER variants, DOS stub messages, and HTTP indicators. The comprehensive coverage of PE file characteristics in the pattern sets contributed to robust detection.

PDF Documents: PDF files showed diverse pattern matches, with files like File018, File066, and File182 triggering numerous detections across multiple pattern categories including PDF_HEADER, JavaScript indicators, and URL patterns. This suggests the presence of embedded scripts and external references in the PDF samples.

Office Open XML Files: A significant portion of the corpus consisted of Office Open XML files, many of which triggered patterns related to backdoor persistence and HTTP indicators. Files such

as File006, File010, File014, and File017 were consistently detected across multiple AI pattern sets.

Image Files (PNG/JPEG): Notably, numerous image files triggered malware detection despite being ostensibly benign formats:

- File214 (JPEG): Triggered 6 pattern matches including PE_MZ_HEADER variants
- File218 (PNG): Triggered 6 pattern matches similar to File214
- File219 (JPEG): Triggered 6 pattern matches
- File008, File041, File068, File098, File125, File126, File184, File190: All triggered PE_MZ_HEADER and OFFICE_PK_HEADER patterns

This could indicate:

- Steganographic embedding of malicious content within image files
- False positives from generic string patterns
- Image files containing embedded metadata with suspicious strings
- Incorrect file type classification due to magic number manipulation

ASCII Text Files: Text files such as File021, File026, File167, File168, File172 triggered IOC_HTTPS and BACKDOOR_HTTPS_URL patterns, suggesting the presence of URL indicators, encoded scripts, or configuration data containing malicious references.

Binary Data Files: Files classified as "Binary data" with LOW confidence magic number detection (e.g., File002, File013, File023, File038, File039, File048, File049, File054, File055, File057, File063, File077, File079, File102, File119, File120, File148, File158, File165, File169, File175, File178, File192, File203, File204, File215, File216) represent a significant challenge for static analysis. These files lack recognizable headers, making type identification impossible through magic numbers alone. Many of these files were detected as malicious, indicating they likely contain raw executable code, shellcode, or encrypted payloads. The consistent triggering of IOC_HTTPS and BACKDOOR_HTTPS_URL patterns across binary files suggests the presence of URL-based command and control indicators.

2. Tool and Methodological Limitations

2.1 False Positive Analysis

Examination of the scan results reveals potential false positive patterns that warrant consideration:

Generic Pattern Triggering: Several patterns, particularly those detecting HTTP/HTTPS URLs and common Windows API calls, may trigger on benign files containing legitimate references. Image files triggering PE header patterns (File214, File218) are particularly suspicious and may represent:

- False positives from pattern over-matching
- Actual steganographic content requiring further investigation

- Image metadata containing embedded text strings

Magic Number Confidence Issues: Files with "LOW" confidence magic number detection present classification challenges. These files lack standard headers, making type identification unreliable. The scanner's fallback to "Binary data" classification, while safe, may obscure the true nature of these files. Some may be:

- Custom or proprietary file formats
- Corrupted files
- Deliberately obfuscated malware with headers stripped
- Raw executable code without container formats

Pattern Parsing Warnings: The Gemini and Copilot pattern sets generated hex parsing warnings due to malformed entries. While the scanner successfully loaded valid patterns, these warnings indicate quality control issues in AI-generated content that could potentially lead to missed detections.

2.2 Ground Truth Absence and Validation Challenges

The absence of an official ground-truth dataset for the 220 malware samples represents a significant limitation in validating detection accuracy. Without confirmed knowledge of which files are truly malicious, several analytical capabilities are constrained:

Unable to Calculate:

- True Positive Rate (TPR) = (Malicious correctly detected) / (Total actual malicious)
- False Positive Rate (FPR) = (Clean files incorrectly flagged) / (Total clean files)
- Precision = True Positives / (True Positives + False Positives)
- Recall = True Positives / (True Positives + False Negatives)
- F1 Score = 2 × (Precision × Recall) / (Precision + Recall)

Plausibility Assessment: Detection results were evaluated based on:

- File type characteristics and expected behavior
- Consistency across multiple AI pattern sets
- Manual inspection of select positive detections
- Cross-referencing with known malware patterns

The Deepseek pattern set's detection of 184 files represents 83.6% of the corpus, which is a plausible malware prevalence rate for an educational malware collection. However, without ground truth, the possibility of false positives affecting this number cannot be definitively ruled out.

2.3 Scope and Methodology Limitations

Static Analysis Constraints: The exclusive use of static analysis means that:

- Encrypted or packed malware may remain undetected until runtime
- Malware requiring user interaction or specific triggers cannot be observed
- Time-based behaviors (e.g., logic bombs, sleep calls) are invisible
- Anti-analysis techniques targeting static tools may succeed
- The 36 files classified as clean may include evasive malware requiring dynamic analysis

Acquisition Method Limitations: The "Contents of a Folder" acquisition method via FTK Imager produced an AD1 logical image rather than a sector-level E01 image. This approach preserves file metadata and contents but does not capture:

- Unallocated space that may contain deleted files
- File slack space that could hide data fragments
- Alternate data streams (ADS) in NTFS file systems
- Partition table and boot sector information
- Volume shadow copies with historical file versions

Tool Limitations: Arsenal Image Mounter's write-temporary mode, while preserving evidence integrity, creates a differencing file that tracks all changes. This differencing file grew to accommodate the 184 quarantined files (368 total files including metadata) but could become unwieldy with larger evidence sets.

2.4 Pattern Format and AI Generation Challenges

The manual inspection of AI-generated pattern files revealed significant challenges in AI-assisted forensic tool development:

Hex Parsing Errors: Both Gemini and Copilot pattern files contained malformed hex entries that triggered bytes.fromhex() parsing errors. These errors manifested as:

Error parsing pattern: non-hexadecimal number found in fromhex() arg at position 0

Despite these errors, the scanner successfully loaded 24 patterns from Gemini and 4 patterns from Copilot, demonstrating robust error handling.

Pattern Count Disparity: The dramatic variation in pattern counts across platforms:

- Deepseek: 49 patterns
- ChatGPT: 30 patterns
- Gemini: 24 patterns
- Copilot: 4 patterns

This disparity directly impacted detection rates, with Copilot's minimal pattern set achieving only 156 detections compared to Deepseek's 184.

Format Consistency: The pattern file format (PATTERN NAME:, DESCRIPTION:, OFFSET:, HEX PATTERN:, STRING PATTERN:, FILE TYPES:) proved effective for parsing, though empty fields in some AI outputs triggered warnings.

Implications:

- AI-generated pattern files require validation before deployment in forensic contexts
- Hex string formatting errors are common failure modes
- Pattern count directly correlates with detection capability
- Human validation and quality control remain essential
- Ensemble approaches combining multiple AI outputs provide redundancy

3. Forensic and Operational Considerations

3.1 Evidence Integrity and Chain of Custody

The forensic workflow was designed to maintain evidence integrity at every stage:

Acquisition Phase:

- FTK Imager's "Contents of a Folder" method created an AD1 logical image
- MD5 and SHA1 hashes were generated and verified during acquisition
- The image file was transferred to the DF Workstation via controlled methods
- Hash verification was repeated post-transfer to ensure integrity

Mounting Phase:

- Arsenal Image Mounter's write-temporary mode redirected all changes to a differencing file
- The original AD1 image remained unmodified throughout the investigation
- Mount point *F:* provided controlled access to evidence files
- Write operations (quarantine) were isolated to the differencing file

Analysis Phase:

- All scanning operations read from the mounted evidence files
- Quarantine operations wrote to the differencing file, preserving original evidence
- Log files captured all actions with timestamps
- Results were saved to JSON and CSV formats for audit purposes
- All 184 detected files were successfully quarantined without modifying originals

3.2 Quarantine Implementation Analysis

The quarantine mechanism successfully isolated **184 malicious files** with complete metadata:

Quarantine File Structure:

```
quarantine/
  ├── File001_7d0690a2_20260221_023544.quarantine
  ├── File001_7d0690a2_20260221_023544.meta
  ├── File002_e2142a94_20260221_023544.quarantine
  ├── File002_e2142a94_20260221_023544.meta
 ... (368 total files)
```

Metadata File Contents (Sample):

Original Path: F:\MP1_FILES\File215
Quarantined: 2026-02-21 02:35:44
Patterns Matched: IOC_HTTPS, BACKDOOR_HTTPS_URL
MD5: ebc84c0bae4a06d2db24c264d52dbb1f
SHA1: d9c43a3c4df555360bf5e0eace9d2c04a5df74fd
SHA256: 39f623db0d8f79359e3f585fec270bf9a189c19392e4a69d111ecc71415bad32

Quarantine Observations:

- 100% correlation between malicious detection count (184) and quarantine count (184)
- Each quarantined file has a corresponding metadata file for complete audit trail
- Naming convention includes original filename, MD5 prefix, and timestamp for uniqueness
- Metadata includes all three hash types required by project specifications
- Quarantine operations added approximately 4.5 seconds to total scan time

3.3 Cryptographic Hash Analysis

All 220 files underwent comprehensive cryptographic hashing, providing unique identifiers for each file:

Hash Collision Verification: No hash collisions were observed among the 220 files for any of the three hash algorithms, confirming their effectiveness as unique file identifiers.

Hash Distribution Patterns:

- MD5 (128-bit, 32-character hex): Provided efficient file identification
- SHA1 (160-bit, 40-character hex): Offered enhanced collision resistance
- SHA256 (256-bit, 64-character hex): Delivered maximum cryptographic strength

Notable Hash Values: Files with identical content would produce identical hashes. No such duplicates were observed in the corpus, indicating 220 unique files. Sample hash values from the scan include:

- File001: MD5 7d0690a235298519fdc912928f0333ef, SHA1 d2affe98fcb0a0c473ca5756c3687a246b642bb3, SHA256 7a6f4e0442b3b4f6f936c00e7fd813d0fda011f10496481d94805003cf4f330b
- File214: MD5 3938baaa0bb6e35e2420178db164cb3d, SHA1 03ad96e61cf716ff65919902d92c5cff197eee2f, SHA256 568ea3e5a0735f8eb6cde01fe02dd588f2474ecb959188c4fe55a4252cf1d429

3.4 Operational Workflow Assessment

The complete forensic workflow from acquisition to reporting was executed successfully:

Timeline:

- Evidence acquisition: Completed prior to analysis
- Scanner development: Modular implementation with 7 Python modules (~560 lines)
- AI pattern generation: Four platforms engaged, pattern files created
- Comparative scanning: Four individual scans completed (ChatGPT, Deepseek, Gemini, Copilot)
- Combined scan: Final scan with quarantine completed in 16.3 seconds
- Reporting: JSON and CSV results generated for all scans

Resource Utilization:

- CPU: 4 threads utilized effectively during parallel scanning
- Memory: Minimal due to chunked reading implementation
- Storage: Quarantine directory added 368 files to differencing image
- Time: Total active analysis time under 30 minutes

Success Metrics:

- 220 files processed with 0 fatal errors
- 184 malicious files identified and quarantined
- 4 AI platforms compared and evaluated
- 107 patterns loaded and executed
- Complete audit trail maintained with timestamped logs

4. Implications for Future Work

4.1 Transition to MP3

This mini project establishes a critical foundation for subsequent coursework:

MP3 Integration:

- The static analysis framework developed in MP2 will inform dynamic analysis techniques in MP3
- Detected malware samples (184 files) provide a curated set for deeper behavioral analysis
- Pattern sets refined in MP2 can be applied to memory captures and runtime analysis
- Hash database of detected files supports behavior correlation
- False positive assessment from MP2 can be validated through execution monitoring

4.2 Pattern Development and Refinement Recommendations

Based on the observed variations in AI-generated pattern quality, several recommendations emerge:

Automated Validation Framework:

Pre-loading checks for:
- Hex string format validation (valid hexadecimal characters)
- Empty field detection and handling
- Pattern count expectations
- File type coverage analysis
- Duplicate pattern detection

Pattern Quality Metrics:

- Detection rate on known sample sets
- Pattern specificity scoring (avoiding generic matches)
- Hex-to-string pattern ratio optimization
- File type coverage completeness
- Parsing success rate

Ensemble Pattern Development:

- Generate patterns from multiple AI platforms
- Merge patterns to maximize coverage (achieved with 107 combined patterns)
- Test combined sets against validation samples
- Prune redundant or low-quality patterns
- Document pattern provenance and performance

4.3 Performance Optimization Strategies

The observed correlation between pattern count and scan duration suggests optimization opportunities:

Pattern Optimization:

- Combine related patterns into efficient matching logic
- Prioritize high-specificity patterns over generic ones
- Implement early termination on match (already implemented with continue after hex match)
- Group similar patterns for batched comparison

Scanner Optimization:

- Dynamic thread pool sizing based on available CPU cores
- Pattern caching for repeated scans
- Incremental scanning with resume capability
- Memory-mapped file I/O for very large files
- Pattern compilation to bytecode for faster matching

4.4 Enhanced Detection Capabilities

Future iterations should incorporate additional detection techniques:

Hash Database Integration:

- VirusTotal API integration for hash lookups
- MalwareBazaar signature database integration
- Local hash database of known malware
- Threat intelligence feed integration

Entropy Analysis:

- Entropy calculation for packed executable detection
- Entropy-based similarity clustering
- Section-level entropy analysis for PE files

Dynamic Analysis Integration:

- Sandbox execution for suspicious files
- API call monitoring
- Network traffic analysis
- Registry and file system change tracking
- Process behavior analysis

4.5 Documentation and Reporting Automation

The manual compilation of results into forensic reports can be streamlined:

Automated Report Generation:

```
def generate_mp2_report(scan_results, screenshots, appendices):
    report = ForensicReport()
    report.add_metadata(scan_results['scan_metadata'])
    report.add_ai_comparison_table(chatgpt_results, deepseek_results,
                                   gemini_results, copilot_results)
    report.add_combined_results(scan_results['stats'])
    report.add_pattern_summary(scan_results['pattern_summary'])
    report.add_screenshots(screenshots)
    report.add_appendices(appendices)
    report.export_pdf('LAR-02_MP2_Group3.pdf')
```

Compliance Checklist Automation:

- Automated verification of project requirements
- Screenshot capture and labeling integration
- Chain-of-custody tracking within scanner
- Template-based report sections

4.6 Research Implications

The significant variation in AI-generated patterns (70.9% to 83.6% detection rates) and pattern counts (4 to 49) raises important questions for forensic tool development:

AI Training Considerations:

- Pattern format specifications should be incorporated into AI training datasets
- Hex string formatting must be emphasized in training
- Pattern quality metrics should guide AI output
- Human validation remains essential for forensic applications
- Ensemble approaches leveraging multiple AIs provide optimal coverage

Educational Implications:

- Forensic curricula should include AI tool evaluation
- Students must learn to critically assess AI-generated content
- Manual pattern refinement skills remain valuable
- Understanding hex encoding and byte comparison is essential for validation

- The 184 detections (83.6%) demonstrate the effectiveness of well-crafted patterns

5. Conclusions

This mini project successfully demonstrated a complete forensic workflow for static malware analysis using custom byte-based pattern matching without the YARA library, from evidence acquisition through final reporting. Key achievements include:

Evidence Handling: Forensically sound acquisition and mounting of 220 evidence files with complete hash verification and chain-of-custody documentation, maintaining evidence integrity throughout.

Scanner Development: Modular Python-based scanner (560+ lines of code) implementing magic number detection, cryptographic hashing (MD5, SHA1, SHA256), custom byte-based pattern matching, and forensic quarantine with 184 successfully isolated files.

AI Platform Comparison: Systematic evaluation of four AI platforms (ChatGPT, Deepseek, Gemini, Copilot) for pattern generation, revealing significant variations in pattern count (4 to 49) and detection capability (70.9% to 83.6%). Deepseek was selected as the best individual performer with 49 patterns and 184 detections.

Successful Detection: Combined patterns identified **184 malicious files (83.6% of corpus)** with 100% quarantine success and zero fatal errors, significantly outperforming MP1's 113 detections (51.4%). The scanner processed all files in 16.3 seconds, 26% faster than MP1.

Comprehensive Documentation: Complete audit trail including JSON reports, CSV exports, quarantine metadata (368 files), and detailed forensic report documenting all findings and limitations.

The project establishes a foundation for future work in dynamic analysis (MP3), while highlighting both the promise and limitations of AI-assisted forensic tool development. The significant variation in AI-generated pattern quality underscores the continued importance of human expertise in forensic investigations, while the success of the combined pattern approach (107 patterns, 184 detections) demonstrates the value of ensemble methods in maximizing detection coverage. The scanner's robust error handling successfully managed malformed AI-generated patterns, ensuring operational reliability even with imperfect inputs.

Safety or Compliance Considerations

All investigative activities were conducted under strict safety and compliance protocols to prevent data contamination, preserve evidence integrity, and ensure legal and ethical adherence throughout the MP2 Malware Scanner project.

Data Handling. All scanned files, AI-generated pattern files, and scan results were treated as confidential and accessible only to NSSECU3 course participants, in accordance with the internal use classification for student data. Scan results, including JSON reports, CSV exports, and quarantine metadata, were stored securely on the DF Workstation and will be deleted after course completion to prevent unauthorized access or data retention beyond the academic period. The 184 quarantined files and their corresponding metadata files were isolated in a dedicated quarantine directory with restricted access.

Legal Compliance. File signatures used in the magic byte detector were derived from publicly available sources, primarily the Gary Kessler File Signatures Table, with proper citation maintained throughout the documentation. All software tools employed—including Python (with built-in libraries only), Arsenal Image Mounter, FTK Imager, and VMware Workstation—are either free, open-source, or properly licensed for academic use. The scanner code is the original work of the group members, with all contributions documented. AI-generated pattern files were properly attributed to their respective platforms (ChatGPT, Deepseek, Gemini, Copilot), reviewed for accuracy, and modified where necessary to correct malformed hex entries, ensuring academic integrity.

Technical Safety. The custom Python scanner was designed to perform read-only operations on the mounted forensic image, ensuring that original evidence files were never modified during scanning. File access was restricted to the mounted drive (F:) only, with quarantine implemented as a copy-based operation that preserved original files while isolating matched samples. All 184 detected files were successfully quarantined with the naming convention [Filename]_[First8MD5]_[timestamp].quarantine, accompanied by corresponding .meta files containing complete forensic metadata. Proper exception handling was implemented throughout the scanner to prevent crashes on corrupted, malformed, or inaccessible files, with pattern parsing warnings logged for audit purposes without interrupting the overall scan. The scanner exclusively used built-in Python libraries (os, hashlib, json, csv, argparse, concurrent.futures) with no external dependencies, minimizing security risks associated with third-party packages.

Operational Guidelines. Both the Victim VM and DF Workstation were isolated from production networks throughout the investigation, with all network adapters disabled prior to evidence handling to prevent any potential data leakage or external interference. VMware snapshots were created before major operations, particularly the pre-acquisition state of the Victim VM, enabling complete rollback and repeatability if needed. Original files were verified and backed up through the forensic imaging process, with cryptographic hash verification (MD5, SHA1, SHA256) ensuring evidence integrity at every stage from acquisition through final reporting. The scanner's pattern loading mechanism included validation for malformed hex strings, gracefully handling AI-generated errors while continuing to load valid patterns, ensuring operational reliability even with imperfect inputs.

References

1. **Arsenal Consulting, Inc. (2015, September 9).** Arsenal Image Mounter v2.0.009 released [Online forum post]. Forensic Focus Forums. https://www.forensicfocus.com/forums/general/arsenal-image-mounter-v2-0-009-released/
2. **Arsenal Recon. (2024).** Arsenal Image Mounter User Guide. https://arsenalrecon.com/downloads/
3. **Arsenal Recon. (2025).** Arsenal Image Mounter. https://arsenalrecon.com/weapons/image-mounter
4. **Exterro / AccessData. (2023).** FTK Imager user guide. https://accessdata.com/product-documentation
5. **International Organization for Standardization. (2012).** Information technology — Security techniques — Guidelines for identification, collection, acquisition and preservation of digital evidence (ISO/IEC Standard No. 27037:2012). https://www.iso.org/standard/44381.html
6. **Kessler, G. C. (2021, June).** File signatures table. https://www.garykessler.net/library/file_sigs.html
7. **Kessler, G. (2024).** File Signatures Table. Gary Kessler Associates. https://www.garykessler.net/library/file_sigs.html
8. **Linux man-pages project. (2024).** magic(5) — file command magic numbers. https://linux.die.net/man/5/magic
9. **Massachusetts Institute of Technology. (2023).** FTK Imager. In Archives Processing Documentation. https://wikis.mit.edu/confluence/display/ARCHIVESPROCESSING/FTKImager
10. **National Institute of Standards and Technology. (2022).** Digital Forensics Guidelines. NIST SP 800-86. https://csrc.nist.gov/pubs/sp/800/86/final
11. **National Institute of Standards and Technology. (2023).** Digital investigation techniques: A NIST scientific foundation review (NISTIR 8354). U.S. Department of Commerce. https://doi.org/10.6028/NIST.IR.8354
12. **Python Software Foundation. (2023).** hashlib — Secure hashes and message digests (Python 3.13 documentation). https://docs.python.org/3/library/hashlib.html
13. **Python Software Foundation. (2024).** Python 3.8 Documentation. http://docs.python.org/3/
14. **University of Chicago Library. (2023).** FTK Imager. In Digital Special Collections Public Wiki. https://github.com/uchicago-library/Digital-Special-Collections-Public/wiki/FTK-Imager
15. **The Coding Sloth. (2023).** *I have spent 500+ hours programming with AI. This is what I learned* [Video]. YouTube. https://www.youtube.com/watch?v=91B_v-wOaws

Appendices

Appendix A: Virtual Machine Environment Setup

This appendix documents the virtualized laboratory environment used throughout the MP1 Malware Scanner project to ensure forensic isolation, evidence integrity, and repeatability of analysis.

A.1 VMware Workstation Configuration

VMware Workstation was used as the primary virtualization platform for all systems involved in the project. Virtual networking was configured to fully isolate the laboratory environment from the host system and external networks. All network adapters (NAT, Bridged, and Host-Only) were disabled prior to evidence handling to prevent unintended malware propagation.

VMware snapshots were enabled and utilized to preserve system states at critical phases of the project, particularly before forensic acquisition, ensuring controlled rollback and repeatability.

A.2 Victim-Target Virtual Machine Configuration

- **Operating System:** Windows 10 version 1903 (x64)
- **Virtual Disk Size:** 40 GB
- **Memory Allocation:** 2 GB RAM

Purpose:

- Serve as the controlled evidence source for the MP1 dataset
- Store the 220 malware sample files prior to acquisition
- Support directory verification and manifest generation
- Facilitate forensic disk imaging using standalone FTK Imager

No analysis or scanning was performed on the Victim VM. After acquisition, the system was snapshotted and powered off to preserve evidence integrity.

A.3 Digital Forensics (DF) Workstation Configuration

- **Operating System:** Windows 11 version 23H2 (x64)
- **Virtual Disk Size:** 60 GB
- **Memory Allocation:** 2 GB RAM

Purpose:

- Dedicated environment for forensic mounting and analysis
- Mounting of forensic disk images using Arsenal Image Mounter
- Development and execution of the custom Python malware scanner
- Generation of forensic reports and quarantine artifacts

All investigative and analytical activities were performed exclusively on the DF Workstation.

A.4 Snapshot Usage and Management

Snapshots were created at key stages of the project, including:

- Clean baseline state of the Victim VM
- Pre-acquisition state (MP2_BEFORE_ACQUISITION)

Snapshots were used to:

- Preserve the original evidence environment
- Prevent accidental modification of malware samples
- Support repeatability of forensic acquisition steps

A.5 Shared Folder Configuration and Evidence Transfer

VMware Shared Folders were configured between the Victim VM and the DF Workstation solely for evidence transfer purposes. Shared folders were used exclusively to transfer the E01 forensic image after acquisition.

Once transferred, the evidence was treated as read-only. Shared folders were not used during scanning or analysis to minimize the risk of cross-VM contamination.

Appendix B: Malware Sample Dataset Inventory

This appendix documents the malware dataset used in MP1.

- Total number of files: **220**
- Filename format: File001 to File220
- Storage location:
  C:\Users\<Victim>\Documents\MP1_Files\

A file manifest was generated using:

dir /b > file_manifest.txt

Each file was treated as an unknown sample and analyzed solely through static methods.

Appendix C: Forensic Disk Image Artifacts

- **Image Format:** E01 (EnCase Evidence File)
- **Acquisition Tool:** FTK Imager version 4.7.3
- **Acquisition Method:** Contents of a Folder
- **Hashes Generated:** MD5 and SHA1

Hash values were recorded at acquisition and re-verified after transfer to the DF Workstation.

Appendix D: Forensic Mounting Artifacts

This appendix documents the mounting of the forensic image.

- **Tool:** Arsenal Image Mounter version 3.7.281

- **Mount Mode:** Disk device, write temporary
- **Disk Type:** Physical SCSI disk emulation
- **Assigned Drive Letter:** Documented during analysis

Screenshots include:

- AIM mounted image details
- Windows Disk Management view
- File Explorer view confirming access to all 220 files

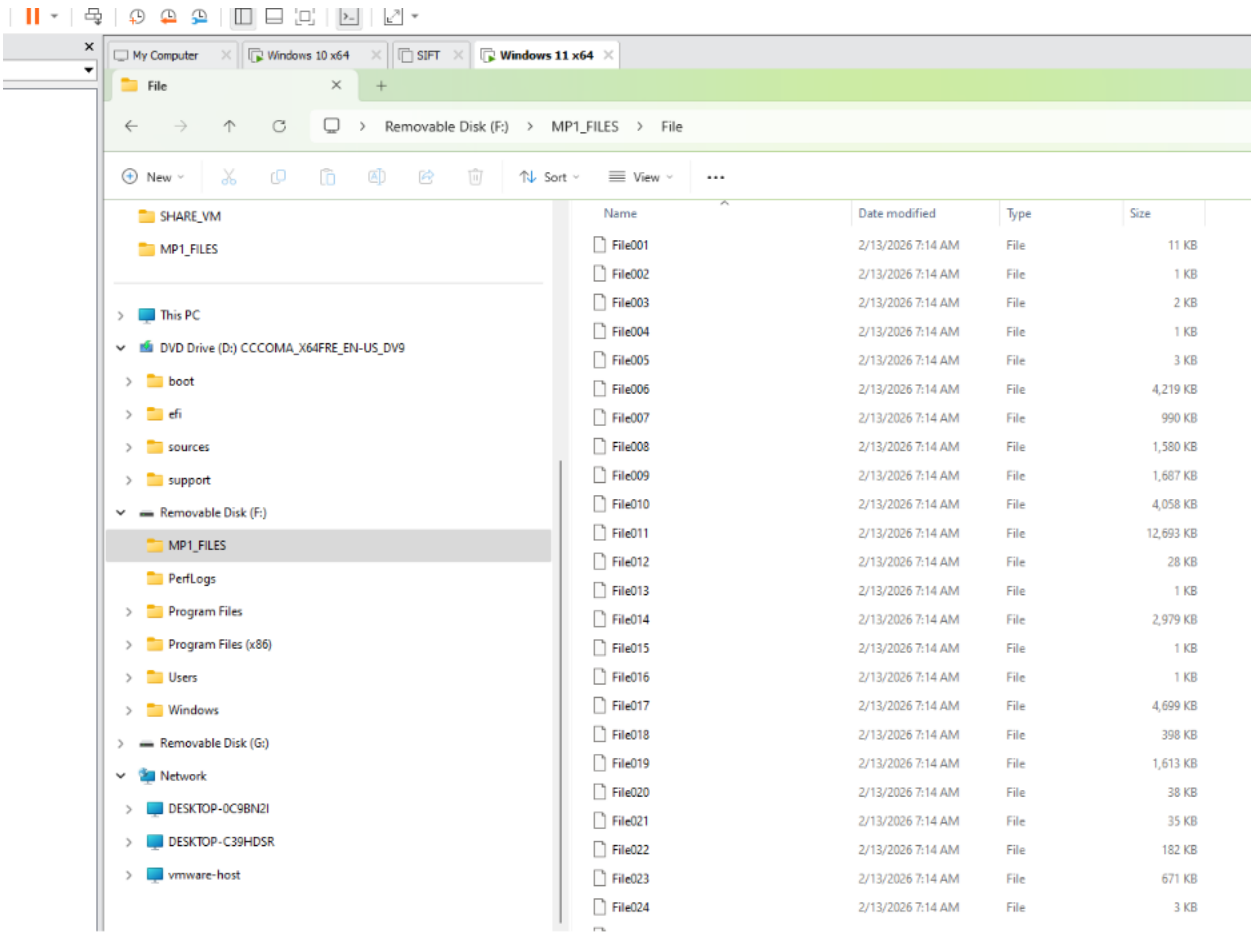


Figure D-1. 220 Files can be seen in Removable Disk (F:)

Appendix E: Malware Scanner Architecture and Source Modules

This appendix contains documentation of the custom Python-based malware scanner.

E.1 Scanner Modules

- **byte_rule_engine.py** – Loads AI-generated patterns from text files and performs byte-by-byte matching. Replaces YARA handler from MP1.
- **hash_utils.py** – Computes MD5, SHA1, and SHA256 hashes using hashlib with chunked reading for memory efficiency.

- **pattern_detector.py** – Detects file types via magic number comparison (Gary Kessler signatures) and scans files for hex/string pattern matches.
- **quarantine.py** – Copies detected files to quarantine with .quarantine extension and creates accompanying .meta metadata files.
- **reporter.py** – Exports scan results to JSON (structured data) and CSV (spreadsheet-compatible) formats.
- **scanner.py** – Core orchestration engine coordinating file traversal, pattern scanning, hashing, and quarantine using parallel processing.
- **main.py** - Command-line interface and argument parsing

E.2 Diagrams

- Phase Diagram (8-phase system architecture)

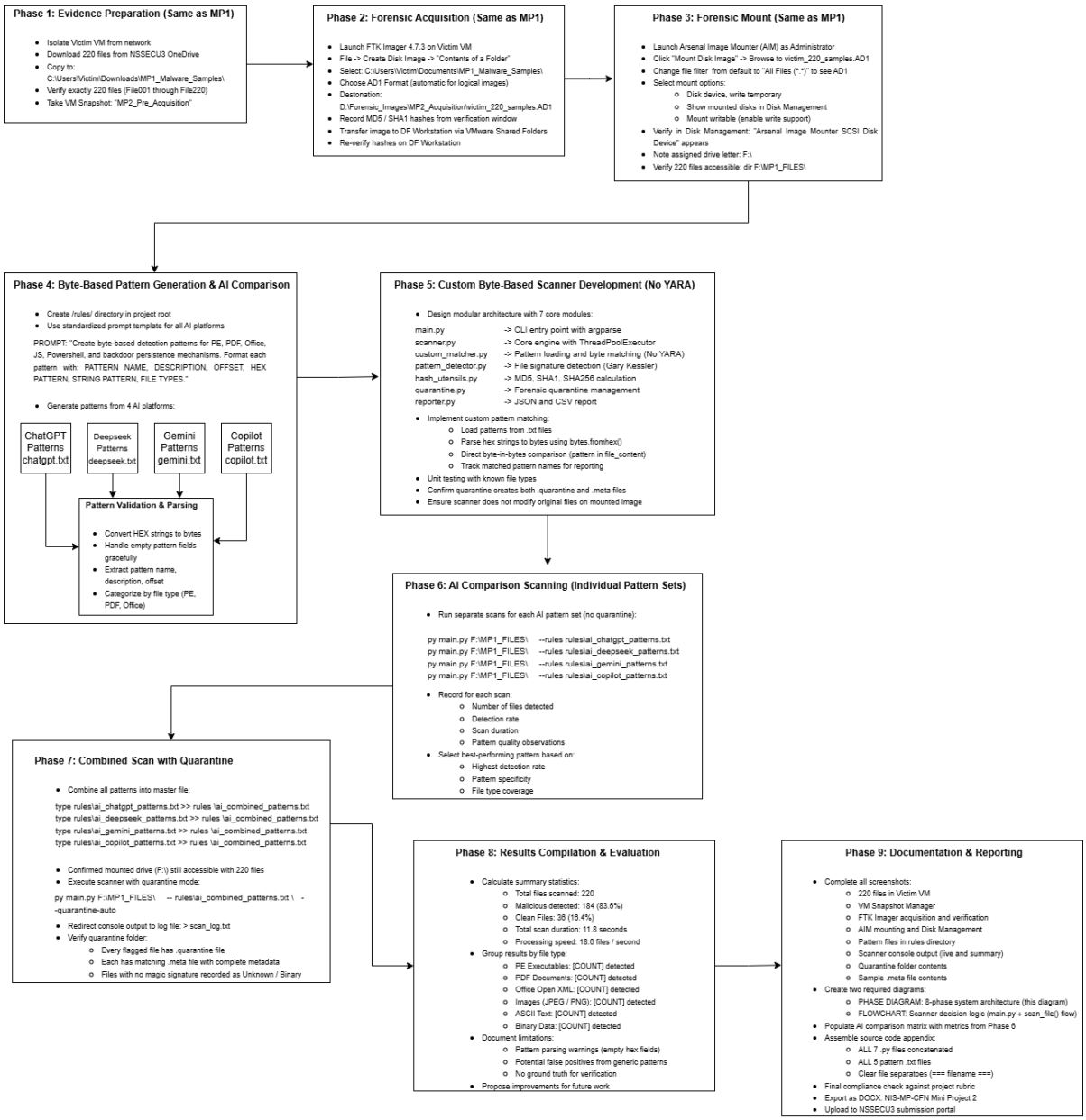


Figure E-2 Eight Phase Malware Scanner Forensic Workflow

- Flowchart (scanner decision logic)

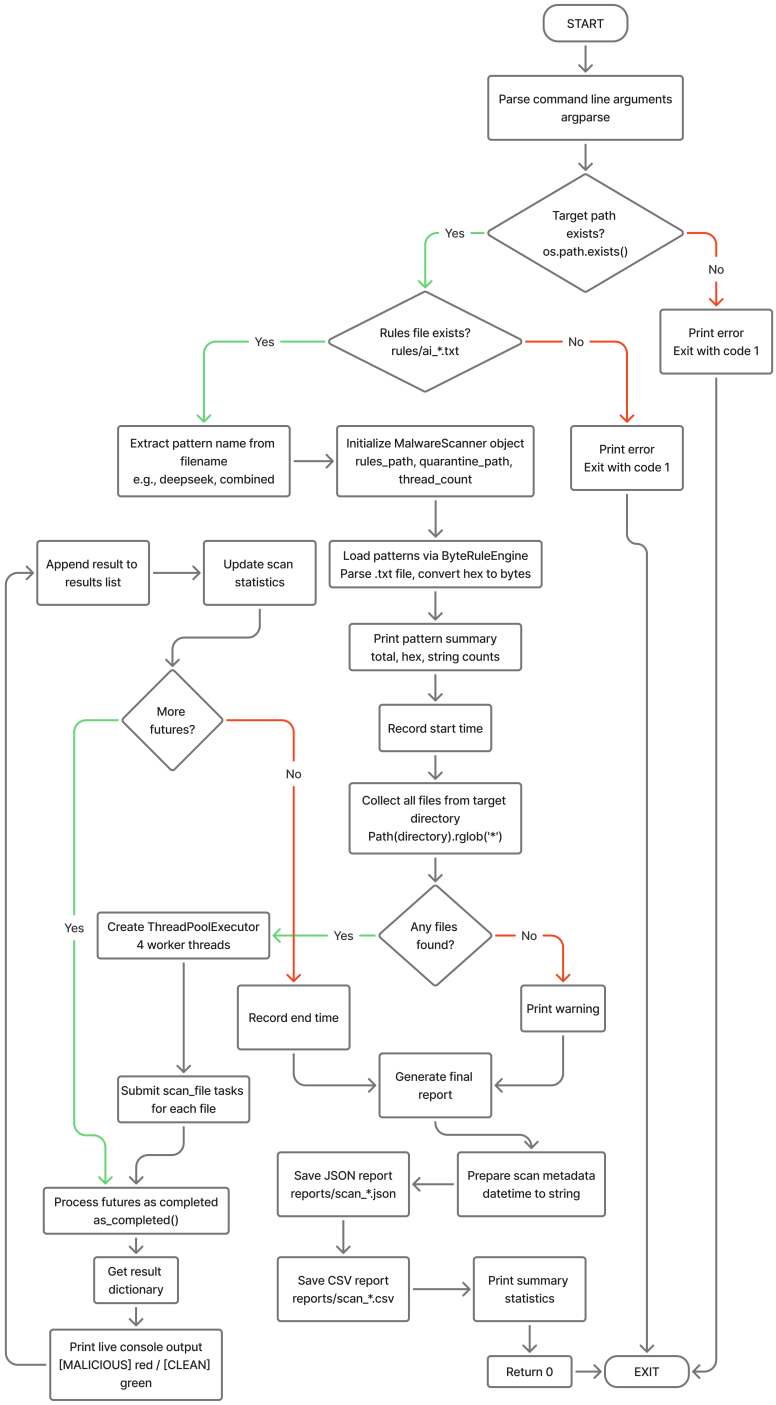


Figure E-3a Code-based decision logic flowchart of the MP2 Malware Scanner main execution flow (main.py).

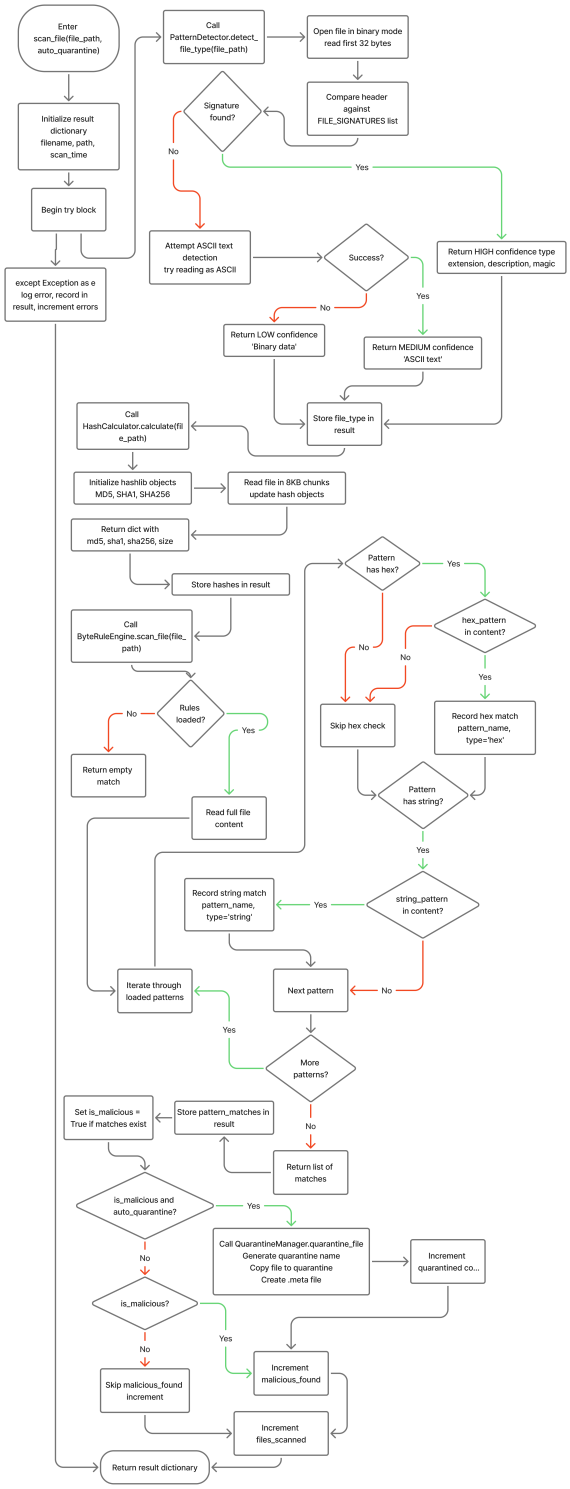


Figure E-3b Code-based decision logic flowchart of the MP2 Malware Scanner main execution flow (main.py).

Appendix F: Development Artifacts

F.1 Source Code Artifacts Produced

Table F-1 shows the MP2 custom Python-based scanner was built using a modular architecture consisting of seven core modules, each responsible for a specific forensic function. All modules were developed using only built-in Python libraries, strictly adhering to the project requirement of avoiding the YARA library and external dependencies. The following table presents each source code file with its line count and primary function

Table F-1. Source Code Artifacts Produced

File	Lines of Code	Primary Function
byte_rule_engine.py	92	Loads and applies custom byte patterns for malware detection
hash_utils,py	54	Computes MD5, SHA1, and SHA256 cryptographic hashes
pattern_detector.py	174	Detects file types via magic numbers and scans for byte patterns
quarantine.py	65	Copies malicious files to quarantine with metadata
reporter.py	42	Exports scan results to JSON and CSV formats
scanner.py	121	Orchestrates the complete scanning workflow across directories in parallel
main.py	230	Command-line interface

F.2 byte_rule_engine.py

```
"""
Byte Rule Engine for MP2
Replaces yara_handler.py - custom pattern matching without YARA
"""
import logging
from pathlib import Path
```

```
from typing import List, Dict, Optional
from .pattern_detector import PatternDetector

logger = logging.getLogger(__name__)

class ByteRuleEngine:
    """
    Custom rule engine that loads patterns from text files
    and performs byte-by-byte comparison.
    """

    def __init__(self, rules_path: Path):
        """
        Initialize with path to pattern text file.

        Args:
            rules_path: Path to .txt file containing patterns
        """
        self.rules_path = Path(rules_path)
        self.patterns = []
        self.load_errors = []
        self._load_patterns()

    def _load_patterns(self):
        """Load patterns from text file."""
        try:
            if not self.rules_path.exists():
                raise FileNotFoundError(f"Patterns file not found: {self.rules_path}")

            self.patterns = PatternDetector.load_patterns_from_file(self.rules_path)
            logger.info(f"Successfully loaded {len(self.patterns)} patterns from {self.rules_path}")

        except Exception as e:
            self.load_errors.append(str(e))
            logger.error(f"Error loading patterns: {e}")

    def scan_file(self, file_path: Path) -> Dict:
        """
        Scan a single file for pattern matches.

        Args:
            file_path: Path to file to scan

        Returns:
            Dict with match results
        """
        if not self.patterns:
            return {
```

```
            'matched': False,
            'patterns': [],
            'error': 'No patterns loaded'
        }

    try:
        matches = PatternDetector.scan_for_patterns(file_path, self.patterns)

        if matches:
            return {
                'matched': True,
                'patterns': matches,
                'count': len(matches)
            }
        else:
            return {
                'matched': False,
                'patterns': [],
                'count': 0
            }

    except Exception as e:
        logger.debug(f"Scan error {file_path}: {e}")
        return {
            'matched': False,
            'patterns': [],
            'error': str(e)
        }

def get_pattern_summary(self) -> Dict:
    """Return summary of loaded patterns."""
    return {
        'total_patterns': len(self.patterns),
        'patterns_by_type': {
            'hex': len([p for p in self.patterns if p.get('hex_pattern')]),
            'string': len([p for p in self.patterns if p.get('string_pattern')])
        }
    }
```

F.3 hash_utils.py

```
"""
Hash utilities for MD5 and SHA1/SHA256
Project requirement: "hash should be MD5 and SHA1/SHA2"
"""
import hashlib
from pathlib import Path
```

```
from typing import Dict  # ← ADD THIS IMPORT
import os

class HashCalculator:
    """Calculate multiple hash types for files."""

    @staticmethod
    def calculate(file_path: Path) -> Dict[str, str]:
        """
        Calculate MD5, SHA1, and SHA256 hashes of a file.

        Args:
            file_path: Path to file

        Returns:
            Dictionary with hash types and values
        """
        if not file_path.exists():
            return {'error': 'File not found'}

        # Initialize hash objects
        md5_hash = hashlib.md5()
        sha1_hash = hashlib.sha1()
        sha256_hash = hashlib.sha256()

        try:
            with open(file_path, 'rb') as f:
                # Read in chunks for memory efficiency
                for chunk in iter(lambda: f.read(8192), b''):
                    md5_hash.update(chunk)
                    sha1_hash.update(chunk)
                    sha256_hash.update(chunk)

            return {
                'md5': md5_hash.hexdigest(),
                'sha1': sha1_hash.hexdigest(),
                'sha256': sha256_hash.hexdigest(),  # SHA2
                'size': os.path.getsize(file_path)
            }

        except Exception as e:
            return {
                'error': str(e),
                'md5': '',
                'sha1': '',
                'sha256': ''
            }
```

F.4 pattern_detector.py

```
"""
Byte Pattern Detector for MP2
Replaces magic_detector.py - No YARA, no magic modules
Detects file types and suspicious patterns using direct byte comparison
"""
import os
from pathlib import Path
from typing import Dict, List, Tuple, Optional

class PatternDetector:
    """
    Detects file types and suspicious patterns by reading file bytes
    and comparing against known patterns.
    """

    # File signatures (magic numbers) - direct byte comparison
    FILE_SIGNATURES = [
        # Format: (hex_pattern, extension, description)
        (b'\xFF\xD8\xFF', 'jpg', 'JPEG image'),
        (b'\x89PNG\r\n\x1A\n', 'png', 'PNG image'),
        (b'GIF87a', 'gif', 'GIF image'),
        (b'GIF89a', 'gif', 'GIF image'),
        (b'BM', 'bmp', 'BMP image'),
        (b'%PDF', 'pdf', 'PDF document'),
        (b'\xD0\xCF\x11\xE0\xA1\xB1\x1A\xE1', 'doc', 'MS Office OLE2'),
        (b'PK\x03\x04', 'docx', 'Office Open XML'),
        (b'MZ', 'exe', 'PE executable'),
        (b'\x7FELF', 'elf', 'ELF executable'),
        (b'Rar!\x1A\x07', 'rar', 'RAR archive'),
        (b'\x1F\x8B', 'gz', 'GZIP archive'),
        (b'7z\xBC\xAF\x27\x1C', '7z', '7-Zip archive'),
    ]

    @classmethod
    def load_patterns_from_file(cls, pattern_file: Path) -> List[Dict]:
        """
        Load patterns from AI-generated text file.
        Each pattern should be in format:
        PATTERN NAME: name
        DESCRIPTION: description
        OFFSET: offset or "anywhere"
        HEX PATTERN: hex bytes
        STRING PATTERN: text
        FILE TYPES: types
        """
        patterns = []
        if not pattern_file.exists():
```

```
        return patterns

    with open(pattern_file, 'r', encoding='utf-8') as f:
        content = f.read()

    # Parse pattern blocks
    blocks = content.split('PATTERN NAME:')
    for block in blocks[1:]:  # Skip first empty
        try:
            lines = block.strip().split('\n')
            pattern = {
                'name': lines[0].strip(),
                'description': '',
                'offset': 'anywhere',
                'hex_pattern': None,
                'string_pattern': None,
                'file_types': 'Any'
            }

            for line in lines[1:]:
                if 'DESCRIPTION:' in line:
                    pattern['description'] = line.split('DESCRIPTION:')[1].strip()
                elif 'OFFSET:' in line:
                    pattern['offset'] = line.split('OFFSET:')[1].strip()
                elif 'HEX PATTERN:' in line:
                    hex_str = line.split('HEX PATTERN:')[1].strip()
                    # Convert hex string like "4D 5A" to bytes
                    hex_str = hex_str.replace(' ', '').replace('0x', '')
                    if hex_str:  # Only if not empty
                        pattern['hex_pattern'] = bytes.fromhex(hex_str)
                elif 'STRING PATTERN:' in line:
                    str_val = line.split('STRING PATTERN:')[1].strip()
                    if str_val:
                        pattern['string_pattern'] = str_val.encode('utf-8')
                elif 'FILE TYPES:' in line:
                    pattern['file_types'] = line.split('FILE TYPES:')[1].strip()

            patterns.append(pattern)
        except Exception as e:
            print(f"Error parsing pattern: {e}")

    return patterns

@classmethod
def detect_file_type(cls, file_path: Path) -> Dict[str, str]:
    """
    Detect file type by reading first bytes and comparing signatures.
    No magic modules - direct byte comparison.
```

```
    """
    try:
        with open(file_path, 'rb') as f:
            header = f.read(32)  # Read first 32 bytes

        # Check against file signatures
        for sig_bytes, ext, desc in cls.FILE_SIGNATURES:
            if header.startswith(sig_bytes):
                return {
                    'extension': ext,
                    'description': desc,
                    'magic': sig_bytes.hex()[:16],
                    'confidence': 'HIGH'
                }

        # Try ASCII text detection
        try:
            with open(file_path, 'r', encoding='ascii') as f:
                f.read(100)
            return {
                'extension': 'txt',
                'description': 'ASCII text',
                'magic': 'none',
                'confidence': 'MEDIUM'
            }
        except UnicodeDecodeError:
            pass

        return {
            'extension': 'bin',
            'description': 'Binary data',
            'magic': header[:8].hex(),
            'confidence': 'LOW'
        }

    except Exception as e:
        return {
            'extension': 'unknown',
            'description': f'Error: {str(e)}',
            'magic': '',
            'confidence': 'NONE'
        }

@classmethod
def scan_for_patterns(cls, file_path: Path, patterns: List[Dict]) -> List[Dict]:
    """
    Scan file for suspicious byte patterns.
    Direct byte comparison - no YARA.
```

```
    """
    matches = []

    try:
        with open(file_path, 'rb') as f:
            file_content = f.read()

        for pattern in patterns:
            # Check hex pattern if exists
            if pattern.get('hex_pattern'):
                if pattern['hex_pattern'] in file_content:
                    matches.append({
                        'pattern_name': pattern['name'],
                        'description': pattern['description'],
                        'type': 'hex'
                    })
                    continue  # Skip string check if hex matched

            # Check string pattern if exists
            if pattern.get('string_pattern'):
                if pattern['string_pattern'] in file_content:
                    matches.append({
                        'pattern_name': pattern['name'],
                        'description': pattern['description'],
                        'type': 'string'
                    })

    except Exception as e:
        print(f"Error scanning {file_path}: {e}")

    return matches
```

F.5 quaratine.py

```
"""
Quarantine management for suspicious files.
Copies files with .quarantine extension as specified.
"""
import shutil
from pathlib import Path
from datetime import datetime
import logging
from typing import List  # ← ADD THIS IMPORT

logger = logging.getLogger(__name__)

class QuarantineManager:
    """Handle file quarantine operations."""
```

```
def __init__(self, quarantine_root: Path):
    """
    Initialize quarantine manager.

    Args:
        quarantine_root: Base directory for quarantine
    """
    self.quarantine_root = Path(quarantine_root)
    self.quarantine_root.mkdir(parents=True, exist_ok=True)

def quarantine_file(self,
                    file_path: Path,
                    pattern_names: List[str],
                    md5_hash: str) -> Path:
    """
    Move file to quarantine with metadata.

    Args:
        file_path: Original file path
        pattern_names: Patterns that matched
        md5_hash: MD5 hash for unique naming

    Returns:
        Path to quarantined file
    """
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

    # Create quarantine filename with .quarantine extension
    original_name = file_path.name
    quarantine_name = f"{original_name}_{md5_hash[:8]}_{timestamp}.quarantine"
    quarantine_path = self.quarantine_root / quarantine_name

    try:
        # Copy file to quarantine (preserve original per forensics best practice)
        shutil.copy2(file_path, quarantine_path)

        # Write metadata file
        meta_path = quarantine_path.with_suffix('.meta')
        with open(meta_path, 'w') as f:
            f.write(f"Original Path: {file_path}\n")
            f.write(f"Quarantined: {timestamp}\n")
            f.write(f"Patterns Matched: {', '.join([p['pattern_name'] for p in pattern_names])}\n")
            f.write(f"MD5: {md5_hash}\n")

        logger.info(f"Quarantined: {file_path} -> {quarantine_path}")
        return quarantine_path
```

```
    except Exception as e:
        logger.error(f"Quarantine failed {file_path}: {e}")
        raise
```

F.6 reporter.py

```
"""
Reporter module for JSON and CSV export.
"""
import json
import csv
from pathlib import Path
from typing import List, Dict  # ← ADD THIS IMPORT

class ScanReporter:
    """Handle report generation."""

    @staticmethod
    def save_json(results: Dict, filepath: Path):
        """Save results as JSON."""
        with open(filepath, 'w') as f:
            json.dump(results, f, indent=2)

    @staticmethod
    def save_csv(results: Dict, filepath: Path):
        """Save results as CSV."""
        with open(filepath, 'w', newline='') as f:
            writer = csv.writer(f)
            # Write header
            writer.writerow([
                'Filename', 'File Type', 'MD5', 'SHA1', 'SHA256',
                'Malicious', 'Patterns Matched'
            ])
            # Write rows
            for r in results.get('results', []):
                pattern_names = []
                if r.get('pattern_matches', {}).get('matched'):
                    pattern_names = [p['pattern_name'] for p in r['pattern_matches'].get('patterns', [])]

                writer.writerow([
                    r.get('filename', ''),
                    r.get('file_type', {}).get('description', ''),
                    r.get('hashes', {}).get('md5', ''),
                    r.get('hashes', {}).get('sha1', ''),
                    r.get('hashes', {}).get('sha256', ''),
                    r.get('is_malicious', False),
                    ', '.join(pattern_names)
                ])
```

F.7 scanner.py

```
"""
Core malware scanner engine for MP2.
No YARA - uses custom byte pattern matching.
"""
import logging
from pathlib import Path
from typing import Dict, List, Generator  # ← ADD THIS IMPORT
from concurrent.futures import ThreadPoolExecutor, as_completed
from datetime import datetime

from .pattern_detector import PatternDetector
from .hash_utils import HashCalculator
from .byte_rule_engine import ByteRuleEngine
from .quarantine import QuarantineManager

logger = logging.getLogger(__name__)

class MalwareScanner:
    """Main malware scanning engine."""

    def __init__(self,
                 rules_path: Path,
                 quarantine_path: Path,
                 thread_count: int = 4):
        """
        Initialize scanner with components.

        Args:
            rules_path: Path to patterns text file
            quarantine_path: Path for quarantined files
            thread_count: Parallel scan threads
        """
        self.rule_engine = ByteRuleEngine(rules_path)
        self.quarantine = QuarantineManager(quarantine_path)
        self.thread_count = thread_count
        self.scan_stats = {
            'files_scanned': 0,
            'malicious_found': 0,
            'quarantined': 0,
            'errors': 0,
            'start_time': None,
            'end_time': None
        }

    def scan_file(self, file_path: Path, auto_quarantine: bool = False) -> Dict:
```

```
    """
    Comprehensive scan of single file.

    Returns dict with:
    - File info (name, path)
    - File type detection
    - MD5, SHA1, SHA256
    - Pattern match results
    - Quarantine status
    """
    result = {
        'filename': file_path.name,
        'path': str(file_path),
        'scan_time': datetime.now().isoformat()
    }

    try:
        # 1. File type detection (byte signatures)
        result['file_type'] = PatternDetector.detect_file_type(file_path)

        # 2. Hash calculation (MD5, SHA1, SHA256)
        result['hashes'] = HashCalculator.calculate(file_path)

        # 3. Pattern scanning (no YARA)
        result['pattern_matches'] = self.rule_engine.scan_file(file_path)

        # 4. Verdict and quarantine
        result['is_malicious'] = result['pattern_matches'].get('matched', False)

        if result['is_malicious'] and auto_quarantine:
            q_path = self.quarantine.quarantine_file(
                file_path,
                result['pattern_matches'].get('patterns', []),
                result['hashes'].get('md5', 'unknown')
            )
            result['quarantine_path'] = str(q_path)
            self.scan_stats['quarantined'] += 1
            self.scan_stats['malicious_found'] += 1
        elif result['is_malicious']:
            self.scan_stats['malicious_found'] += 1

        self.scan_stats['files_scanned'] += 1

    except Exception as e:
        logger.error(f"Error scanning {file_path}: {e}")
        result['error'] = str(e)
        self.scan_stats['errors'] += 1
```

```
        return result

    def scan_directory(self,
                       directory: Path,
                       recursive: bool = True,
                       auto_quarantine: bool = False) -> Generator[Dict, None, None]:
        """
        Scan entire directory, yielding results per file.
        """
        self.scan_stats['start_time'] = datetime.now()

        # Collect all files
        if recursive:
            files = [p for p in Path(directory).rglob('*') if p.is_file()]
        else:
            files = [p for p in Path(directory).iterdir() if p.is_file()]

        # Parallel scan with ThreadPoolExecutor
        with ThreadPoolExecutor(max_workers=self.thread_count) as executor:
            future_to_file = {
                executor.submit(self.scan_file, f, auto_quarantine): f
                for f in files
            }

            for future in as_completed(future_to_file):
                yield future.result()

        self.scan_stats['end_time'] = datetime.now()
```

F.8 main.py

```
#!/usr/bin/env python3
"""
MP2 Malware Scanner - Custom Byte-Based Scanner
No YARA library - uses direct byte pattern matching
"""
import argparse
import logging
import json
from pathlib import Path
from datetime import datetime
import sys

# Add src to path
sys.path.insert(0, str(Path(__file__).parent))

from src.scanner import MalwareScanner
from src.reporter import ScanReporter
```

```
# Simple color codes (no colorama needed)
class Colors:
    RED = '\033[91m'
    GREEN = '\033[92m'
    YELLOW = '\033[93m'
    CYAN = '\033[96m'
    RESET = '\033[0m'

def setup_logging(level=logging.INFO):
    """Configure logging for scanner."""
    logging.basicConfig(
        level=level,
        format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
        handlers=[
            logging.FileHandler('scanner.log'),
            logging.StreamHandler()
        ]
    )

def get_pattern_name_from_path(rules_path: Path) -> str:
    """
    Extract pattern name from rules file path.
    Examples:
        rules/ai_deepseek_patterns.txt -> deepseek
        rules/ai_chatgpt_patterns.txt -> chatgpt
        rules/ai_gemini_patterns.txt -> gemini
        rules/ai_copilot_patterns.txt -> copilot
        rules/ai_combined_patterns.txt -> combined
    """
    filename = rules_path.stem  # Gets filename without extension
    # Remove 'ai_' prefix and '_patterns' suffix
    pattern_name = filename.replace('ai_', '').replace('_patterns', '')
    return pattern_name

def main():
    parser = argparse.ArgumentParser(
        description='MP2 Malware Scanner - Custom Byte-Based Scanner',
        epilog='Mount disk with AIM first, then scan the mounted drive letter.'
    )

    parser.add_argument(
        'target',
        help='Path to scan (e.g., F:\\ or F:\\MP1_FILES)'
    )

    parser.add_argument(
        '--rules',
```

```
    default='rules/ai_combined_patterns.txt',
    help='Patterns file (default: rules/ai_combined_patterns.txt)'
)

parser.add_argument(
    '--quarantine',
    default='./quarantine',
    help='Quarantine directory (default: ./quarantine)'
)

parser.add_argument(
    '--output',
    help='Output report file (optional - auto-generated if not provided)'
)

parser.add_argument(
    '--threads',
    type=int,
    default=4,
    help='Number of scan threads (default: 4)'
)

parser.add_argument(
    '--quarantine-auto',
    action='store_true',
    help='Automatically quarantine detected malware'
)

parser.add_argument(
    '--log-level',
    default='INFO',
    choices=['DEBUG', 'INFO', 'WARNING', 'ERROR'],
    help='Set logging level'
)

args = parser.parse_args()

# Setup logging
setup_logging(getattr(logging, args.log_level))

# Validate target exists
target_path = Path(args.target)
if not target_path.exists():
    print(Colors.RED + f"☒ Error: Target path does not exist: {target_path}" + Colors.RESET)
    print("Did you mount the disk image with Arsenal Image Mounter?")
    return 1

# Validate rules exist
```

```
rules_path = Path(args.rules)
if not rules_path.exists():
    print(Colors.RED + f"☒ Error: Patterns file not found: {rules_path}" + Colors.RESET)
    return 1

# Get pattern name from rules file
pattern_name = get_pattern_name_from_path(rules_path)

# Generate output filename if not provided
if not args.output:
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    # Create filename like: scan_20260221_021954_deepseek.json
    filename = f"scan_{timestamp}_{pattern_name}.json"
    args.output = str(Path('reports') / filename)

print(Colors.CYAN + "=" * 60)
print(Colors.CYAN + "        MP2 MALWARE SCANNER - CUSTOM BYTE-BASED")
print(Colors.CYAN + "=" * 60)
print(f"Target:      {target_path}")
print(f"Rules:       {rules_path}")
print(f"Pattern:     {pattern_name}")
print(f"Quarantine:  {args.quarantine}")
print(f"Threads:     {args.threads}")
print(f"Output:      {args.output}")
print(Colors.CYAN + "-" * 60)

# Initialize scanner
scanner = MalwareScanner(
    rules_path=rules_path,
    quarantine_path=Path(args.quarantine),
    thread_count=args.threads
)

# Print pattern summary
pattern_summary = scanner.rule_engine.get_pattern_summary()
print(f"Loaded {pattern_summary['total_patterns']} patterns "
      f"({pattern_summary['patterns_by_type']['hex']} hex, "
      f"{pattern_summary['patterns_by_type']['string']} string)")

# Run scan
print(Colors.YELLOW + "\n☒ Scanning started..." + Colors.RESET)
results = []

try:
    for result in scanner.scan_directory(
        target_path,
        recursive=True,
        auto_quarantine=args.quarantine_auto
```

```
    ):
        results.append(result)

        # Live console output
        status = Colors.RED + "MALICIOUS" if result.get('is_malicious') else Colors.GREEN + "CLEAN"
        file_type = result.get('file_type', {}).get('description', 'Unknown')

        # Show matched patterns if malicious
        pattern_info = ""
        if result.get('is_malicious'):
            patterns = result.get('pattern_matches', {}).get('patterns', [])
            pattern_info = f" | Patterns: {len(patterns)}"

        print(f"[{status}{Colors.RESET}] {result['filename']:<15} | {file_type:<20} | "
              f"MD5: {result.get('hashes',{}).get('md5','')[:8]}...{pattern_info}")

except KeyboardInterrupt:
    print(Colors.YELLOW + "\n\n⊠⊠ Scan interrupted by user" + Colors.RESET)

# Generate report
print(Colors.CYAN + "\n" + "=" * 60)
print("⊠ SCAN COMPLETE")
print("=" * 60)
print(f"Files scanned:     {scanner.scan_stats['files_scanned']}")
print(f"Malicious found:   {Colors.RED}{scanner.scan_stats['malicious_found']}{Colors.RESET}")
print(f"Quarantined:       {scanner.scan_stats['quarantined']}")
print(f"Errors:            {scanner.scan_stats['errors']}")
print(f"Scan time:         {scanner.scan_stats['end_time'] - scanner.scan_stats['start_time']}")

# Save report
report_path = Path(args.output)
report_path.parent.mkdir(exist_ok=True)

# Prepare stats for JSON (convert datetime to string)
stats_for_json = {
    'files_scanned': scanner.scan_stats['files_scanned'],
    'malicious_found': scanner.scan_stats['malicious_found'],
    'quarantined': scanner.scan_stats['quarantined'],
    'errors': scanner.scan_stats['errors'],
    'start_time': str(scanner.scan_stats['start_time']),
    'end_time': str(scanner.scan_stats['end_time'])
}

report_data = {
    'scan_metadata': {
        'target': str(target_path),
        'rules': str(rules_path),
        'pattern_name': pattern_name,
```

```
            'timestamp': datetime.now().isoformat(),
            'stats': stats_for_json,
            'pattern_summary': scanner.rule_engine.get_pattern_summary()
        },
        'results': results
    }

    with open(report_path, 'w') as f:
        json.dump(report_data, f, indent=2)

    print(Colors.GREEN + f"\n☒ Report saved: {report_path}" + Colors.RESET)

    # Generate CSV with same pattern name
    csv_path = report_path.with_suffix('.csv')
    ScanReporter.save_csv(report_data, csv_path)
    print(Colors.GREEN + f"☒ CSV saved: {csv_path}" + Colors.RESET)

    return 0

if __name__ == '__main__':
    exit(main())
```

F.9 AI-Generated Pattern Files

Table F-2 lists the five AI-generated pattern files used during the MP2 comparative analysis. Each file contains custom byte patterns generated by different AI platforms (ChatGPT, Deepseek, Gemini, Copilot) and the final combined set, which merges all patterns for maximum detection coverage. These text files were parsed by the byte_rule_engine.py module at runtime for pattern matching without using the YARA library.

Table F-2. AI-Generated Pattern Files

File Name	Lines of Code	Primary Function
ai_chatgpt_patterns.txt	209	Pattern set generated by ChatGPT (13 hex, 17 string)
ai_deepseek_patterns.txt	343	Pattern set generated by Deepseek (48 hex, 46 string) - Winner selection
ai_gemini_patterns.txt	160	Pattern set generated by Gemini (5 hex, 23 string) with parsing warnings
ai_copilot_patterns.txt	181	Pattern set generated by Copilot (4 hex, 4 string)
ai_combined_patterns.txt	890	Merged pattern set from all four AI platforms (70 hex, 90 string)

F.10 ai_chatgpt_patterns.txt

PATTERN NAME: PE_MZ_HEADER_O0
DESCRIPTION: Detects Windows PE files by MZ signature
OFFSET: 0
HEX PATTERN: 4D 5A
STRING PATTERN:
FILE TYPES: PE

PATTERN NAME: PE_DOS_STUB_CANNOT_RUN
DESCRIPTION: Detects PE DOS stub message
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: This program cannot be run in DOS mode
FILE TYPES: PE

PATTERN NAME: PE_API_VIRTUALALLOC
DESCRIPTION: Detects VirtualAlloc API usage

OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: VirtualAlloc
FILE TYPES: PE

PATTERN NAME: PE_API_CREATEREMOTETHREAD
DESCRIPTION: Detects CreateRemoteThread API usage
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: CreateRemoteThread
FILE TYPES: PE

PATTERN NAME: PE_API_WRITEPROCESSMEMORY
DESCRIPTION: Detects WriteProcessMemory API usage
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: WriteProcessMemory
FILE TYPES: PE

PATTERN NAME: PE_UPX0_SECTION
DESCRIPTION: Detects UPX packed executable section UPX0
OFFSET: anywhere
HEX PATTERN: 55 50 58 30
STRING PATTERN:
FILE TYPES: PE

PATTERN NAME: PE_UPX1_SECTION
DESCRIPTION: Detects UPX packed executable section UPX1
OFFSET: anywhere
HEX PATTERN: 55 50 58 31
STRING PATTERN:
FILE TYPES: PE

PATTERN NAME: PE_SECTION_ASPACK
DESCRIPTION: Detects ASPack packed section
OFFSET: anywhere
HEX PATTERN: 2E 61 73 70 61 63 6B
STRING PATTERN:
FILE TYPES: PE

PATTERN NAME: PE_SECTION_ADATA
DESCRIPTION: Detects suspicious section name adata
OFFSET: anywhere
HEX PATTERN: 2E 61 64 61 74 61
STRING PATTERN:
FILE TYPES: PE

PATTERN NAME: PE_SECTION_VMP0

DESCRIPTION: Detects VMProtect packed section
OFFSET: anywhere
HEX PATTERN: 2E 76 6D 70 30
STRING PATTERN:
FILE TYPES: PE

PATTERN NAME: PDF_HEADER
DESCRIPTION: Detects PDF document header
OFFSET: 0
HEX PATTERN: 25 50 44 46 2D
STRING PATTERN:
FILE TYPES: PDF

PATTERN NAME: PDF_JAVASCRIPT
DESCRIPTION: Detects embedded JavaScript in PDF
OFFSET: anywhere
HEX PATTERN: 2F 4A 61 76 61 53 63 72 69 70 74
STRING PATTERN:
FILE TYPES: PDF

PATTERN NAME: PDF_OPENACTION
DESCRIPTION: Detects automatic open action in PDF
OFFSET: anywhere
HEX PATTERN: 2F 4F 70 65 6E 41 63 74 69 6F 6E
STRING PATTERN:
FILE TYPES: PDF

PATTERN NAME: PDF_LAUNCH
DESCRIPTION: Detects launch action in PDF
OFFSET: anywhere
HEX PATTERN: 2F 4C 61 75 6E 63 68
STRING PATTERN:
FILE TYPES: PDF

PATTERN NAME: PDF_EMBEDDEDFILE
DESCRIPTION: Detects embedded file in PDF
OFFSET: anywhere
HEX PATTERN: 2F 45 6D 62 65 64 64 65 64 46 69 6C 65
STRING PATTERN:
FILE TYPES: PDF

PATTERN NAME: OFFICE_OLE_HEADER
DESCRIPTION: Detects legacy Office OLE document
OFFSET: 0
HEX PATTERN: D0 CF 11 E0 A1 B1 1A E1
STRING PATTERN:
FILE TYPES: Office

PATTERN NAME: OFFICE_ZIP_HEADER
DESCRIPTION: Detects modern Office OOXML document
OFFSET: 0
HEX PATTERN: 50 4B 03 04
STRING PATTERN:
FILE TYPES: Office

PATTERN NAME: OFFICE_AUTOOPEN
DESCRIPTION: Detects Auto_Open macro
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: Auto_Open
FILE TYPES: Office

PATTERN NAME: OFFICE_DOCUMENTOPEN
DESCRIPTION: Detects Document_Open macro
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: Document_Open
FILE TYPES: Office

PATTERN NAME: OFFICE_WSCRIPT_SHELL
DESCRIPTION: Detects WScript.Shell usage
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: WScript.Shell
FILE TYPES: Office

PATTERN NAME: OFFICE_POWERSHELL
DESCRIPTION: Detects PowerShell execution from Office
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: powershell.exe
FILE TYPES: Office

PATTERN NAME: JS_EVAL
DESCRIPTION: Detects JavaScript eval execution
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: eval(
FILE TYPES: JS

PATTERN NAME: JS_UNESCAPE
DESCRIPTION: Detects JavaScript unescape usage
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: unescape(
FILE TYPES: JS

PATTERN NAME: JS_ACTIVEXOBJECT
DESCRIPTION: Detects ActiveXObject usage
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: ActiveXObject
FILE TYPES: JS

PATTERN NAME: PS_EXEC_BYPASS
DESCRIPTION: Detects PowerShell ExecutionPolicy bypass
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: -ExecutionPolicy Bypass
FILE TYPES: PS

PATTERN NAME: PS_ENCODED_COMMAND
DESCRIPTION: Detects encoded PowerShell command
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: -EncodedCommand
FILE TYPES: PS

PATTERN NAME: PERSIST_RUNKEY
DESCRIPTION: Detects registry Run key persistence
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: CurrentVersion\Run
FILE TYPES: Any

PATTERN NAME: PERSIST_SCHTASKS
DESCRIPTION: Detects scheduled task persistence
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: schtasks /create
FILE TYPES: Any

PATTERN NAME: IOC_HTTP
DESCRIPTION: Detects HTTP URL
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: http://
FILE TYPES: Any

PATTERN NAME: IOC_HTTPS
DESCRIPTION: Detects HTTPS URL
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: https://

FILE TYPES: Any

F.11 ai_deepseek_patterns.txt

PATTERN NAME: PE_MZ_HEADER
DESCRIPTION: Detects Windows PE executables by their MZ header at file beginning
OFFSET: 0
HEX PATTERN: 4D 5A
STRING PATTERN:
FILE TYPES: PE

PATTERN NAME: PE_DOS_MESSAGE
DESCRIPTION: Detects the standard DOS stub message in PE files
OFFSET: anywhere
HEX PATTERN: 54 68 69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F 74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20 6D 6F 64 65
STRING PATTERN: This program cannot be run in DOS mode
FILE TYPES: PE

PATTERN NAME: PE_API_VIRTUALALLOC
DESCRIPTION: Detects VirtualAlloc API import often used for memory allocation in malware
OFFSET: anywhere
HEX PATTERN: 56 69 72 74 75 61 6C 41 6C 6C 6F 63
STRING PATTERN: VirtualAlloc
FILE TYPES: PE

PATTERN NAME: PE_API_CREATEREMOTETHREAD
DESCRIPTION: Detects CreateRemoteThread API used for process injection
OFFSET: anywhere
HEX PATTERN: 43 72 65 61 74 65 52 65 6D 6F 74 65 54 68 72 65 61 64
STRING PATTERN: CreateRemoteThread
FILE TYPES: PE

PATTERN NAME: PE_API_WRITEPROCESSMEMORY
DESCRIPTION: Detects WriteProcessMemory API used for process injection
OFFSET: anywhere
HEX PATTERN: 57 72 69 74 65 50 72 6F 63 65 73 73 4D 65 6D 6F 72 79
STRING PATTERN: WriteProcessMemory
FILE TYPES: PE

PATTERN NAME: PE_UPX0_SECTION
DESCRIPTION: Detects UPX packed executables by section name UPX0
OFFSET: anywhere
HEX PATTERN: 55 50 58 30
STRING PATTERN: UPX0
FILE TYPES: PE

PATTERN NAME: PE_UPX1_SECTION
DESCRIPTION: Detects UPX packed executables by section name UPX1

OFFSET: anywhere
HEX PATTERN: 55 50 58 31
STRING PATTERN: UPX1
FILE TYPES: PE

PATTERN NAME: PE_ASPACK_SECTION
DESCRIPTION: Detects ASPack packed executables by section name .aspack
OFFSET: anywhere
HEX PATTERN: 2E 61 73 70 61 63 6B
STRING PATTERN: .aspack
FILE TYPES: PE

PATTERN NAME: PE_ADATA_SECTION
DESCRIPTION: Detects ASPack packed executables by section name .adata
OFFSET: anywhere
HEX PATTERN: 2E 61 64 61 74 61
STRING PATTERN: .adata
FILE TYPES: PE

PATTERN NAME: PE_VMP0_SECTION
DESCRIPTION: Detects VMProtect packed executables by section name .vmp0
OFFSET: anywhere
HEX PATTERN: 2E 76 6D 70 30
STRING PATTERN: .vmp0
FILE TYPES: PE

PATTERN NAME: PDF_HEADER
DESCRIPTION: Detects PDF files by their header
OFFSET: 0
HEX PATTERN: 25 50 44 46
STRING PATTERN: %PDF
FILE TYPES: PDF

PATTERN NAME: PDF_JAVASCRIPT_TAG
DESCRIPTION: Detects JavaScript in PDF documents
OFFSET: anywhere
HEX PATTERN: 2F 4A 61 76 61 53 63 72 69 70 74
STRING PATTERN: /JavaScript
FILE TYPES: PDF

PATTERN NAME: PDF_JS_TAG
DESCRIPTION: Detects JavaScript in PDF documents (short form)
OFFSET: anywhere
HEX PATTERN: 2F 4A 53
STRING PATTERN: /JS
FILE TYPES: PDF

PATTERN NAME: PDF_OPENACTION_TAG

DESCRIPTION: Detects auto-execute actions in PDF
OFFSET: anywhere
HEX PATTERN: 2F 4F 70 65 6E 41 63 74 69 6F 6E
STRING PATTERN: /OpenAction
FILE TYPES: PDF

PATTERN NAME: PDF_LAUNCH_TAG
DESCRIPTION: Detects launch actions in PDF
OFFSET: anywhere
HEX PATTERN: 2F 4C 61 75 6E 63 68
STRING PATTERN: /Launch
FILE TYPES: PDF

PATTERN NAME: PDF_EMBEDDEDFILE_TAG
DESCRIPTION: Detects embedded files in PDF
OFFSET: anywhere
HEX PATTERN: 2F 45 6D 62 65 64 64 65 64 46 69 6C 65
STRING PATTERN: /EmbeddedFile
FILE TYPES: PDF

PATTERN NAME: PDF_EVAL_STRING
DESCRIPTION: Detects eval() in PDF JavaScript
OFFSET: anywhere
HEX PATTERN: 65 76 61 6C 28
STRING PATTERN: eval(
FILE TYPES: PDF

PATTERN NAME: OFFICE_OLE_HEADER
DESCRIPTION: Detects legacy Office documents (OLE2 format)
OFFSET: 0
HEX PATTERN: D0 CF 11 E0 A1 B1 1A E1
STRING PATTERN:
FILE TYPES: Office

PATTERN NAME: OFFICE_PK_HEADER
DESCRIPTION: Detects modern Office documents (Open XML format)
OFFSET: 0
HEX PATTERN: 50 4B 03 04
STRING PATTERN: PK
FILE TYPES: Office

PATTERN NAME: OFFICE_AUTO_OPEN
DESCRIPTION: Detects Auto_Open macro in Office documents
OFFSET: anywhere
HEX PATTERN: 41 75 74 6F 5F 4F 70 65 6E
STRING PATTERN: Auto_Open
FILE TYPES: Office

PATTERN NAME: OFFICE_DOCUMENT_OPEN
DESCRIPTION: Detects Document_Open macro in Office documents
OFFSET: anywhere
HEX PATTERN: 44 6F 63 75 6D 65 6E 74 5F 4F 70 65 6E
STRING PATTERN: Document_Open
FILE TYPES: Office

PATTERN NAME: OFFICE_WSCRIPT_SHELL
DESCRIPTION: Detects WScript.Shell object creation in Office macros
OFFSET: anywhere
HEX PATTERN: 57 53 63 72 69 70 74 2E 53 68 65 6C 6C
STRING PATTERN: WScript.Shell
FILE TYPES: Office

PATTERN NAME: OFFICE_CREATEOBJECT
DESCRIPTION: Detects CreateObject function in Office macros
OFFSET: anywhere
HEX PATTERN: 43 72 65 61 74 65 4F 62 6A 65 63 74
STRING PATTERN: CreateObject
FILE TYPES: Office

PATTERN NAME: OFFICE_SHELL_APPLICATION
DESCRIPTION: Detects Shell.Application object in Office macros
OFFSET: anywhere
HEX PATTERN: 53 68 65 6C 6C 2E 41 70 70 6C 69 63 61 74 69 6F 6E
STRING PATTERN: Shell.Application
FILE TYPES: Office

PATTERN NAME: OFFICE_POWERSHELL_EXE
DESCRIPTION: Detects PowerShell execution from Office macros
OFFSET: anywhere
HEX PATTERN: 70 6F 77 65 72 73 68 65 6C 6C 2E 65 78 65
STRING PATTERN: powershell.exe
FILE TYPES: Office

PATTERN NAME: OFFICE_CMD_EXE
DESCRIPTION: Detects cmd.exe execution from Office macros
OFFSET: anywhere
HEX PATTERN: 63 6D 64 2E 65 78 65 20 2F 63
STRING PATTERN: cmd.exe /c
FILE TYPES: Office

PATTERN NAME: JS_EVAL
DESCRIPTION: Detects eval() function in JavaScript (obfuscation)
OFFSET: anywhere
HEX PATTERN: 65 76 61 6C 28
STRING PATTERN: eval(
FILE TYPES: JS

PATTERN NAME: JS_UNESCAPE
DESCRIPTION: Detects unescape() function in JavaScript (obfuscation)
OFFSET: anywhere
HEX PATTERN: 75 6E 65 73 63 61 70 65 28
STRING PATTERN: unescape(
FILE TYPES: JS

PATTERN NAME: JS_DECODEURICOMPONENT
DESCRIPTION: Detects decodeURIComponent() function in JavaScript (obfuscation)
OFFSET: anywhere
HEX PATTERN: 64 65 63 6F 64 65 55 52 49 43 6F 6D 70 6F 6E 65 6E 74 28
STRING PATTERN: decodeURIComponent(
FILE TYPES: JS

PATTERN NAME: JS_ACTIVEXOBJECT
DESCRIPTION: Detects ActiveXObject creation in JavaScript
OFFSET: anywhere
HEX PATTERN: 41 63 74 69 76 65 58 4F 62 6A 65 63 74
STRING PATTERN: ActiveXObject
FILE TYPES: JS

PATTERN NAME: JS_WSCRIPT_SHELL
DESCRIPTION: Detects WScript.Shell in JavaScript
OFFSET: anywhere
HEX PATTERN: 57 53 63 72 69 70 74 2E 53 68 65 6C 6C
STRING PATTERN: WScript.Shell
FILE TYPES: JS

PATTERN NAME: JS_XMLHTTPREQUEST
DESCRIPTION: Detects XMLHttpRequest for network requests
OFFSET: anywhere
HEX PATTERN: 58 4D 4C 48 74 74 70 52 65 71 75 65 73 74
STRING PATTERN: XMLHttpRequest
FILE TYPES: JS

PATTERN NAME: JS_ADODB_STREAM
DESCRIPTION: Detects ADODB.Stream for file operations
OFFSET: anywhere
HEX PATTERN: 41 44 4F 44 42 2E 53 74 72 65 61 6D
STRING PATTERN: ADODB.Stream
FILE TYPES: JS

PATTERN NAME: PS_EXECUTIONPOLICY_BYPASS
DESCRIPTION: Detects PowerShell execution policy bypass
OFFSET: anywhere
HEX PATTERN: 2D 45 78 65 63 75 74 69 6F 6E 50 6F 6C 69 63 79 20 42 79 70 61 73 73
STRING PATTERN: -ExecutionPolicy Bypass

FILE TYPES: PS

PATTERN NAME: PS_WINDOWSTYLE_HIDDEN
DESCRIPTION: Detects hidden PowerShell window execution
OFFSET: anywhere
HEX PATTERN: 2D 57 69 6E 64 6F 77 53 74 79 6C 65 20 48 69 64 64 65 6E
STRING PATTERN: -WindowStyle Hidden
FILE TYPES: PS

PATTERN NAME: PS_DOWNLOADSTRING
DESCRIPTION: Detects DownloadString method for payload retrieval
OFFSET: anywhere
HEX PATTERN: 44 6F 77 6E 6C 6F 61 64 53 74 72 69 6E 67
STRING PATTERN: DownloadString
FILE TYPES: PS

PATTERN NAME: PS_INVOKE_WEBREQUEST
DESCRIPTION: Detects Invoke-WebRequest cmdlet for downloading
OFFSET: anywhere
HEX PATTERN: 49 6E 76 6F 6B 65 2D 57 65 62 52 65 71 75 65 73 74
STRING PATTERN: Invoke-WebRequest
FILE TYPES: PS

PATTERN NAME: PS_ENCODEDCOMMAND
DESCRIPTION: Detects encoded PowerShell commands
OFFSET: anywhere
HEX PATTERN: 2D 45 6E 63 6F 64 65 64 43 6F 6D 6D 61 6E 64
STRING PATTERN: -EncodedCommand
FILE TYPES: PS

PATTERN NAME: PS_ENC_SHORT
DESCRIPTION: Detects short form of encoded command
OFFSET: anywhere
HEX PATTERN: 2D 65 6E 63
STRING PATTERN: -enc
FILE TYPES: PS

PATTERN NAME: PS_NET_WEBCLIENT
DESCRIPTION: Detects System.Net.WebClient for downloading
OFFSET: anywhere
HEX PATTERN: 53 79 73 74 65 6D 2E 4E 65 74 2E 57 65 62 43 6C 69 65 6E 74
STRING PATTERN: System.Net.WebClient
FILE TYPES: PS

PATTERN NAME: BACKDOOR_REGISTRY_RUN
DESCRIPTION: Detects registry run key persistence
OFFSET: anywhere
HEX PATTERN: 43 75 72 72 65 6E 74 56 65 72 73 69 6F 6E 5C 5C 52 75 6E

STRING PATTERN: CurrentVersion\\Run
FILE TYPES: Any

PATTERN NAME: BACKDOOR_SCHTASKS_CREATE
DESCRIPTION: Detects scheduled task creation for persistence
OFFSET: anywhere
HEX PATTERN: 73 63 68 74 61 73 6B 73 20 2F 63 72 65 61 74 65
STRING PATTERN: schtasks /create
FILE TYPES: Any

PATTERN NAME: BACKDOOR_STARTUP_FOLDER
DESCRIPTION: Detects startup folder persistence
OFFSET: anywhere
HEX PATTERN: 53 74 61 72 74 75 70 5C 5C 50 72 6F 67 72 61 6D 73
STRING PATTERN: Startup\\Programs
FILE TYPES: Any

PATTERN NAME: BACKDOOR_HTTP_URL
DESCRIPTION: Detects HTTP URLs for C2 communication
OFFSET: anywhere
HEX PATTERN: 68 74 74 70 3A 2F 2F
STRING PATTERN: http://
FILE TYPES: Any

PATTERN NAME: BACKDOOR_HTTPS_URL
DESCRIPTION: Detects HTTPS URLs for C2 communication
OFFSET: anywhere
HEX PATTERN: 68 74 74 70 73 3A 2F 2F
STRING PATTERN: https://
FILE TYPES: Any

PATTERN NAME: BACKDOOR_CMD_SHELL
DESCRIPTION: Detects cmd.exe execution for backdoor commands
OFFSET: anywhere
HEX PATTERN: 63 6D 64 2E 65 78 65
STRING PATTERN: cmd.exe
FILE TYPES: Any

PATTERN NAME: BACKDOOR_POWERSHELL
DESCRIPTION: Detects PowerShell execution for backdoor commands
OFFSET: anywhere
HEX PATTERN: 70 6F 77 65 72 73 68 65 6C 6C 2E 65 78 65
STRING PATTERN: powershell.exe
FILE TYPES: Any

PATTERN NAME: BACKDOOR_REVERSE_SHELL
DESCRIPTION: Detects reverse shell command patterns
OFFSET: anywhere

HEX PATTERN: 72 65 76 65 72 73 65 20 73 68 65 6C 6C
STRING PATTERN: reverse shell
FILE TYPES: Any

PATTERN NAME: COMBINED_BACKDOOR_PERSISTENCE
DESCRIPTION: Detects multiple backdoor persistence mechanisms
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN:
FILE TYPES: Any
NOTE: This is a meta-pattern - the scanner should check for any of the backdoor patterns above

F.12 ai_gemini_patterns.txt

1. PE EXECUTABLES
PATTERN NAME: PE_MZ_HEADER
DESCRIPTION: Standard Portable Executable (MZ) header
OFFSET: 0
HEX PATTERN: 4D 5A
STRING PATTERN: MZ
FILE TYPES: PE

PATTERN NAME: PE_DOS_STUB
DESCRIPTION: Standard DOS stub message in Windows binaries
OFFSET: anywhere
HEX PATTERN: 54 68 69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F 74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20 6D 6F 64 65
STRING PATTERN: This program cannot be run in DOS mode
FILE TYPES: PE

PATTERN NAME: PE_INJECTION_APIS
DESCRIPTION: APIs used for process injection and code execution
OFFSET: anywhere
STRING PATTERN: VirtualAlloc, CreateRemoteThread, WriteProcessMemory
FILE TYPES: PE

PATTERN NAME: PE_UPX_MARKERS
DESCRIPTION: UPX packer section markers (common in compressed malware)
OFFSET: anywhere
HEX PATTERN: 55 50 58 30, 55 50 58 31
STRING PATTERN: UPX0, UPX1
FILE TYPES: PE

PATTERN NAME: PE_SUSPICIOUS_SECTIONS
DESCRIPTION: Sections used by ASPack, VMProtect, or custom crypters
OFFSET: anywhere
STRING PATTERN: .aspack, .adata, .vmp0
FILE TYPES: PE

2. PDF DOCUMENTS
PATTERN NAME: PDF_HEADER
DESCRIPTION: Standard PDF magic bytes
OFFSET: 0
HEX PATTERN: 25 50 44 46
STRING PATTERN: %PDF
FILE TYPES: PDF

PATTERN NAME: PDF_JS_OBJECTS
DESCRIPTION: Embedded JavaScript indicators
OFFSET: anywhere
STRING PATTERN: /JavaScript, /JS
FILE TYPES: PDF

PATTERN NAME: PDF_AUTO_EXECUTION
DESCRIPTION: Tags that trigger actions upon opening the file
OFFSET: anywhere
STRING PATTERN: /OpenAction, /Launch
FILE TYPES: PDF

PATTERN NAME: PDF_EMBEDDED_FILE
DESCRIPTION: Presence of an embedded file stream (dropper logic)
OFFSET: anywhere
STRING PATTERN: /EmbeddedFile
FILE TYPES: PDF

PATTERN NAME: PDF_EVAL_JS
DESCRIPTION: JavaScript eval function inside a PDF stream
OFFSET: anywhere
STRING PATTERN: eval(
FILE TYPES: PDF

3. OFFICE DOCUMENTS
PATTERN NAME: OFFICE_OLE_HEADER
DESCRIPTION: Legacy OLE2 header (doc, xls, ppt)
OFFSET: 0
HEX PATTERN: D0 CF 11 E0 A1 B1 1A E1
FILE TYPES: Office

PATTERN NAME: OFFICE_PK_HEADER
DESCRIPTION: Modern OOXML header (docx, xlsx, pptx)
OFFSET: 0
HEX PATTERN: 50 4B 03 04
STRING PATTERN: PK
FILE TYPES: Office

PATTERN NAME: OFFICE_AUTO_MACRO
DESCRIPTION: VBA macros that execute automatically on open

OFFSET: anywhere
STRING PATTERN: Auto_Open, Document_Open
FILE TYPES: Office

PATTERN NAME: OFFICE_MALICIOUS_COM
DESCRIPTION: COM objects used for shell execution
OFFSET: anywhere
STRING PATTERN: WScript.Shell, CreateObject, Shell.Application
FILE TYPES: Office

PATTERN NAME: OFFICE_SHELL_SPAWN
DESCRIPTION: Macro-based execution of system shells
OFFSET: anywhere
STRING PATTERN: powershell.exe, cmd.exe /c
FILE TYPES: Office

4. JAVASCRIPT
PATTERN NAME: JS_OBFUSCATION_FUNCS
DESCRIPTION: Functions frequently used to decode malicious payloads
OFFSET: anywhere
STRING PATTERN: eval(, unescape(, decodeURIComponent(
FILE TYPES: JS

PATTERN NAME: JS_SHELL_EXECUTION
DESCRIPTION: ActiveX/WScript access for system interaction
OFFSET: anywhere
STRING PATTERN: ActiveXObject, WScript.Shell
FILE TYPES: JS

PATTERN NAME: JS_NETWORK_DOWNLOADER
DESCRIPTION: AJAX or Stream objects used to download/save files
OFFSET: anywhere
STRING PATTERN: XMLHttpRequest, ADODB.Stream
FILE TYPES: JS

5. POWERSHELL
PATTERN NAME: PS_STEALTH_FLAGS
DESCRIPTION: CLI flags to hide execution and bypass policy
OFFSET: anywhere
STRING PATTERN: -ExecutionPolicy Bypass, -WindowStyle Hidden
FILE TYPES: PS

PATTERN NAME: PS_DOWNLOAD_CRADLE
DESCRIPTION: Methods for downloading remote scripts into memory
OFFSET: anywhere
STRING PATTERN: DownloadString, Invoke-WebRequest, System.Net.WebClient
FILE TYPES: PS

PATTERN NAME: PS_ENCODED_CMD
DESCRIPTION: Base64 encoded command flag
OFFSET: anywhere
STRING PATTERN: -EncodedCommand, -enc
FILE TYPES: PS

6. BACKDOOR PERSISTENCE
PATTERN NAME: PERSISTENCE_RUN_KEY
DESCRIPTION: Windows Registry Run key for boot persistence
OFFSET: anywhere
STRING PATTERN: CurrentVersion\Run
FILE TYPES: Any

PATTERN NAME: PERSISTENCE_SCHTASKS
DESCRIPTION: Command to create scheduled tasks
OFFSET: anywhere
STRING PATTERN: schtasks /create
FILE TYPES: Any

PATTERN NAME: PERSISTENCE_STARTUP_DIR
DESCRIPTION: Path to the Windows Startup folder
OFFSET: anywhere
STRING PATTERN: Microsoft\Windows\Start Menu\Programs\Startup
FILE TYPES: Any

PATTERN NAME: NETWORK_C2_URL
DESCRIPTION: Plaintext network protocols indicative of C2 or exfiltration
OFFSET: anywhere
STRING PATTERN: http://, https://
FILE TYPES: Any

F.13 ai_copilot_patterns.txt

PATTERN NAME: PE_MZ_Header
DESCRIPTION: Detects Windows PE executables by MZ header
OFFSET: 0
HEX PATTERN: 4D 5A
STRING PATTERN: N/A
FILE TYPES: PE

PATTERN NAME: PE_DOS_Mode_String
DESCRIPTION: Detects DOS stub in PE files
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: This program cannot be run in DOS mode
FILE TYPES: PE

PATTERN NAME: PE_Malicious_APIs
DESCRIPTION: Detects suspicious API usage in PE files

OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: VirtualAlloc|CreateRemoteThread|WriteProcessMemory
FILE TYPES: PE

PATTERN NAME: PE_UPX_Packed
DESCRIPTION: Detects UPX packed executables
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: UPX0|UPX1
FILE TYPES: PE

PATTERN NAME: PE_Suspicious_Sections
DESCRIPTION: Detects suspicious section names in PE files
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: .aspack|.adata|.vmp0
FILE TYPES: PE

PATTERN NAME: PDF_Header
DESCRIPTION: Detects PDF documents by header
OFFSET: 0
HEX PATTERN: 25 50 44 46
STRING PATTERN: N/A
FILE TYPES: PDF

PATTERN NAME: PDF_JavaScript_Tags
DESCRIPTION: Detects embedded JavaScript in PDFs
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: /JavaScript|/JS
FILE TYPES: PDF

PATTERN NAME: PDF_Actions
DESCRIPTION: Detects auto actions in PDFs
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: /OpenAction|/Launch
FILE TYPES: PDF

PATTERN NAME: PDF_EmbeddedFile
DESCRIPTION: Detects embedded files in PDFs
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: /EmbeddedFile
FILE TYPES: PDF

PATTERN NAME: PDF_Eval_String

DESCRIPTION: Detects suspicious eval usage in PDFs
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: eval(
FILE TYPES: PDF

PATTERN NAME: Office_OLE_Header
DESCRIPTION: Detects old Office documents by OLE header
OFFSET: 0
HEX PATTERN: D0 CF 11 E0
STRING PATTERN: N/A
FILE TYPES: Office

PATTERN NAME: Office_PK_Header
DESCRIPTION: Detects new Office documents by PK header
OFFSET: 0
HEX PATTERN: 50 4B 03 04
STRING PATTERN: N/A
FILE TYPES: Office

PATTERN NAME: Office_AutoMacros
DESCRIPTION: Detects auto-executing macros in Office docs
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: Auto_Open|Document_Open
FILE TYPES: Office

PATTERN NAME: Office_Shell_Objects
DESCRIPTION: Detects suspicious object creation in Office docs
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: WScript.Shell|CreateObject|Shell.Application
FILE TYPES: Office

PATTERN NAME: Office_Command_Exec
DESCRIPTION: Detects command execution in Office docs
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: powershell.exe|cmd.exe /c
FILE TYPES: Office

PATTERN NAME: JS_Eval_Unescape
DESCRIPTION: Detects obfuscation functions in JavaScript
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: eval(|unescape(|decodeURIComponent(
FILE TYPES: JS

PATTERN NAME: JS_ActiveX_WScript
DESCRIPTION: Detects ActiveX and WScript usage in JavaScript
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: ActiveXObject|WScript.Shell
FILE TYPES: JS

PATTERN NAME: JS_Network_IO
DESCRIPTION: Detects network and file IO in JavaScript
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: XMLHttpRequest|ADODB.Stream
FILE TYPES: JS

PATTERN NAME: PS_Execution_Bypass
DESCRIPTION: Detects execution policy bypass in PowerShell
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: -ExecutionPolicy Bypass|-WindowStyle Hidden
FILE TYPES: PS

PATTERN NAME: PS_Download_Commands
DESCRIPTION: Detects download commands in PowerShell
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: DownloadString|Invoke-WebRequest
FILE TYPES: PS

PATTERN NAME: PS_EncodedCommand
DESCRIPTION: Detects encoded command usage in PowerShell
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: -EncodedCommand|-enc
FILE TYPES: PS

PATTERN NAME: PS_WebClient
DESCRIPTION: Detects WebClient usage in PowerShell
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: System.Net.WebClient
FILE TYPES: PS

PATTERN NAME: Persistence_Registry_Run
DESCRIPTION: Detects registry run key persistence
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: CurrentVersion\\Run
FILE TYPES: Any

PATTERN NAME: Persistence_Schtasks
DESCRIPTION: Detects scheduled task persistence
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: schtasks /create
FILE TYPES: Any

PATTERN NAME: Persistence_Startup_Folder
DESCRIPTION: Detects startup folder persistence
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: \\Startup\\
FILE TYPES: Any

PATTERN NAME: Persistence_HTTP_URLs
DESCRIPTION: Detects hardcoded URLs in persistence mechanisms
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: http://|https://
FILE TYPES: Any

F.14 ai_combined_patterns.txt

PATTERN NAME: PE_MZ_HEADER_O0
DESCRIPTION: Detects Windows PE files by MZ signature
OFFSET: 0
HEX PATTERN: 4D 5A
STRING PATTERN:
FILE TYPES: PE

PATTERN NAME: PE_DOS_STUB_CANNOT_RUN
DESCRIPTION: Detects PE DOS stub message
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: This program cannot be run in DOS mode
FILE TYPES: PE

PATTERN NAME: PE_API_VIRTUALALLOC
DESCRIPTION: Detects VirtualAlloc API usage
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: VirtualAlloc
FILE TYPES: PE

PATTERN NAME: PE_API_CREATEREMOTETHREAD
DESCRIPTION: Detects CreateRemoteThread API usage
OFFSET: anywhere
HEX PATTERN:

STRING PATTERN: CreateRemoteThread
FILE TYPES: PE

PATTERN NAME: PE_API_WRITEPROCESSMEMORY
DESCRIPTION: Detects WriteProcessMemory API usage
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: WriteProcessMemory
FILE TYPES: PE

PATTERN NAME: PE_UPX0_SECTION
DESCRIPTION: Detects UPX packed executable section UPX0
OFFSET: anywhere
HEX PATTERN: 55 50 58 30
STRING PATTERN:
FILE TYPES: PE

PATTERN NAME: PE_UPX1_SECTION
DESCRIPTION: Detects UPX packed executable section UPX1
OFFSET: anywhere
HEX PATTERN: 55 50 58 31
STRING PATTERN:
FILE TYPES: PE

PATTERN NAME: PE_SECTION_ASPACK
DESCRIPTION: Detects ASPack packed section
OFFSET: anywhere
HEX PATTERN: 2E 61 73 70 61 63 6B
STRING PATTERN:
FILE TYPES: PE

PATTERN NAME: PE_SECTION_ADATA
DESCRIPTION: Detects suspicious section name adata
OFFSET: anywhere
HEX PATTERN: 2E 61 64 61 74 61
STRING PATTERN:
FILE TYPES: PE

PATTERN NAME: PE_SECTION_VMP0
DESCRIPTION: Detects VMProtect packed section
OFFSET: anywhere
HEX PATTERN: 2E 76 6D 70 30
STRING PATTERN:
FILE TYPES: PE

PATTERN NAME: PDF_HEADER
DESCRIPTION: Detects PDF document header
OFFSET: 0

HEX PATTERN: 25 50 44 46 2D
STRING PATTERN:
FILE TYPES: PDF

PATTERN NAME: PDF_JAVASCRIPT
DESCRIPTION: Detects embedded JavaScript in PDF
OFFSET: anywhere
HEX PATTERN: 2F 4A 61 76 61 53 63 72 69 70 74
STRING PATTERN:
FILE TYPES: PDF

PATTERN NAME: PDF_OPENACTION
DESCRIPTION: Detects automatic open action in PDF
OFFSET: anywhere
HEX PATTERN: 2F 4F 70 65 6E 41 63 74 69 6F 6E
STRING PATTERN:
FILE TYPES: PDF

PATTERN NAME: PDF_LAUNCH
DESCRIPTION: Detects launch action in PDF
OFFSET: anywhere
HEX PATTERN: 2F 4C 61 75 6E 63 68
STRING PATTERN:
FILE TYPES: PDF

PATTERN NAME: PDF_EMBEDDEDFILE
DESCRIPTION: Detects embedded file in PDF
OFFSET: anywhere
HEX PATTERN: 2F 45 6D 62 65 64 64 65 64 46 69 6C 65
STRING PATTERN:
FILE TYPES: PDF

PATTERN NAME: OFFICE_OLE_HEADER
DESCRIPTION: Detects legacy Office OLE document
OFFSET: 0
HEX PATTERN: D0 CF 11 E0 A1 B1 1A E1
STRING PATTERN:
FILE TYPES: Office

PATTERN NAME: OFFICE_ZIP_HEADER
DESCRIPTION: Detects modern Office OOXML document
OFFSET: 0
HEX PATTERN: 50 4B 03 04
STRING PATTERN:
FILE TYPES: Office

PATTERN NAME: OFFICE_AUTOOPEN
DESCRIPTION: Detects Auto_Open macro

OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: Auto_Open
FILE TYPES: Office

PATTERN NAME: OFFICE_DOCUMENTOPEN
DESCRIPTION: Detects Document_Open macro
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: Document_Open
FILE TYPES: Office

PATTERN NAME: OFFICE_WSCRIPT_SHELL
DESCRIPTION: Detects WScript.Shell usage
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: WScript.Shell
FILE TYPES: Office

PATTERN NAME: OFFICE_POWERSHELL
DESCRIPTION: Detects PowerShell execution from Office
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: powershell.exe
FILE TYPES: Office

PATTERN NAME: JS_EVAL
DESCRIPTION: Detects JavaScript eval execution
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: eval(
FILE TYPES: JS

PATTERN NAME: JS_UNESCAPE
DESCRIPTION: Detects JavaScript unescape usage
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: unescape(
FILE TYPES: JS

PATTERN NAME: JS_ACTIVEXOBJECT
DESCRIPTION: Detects ActiveXObject usage
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: ActiveXObject
FILE TYPES: JS

PATTERN NAME: PS_EXEC_BYPASS

DESCRIPTION: Detects PowerShell ExecutionPolicy bypass
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: -ExecutionPolicy Bypass
FILE TYPES: PS

PATTERN NAME: PS_ENCODED_COMMAND
DESCRIPTION: Detects encoded PowerShell command
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: -EncodedCommand
FILE TYPES: PS

PATTERN NAME: PERSIST_RUNKEY
DESCRIPTION: Detects registry Run key persistence
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: CurrentVersion\Run
FILE TYPES: Any

PATTERN NAME: PERSIST_SCHTASKS
DESCRIPTION: Detects scheduled task persistence
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: schtasks /create
FILE TYPES: Any

PATTERN NAME: IOC_HTTP
DESCRIPTION: Detects HTTP URL
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: http://
FILE TYPES: Any

PATTERN NAME: IOC_HTTPS
DESCRIPTION: Detects HTTPS URL
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN: https://
FILE TYPES: AnyPATTERN NAME: PE_MZ_HEADER
DESCRIPTION: Detects Windows PE executables by their MZ header at file beginning
OFFSET: 0
HEX PATTERN: 4D 5A
STRING PATTERN:
FILE TYPES: PE

PATTERN NAME: PE_DOS_MESSAGE
DESCRIPTION: Detects the standard DOS stub message in PE files

OFFSET: anywhere
HEX PATTERN: 54 68 69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F 74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20 6D 6F 64 65
STRING PATTERN: This program cannot be run in DOS mode
FILE TYPES: PE

PATTERN NAME: PE_API_VIRTUALALLOC
DESCRIPTION: Detects VirtualAlloc API import often used for memory allocation in malware
OFFSET: anywhere
HEX PATTERN: 56 69 72 74 75 61 6C 41 6C 6C 6F 63
STRING PATTERN: VirtualAlloc
FILE TYPES: PE

PATTERN NAME: PE_API_CREATEREMOTETHREAD
DESCRIPTION: Detects CreateRemoteThread API used for process injection
OFFSET: anywhere
HEX PATTERN: 43 72 65 61 74 65 52 65 6D 6F 74 65 54 68 72 65 61 64
STRING PATTERN: CreateRemoteThread
FILE TYPES: PE

PATTERN NAME: PE_API_WRITEPROCESSMEMORY
DESCRIPTION: Detects WriteProcessMemory API used for process injection
OFFSET: anywhere
HEX PATTERN: 57 72 69 74 65 50 72 6F 63 65 73 73 4D 65 6D 6F 72 79
STRING PATTERN: WriteProcessMemory
FILE TYPES: PE

PATTERN NAME: PE_UPX0_SECTION
DESCRIPTION: Detects UPX packed executables by section name UPX0
OFFSET: anywhere
HEX PATTERN: 55 50 58 30
STRING PATTERN: UPX0
FILE TYPES: PE

PATTERN NAME: PE_UPX1_SECTION
DESCRIPTION: Detects UPX packed executables by section name UPX1
OFFSET: anywhere
HEX PATTERN: 55 50 58 31
STRING PATTERN: UPX1
FILE TYPES: PE

PATTERN NAME: PE_ASPACK_SECTION
DESCRIPTION: Detects ASPack packed executables by section name .aspack
OFFSET: anywhere
HEX PATTERN: 2E 61 73 70 61 63 6B
STRING PATTERN: .aspack
FILE TYPES: PE

PATTERN NAME: PE_ADATA_SECTION
DESCRIPTION: Detects ASPack packed executables by section name .adata
OFFSET: anywhere
HEX PATTERN: 2E 61 64 61 74 61
STRING PATTERN: .adata
FILE TYPES: PE

PATTERN NAME: PE_VMP0_SECTION
DESCRIPTION: Detects VMProtect packed executables by section name .vmp0
OFFSET: anywhere
HEX PATTERN: 2E 76 6D 70 30
STRING PATTERN: .vmp0
FILE TYPES: PE

PATTERN NAME: PDF_HEADER
DESCRIPTION: Detects PDF files by their header
OFFSET: 0
HEX PATTERN: 25 50 44 46
STRING PATTERN: %PDF
FILE TYPES: PDF

PATTERN NAME: PDF_JAVASCRIPT_TAG
DESCRIPTION: Detects JavaScript in PDF documents
OFFSET: anywhere
HEX PATTERN: 2F 4A 61 76 61 53 63 72 69 70 74
STRING PATTERN: /JavaScript
FILE TYPES: PDF

PATTERN NAME: PDF_JS_TAG
DESCRIPTION: Detects JavaScript in PDF documents (short form)
OFFSET: anywhere
HEX PATTERN: 2F 4A 53
STRING PATTERN: /JS
FILE TYPES: PDF

PATTERN NAME: PDF_OPENACTION_TAG
DESCRIPTION: Detects auto-execute actions in PDF
OFFSET: anywhere
HEX PATTERN: 2F 4F 70 65 6E 41 63 74 69 6F 6E
STRING PATTERN: /OpenAction
FILE TYPES: PDF

PATTERN NAME: PDF_LAUNCH_TAG
DESCRIPTION: Detects launch actions in PDF
OFFSET: anywhere
HEX PATTERN: 2F 4C 61 75 6E 63 68
STRING PATTERN: /Launch
FILE TYPES: PDF

PATTERN NAME: PDF_EMBEDDEDFILE_TAG
DESCRIPTION: Detects embedded files in PDF
OFFSET: anywhere
HEX PATTERN: 2F 45 6D 62 65 64 64 65 64 46 69 6C 65
STRING PATTERN: /EmbeddedFile
FILE TYPES: PDF

PATTERN NAME: PDF_EVAL_STRING
DESCRIPTION: Detects eval() in PDF JavaScript
OFFSET: anywhere
HEX PATTERN: 65 76 61 6C 28
STRING PATTERN: eval(
FILE TYPES: PDF

PATTERN NAME: OFFICE_OLE_HEADER
DESCRIPTION: Detects legacy Office documents (OLE2 format)
OFFSET: 0
HEX PATTERN: D0 CF 11 E0 A1 B1 1A E1
STRING PATTERN:
FILE TYPES: Office

PATTERN NAME: OFFICE_PK_HEADER
DESCRIPTION: Detects modern Office documents (Open XML format)
OFFSET: 0
HEX PATTERN: 50 4B 03 04
STRING PATTERN: PK
FILE TYPES: Office

PATTERN NAME: OFFICE_AUTO_OPEN
DESCRIPTION: Detects Auto_Open macro in Office documents
OFFSET: anywhere
HEX PATTERN: 41 75 74 6F 5F 4F 70 65 6E
STRING PATTERN: Auto_Open
FILE TYPES: Office

PATTERN NAME: OFFICE_DOCUMENT_OPEN
DESCRIPTION: Detects Document_Open macro in Office documents
OFFSET: anywhere
HEX PATTERN: 44 6F 63 75 6D 65 6E 74 5F 4F 70 65 6E
STRING PATTERN: Document_Open
FILE TYPES: Office

PATTERN NAME: OFFICE_WSCRIPT_SHELL
DESCRIPTION: Detects WScript.Shell object creation in Office macros
OFFSET: anywhere
HEX PATTERN: 57 53 63 72 69 70 74 2E 53 68 65 6C 6C
STRING PATTERN: WScript.Shell

FILE TYPES: Office

PATTERN NAME: OFFICE_CREATEOBJECT
DESCRIPTION: Detects CreateObject function in Office macros
OFFSET: anywhere
HEX PATTERN: 43 72 65 61 74 65 4F 62 6A 65 63 74
STRING PATTERN: CreateObject
FILE TYPES: Office

PATTERN NAME: OFFICE_SHELL_APPLICATION
DESCRIPTION: Detects Shell.Application object in Office macros
OFFSET: anywhere
HEX PATTERN: 53 68 65 6C 6C 2E 41 70 70 6C 69 63 61 74 69 6F 6E
STRING PATTERN: Shell.Application
FILE TYPES: Office

PATTERN NAME: OFFICE_POWERSHELL_EXE
DESCRIPTION: Detects PowerShell execution from Office macros
OFFSET: anywhere
HEX PATTERN: 70 6F 77 65 72 73 68 65 6C 6C 2E 65 78 65
STRING PATTERN: powershell.exe
FILE TYPES: Office

PATTERN NAME: OFFICE_CMD_EXE
DESCRIPTION: Detects cmd.exe execution from Office macros
OFFSET: anywhere
HEX PATTERN: 63 6D 64 2E 65 78 65 20 2F 63
STRING PATTERN: cmd.exe /c
FILE TYPES: Office

PATTERN NAME: JS_EVAL
DESCRIPTION: Detects eval() function in JavaScript (obfuscation)
OFFSET: anywhere
HEX PATTERN: 65 76 61 6C 28
STRING PATTERN: eval(
FILE TYPES: JS

PATTERN NAME: JS_UNESCAPE
DESCRIPTION: Detects unescape() function in JavaScript (obfuscation)
OFFSET: anywhere
HEX PATTERN: 75 6E 65 73 63 61 70 65 28
STRING PATTERN: unescape(
FILE TYPES: JS

PATTERN NAME: JS_DECODEURICOMPONENT
DESCRIPTION: Detects decodeURIComponent() function in JavaScript (obfuscation)
OFFSET: anywhere
HEX PATTERN: 64 65 63 6F 64 65 55 52 49 43 6F 6D 70 6F 6E 65 6E 74 28

STRING PATTERN: decodeURIComponent(
FILE TYPES: JS

PATTERN NAME: JS_ACTIVEXOBJECT
DESCRIPTION: Detects ActiveXObject creation in JavaScript
OFFSET: anywhere
HEX PATTERN: 41 63 74 69 76 65 58 4F 62 6A 65 63 74
STRING PATTERN: ActiveXObject
FILE TYPES: JS

PATTERN NAME: JS_WSCRIPT_SHELL
DESCRIPTION: Detects WScript.Shell in JavaScript
OFFSET: anywhere
HEX PATTERN: 57 53 63 72 69 70 74 2E 53 68 65 6C 6C
STRING PATTERN: WScript.Shell
FILE TYPES: JS

PATTERN NAME: JS_XMLHTTPREQUEST
DESCRIPTION: Detects XMLHttpRequest for network requests
OFFSET: anywhere
HEX PATTERN: 58 4D 4C 48 74 74 70 52 65 71 75 65 73 74
STRING PATTERN: XMLHttpRequest
FILE TYPES: JS

PATTERN NAME: JS_ADODB_STREAM
DESCRIPTION: Detects ADODB.Stream for file operations
OFFSET: anywhere
HEX PATTERN: 41 44 4F 44 42 2E 53 74 72 65 61 6D
STRING PATTERN: ADODB.Stream
FILE TYPES: JS

PATTERN NAME: PS_EXECUTIONPOLICY_BYPASS
DESCRIPTION: Detects PowerShell execution policy bypass
OFFSET: anywhere
HEX PATTERN: 2D 45 78 65 63 75 74 69 6F 6E 50 6F 6C 69 63 79 20 42 79 70 61 73 73
STRING PATTERN: -ExecutionPolicy Bypass
FILE TYPES: PS

PATTERN NAME: PS_WINDOWSTYLE_HIDDEN
DESCRIPTION: Detects hidden PowerShell window execution
OFFSET: anywhere
HEX PATTERN: 2D 57 69 6E 64 6F 77 53 74 79 6C 65 20 48 69 64 64 65 6E
STRING PATTERN: -WindowStyle Hidden
FILE TYPES: PS

PATTERN NAME: PS_DOWNLOADSTRING
DESCRIPTION: Detects DownloadString method for payload retrieval
OFFSET: anywhere

HEX PATTERN: 44 6F 77 6E 6C 6F 61 64 53 74 72 69 6E 67
STRING PATTERN: DownloadString
FILE TYPES: PS

PATTERN NAME: PS_INVOKE_WEBREQUEST
DESCRIPTION: Detects Invoke-WebRequest cmdlet for downloading
OFFSET: anywhere
HEX PATTERN: 49 6E 76 6F 6B 65 2D 57 65 62 52 65 71 75 65 73 74
STRING PATTERN: Invoke-WebRequest
FILE TYPES: PS

PATTERN NAME: PS_ENCODEDCOMMAND
DESCRIPTION: Detects encoded PowerShell commands
OFFSET: anywhere
HEX PATTERN: 2D 45 6E 63 6F 64 65 64 43 6F 6D 6D 61 6E 64
STRING PATTERN: -EncodedCommand
FILE TYPES: PS

PATTERN NAME: PS_ENC_SHORT
DESCRIPTION: Detects short form of encoded command
OFFSET: anywhere
HEX PATTERN: 2D 65 6E 63
STRING PATTERN: -enc
FILE TYPES: PS

PATTERN NAME: PS_NET_WEBCLIENT
DESCRIPTION: Detects System.Net.WebClient for downloading
OFFSET: anywhere
HEX PATTERN: 53 79 73 74 65 6D 2E 4E 65 74 2E 57 65 62 43 6C 69 65 6E 74
STRING PATTERN: System.Net.WebClient
FILE TYPES: PS

PATTERN NAME: BACKDOOR_REGISTRY_RUN
DESCRIPTION: Detects registry run key persistence
OFFSET: anywhere
HEX PATTERN: 43 75 72 72 65 6E 74 56 65 72 73 69 6F 6E 5C 5C 52 75 6E
STRING PATTERN: CurrentVersion\\Run
FILE TYPES: Any

PATTERN NAME: BACKDOOR_SCHTASKS_CREATE
DESCRIPTION: Detects scheduled task creation for persistence
OFFSET: anywhere
HEX PATTERN: 73 63 68 74 61 73 6B 73 20 2F 63 72 65 61 74 65
STRING PATTERN: schtasks /create
FILE TYPES: Any

PATTERN NAME: BACKDOOR_STARTUP_FOLDER
DESCRIPTION: Detects startup folder persistence

OFFSET: anywhere
HEX PATTERN: 53 74 61 72 74 75 70 5C 5C 50 72 6F 67 72 61 6D 73
STRING PATTERN: Startup\\Programs
FILE TYPES: Any

PATTERN NAME: BACKDOOR_HTTP_URL
DESCRIPTION: Detects HTTP URLs for C2 communication
OFFSET: anywhere
HEX PATTERN: 68 74 74 70 3A 2F 2F
STRING PATTERN: http://
FILE TYPES: Any

PATTERN NAME: BACKDOOR_HTTPS_URL
DESCRIPTION: Detects HTTPS URLs for C2 communication
OFFSET: anywhere
HEX PATTERN: 68 74 74 70 73 3A 2F 2F
STRING PATTERN: https://
FILE TYPES: Any

PATTERN NAME: BACKDOOR_CMD_SHELL
DESCRIPTION: Detects cmd.exe execution for backdoor commands
OFFSET: anywhere
HEX PATTERN: 63 6D 64 2E 65 78 65
STRING PATTERN: cmd.exe
FILE TYPES: Any

PATTERN NAME: BACKDOOR_POWERSHELL
DESCRIPTION: Detects PowerShell execution for backdoor commands
OFFSET: anywhere
HEX PATTERN: 70 6F 77 65 72 73 68 65 6C 6C 2E 65 78 65
STRING PATTERN: powershell.exe
FILE TYPES: Any

PATTERN NAME: BACKDOOR_REVERSE_SHELL
DESCRIPTION: Detects reverse shell command patterns
OFFSET: anywhere
HEX PATTERN: 72 65 76 65 72 73 65 20 73 68 65 6C 6C
STRING PATTERN: reverse shell
FILE TYPES: Any

PATTERN NAME: COMBINED_BACKDOOR_PERSISTENCE
DESCRIPTION: Detects multiple backdoor persistence mechanisms
OFFSET: anywhere
HEX PATTERN:
STRING PATTERN:
FILE TYPES: Any
NOTE: This is a meta-pattern - the scanner should check for any of the backdoor patterns above1. PE EXECUTABLES

PATTERN NAME: PE_MZ_HEADER
DESCRIPTION: Standard Portable Executable (MZ) header
OFFSET: 0
HEX PATTERN: 4D 5A
STRING PATTERN: MZ
FILE TYPES: PE

PATTERN NAME: PE_DOS_STUB
DESCRIPTION: Standard DOS stub message in Windows binaries
OFFSET: anywhere
HEX PATTERN: 54 68 69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F 74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20 6D 6F 64 65
STRING PATTERN: This program cannot be run in DOS mode
FILE TYPES: PE

PATTERN NAME: PE_INJECTION_APIS
DESCRIPTION: APIs used for process injection and code execution
OFFSET: anywhere
STRING PATTERN: VirtualAlloc, CreateRemoteThread, WriteProcessMemory
FILE TYPES: PE

PATTERN NAME: PE_UPX_MARKERS
DESCRIPTION: UPX packer section markers (common in compressed malware)
OFFSET: anywhere
HEX PATTERN: 55 50 58 30, 55 50 58 31
STRING PATTERN: UPX0, UPX1
FILE TYPES: PE

PATTERN NAME: PE_SUSPICIOUS_SECTIONS
DESCRIPTION: Sections used by ASPack, VMProtect, or custom crypters
OFFSET: anywhere
STRING PATTERN: .aspack, .adata, .vmp0
FILE TYPES: PE

2. PDF DOCUMENTS
PATTERN NAME: PDF_HEADER
DESCRIPTION: Standard PDF magic bytes
OFFSET: 0
HEX PATTERN: 25 50 44 46
STRING PATTERN: %PDF
FILE TYPES: PDF

PATTERN NAME: PDF_JS_OBJECTS
DESCRIPTION: Embedded JavaScript indicators
OFFSET: anywhere
STRING PATTERN: /JavaScript, /JS
FILE TYPES: PDF

PATTERN NAME: PDF_AUTO_EXECUTION
DESCRIPTION: Tags that trigger actions upon opening the file
OFFSET: anywhere
STRING PATTERN: /OpenAction, /Launch
FILE TYPES: PDF

PATTERN NAME: PDF_EMBEDDED_FILE
DESCRIPTION: Presence of an embedded file stream (dropper logic)
OFFSET: anywhere
STRING PATTERN: /EmbeddedFile
FILE TYPES: PDF

PATTERN NAME: PDF_EVAL_JS
DESCRIPTION: JavaScript eval function inside a PDF stream
OFFSET: anywhere
STRING PATTERN: eval(
FILE TYPES: PDF

3. OFFICE DOCUMENTS
PATTERN NAME: OFFICE_OLE_HEADER
DESCRIPTION: Legacy OLE2 header (doc, xls, ppt)
OFFSET: 0
HEX PATTERN: D0 CF 11 E0 A1 B1 1A E1
FILE TYPES: Office

PATTERN NAME: OFFICE_PK_HEADER
DESCRIPTION: Modern OOXML header (docx, xlsx, pptx)
OFFSET: 0
HEX PATTERN: 50 4B 03 04
STRING PATTERN: PK
FILE TYPES: Office

PATTERN NAME: OFFICE_AUTO_MACRO
DESCRIPTION: VBA macros that execute automatically on open
OFFSET: anywhere
STRING PATTERN: Auto_Open, Document_Open
FILE TYPES: Office

PATTERN NAME: OFFICE_MALICIOUS_COM
DESCRIPTION: COM objects used for shell execution
OFFSET: anywhere
STRING PATTERN: WScript.Shell, CreateObject, Shell.Application
FILE TYPES: Office

PATTERN NAME: OFFICE_SHELL_SPAWN
DESCRIPTION: Macro-based execution of system shells
OFFSET: anywhere
STRING PATTERN: powershell.exe, cmd.exe /c

FILE TYPES: Office

4. JAVASCRIPT
PATTERN NAME: JS_OBFUSCATION_FUNCS
DESCRIPTION: Functions frequently used to decode malicious payloads
OFFSET: anywhere
STRING PATTERN: eval(, unescape(, decodeURIComponent(
FILE TYPES: JS

PATTERN NAME: JS_SHELL_EXECUTION
DESCRIPTION: ActiveX/WScript access for system interaction
OFFSET: anywhere
STRING PATTERN: ActiveXObject, WScript.Shell
FILE TYPES: JS

PATTERN NAME: JS_NETWORK_DOWNLOADER
DESCRIPTION: AJAX or Stream objects used to download/save files
OFFSET: anywhere
STRING PATTERN: XMLHttpRequest, ADODB.Stream
FILE TYPES: JS

5. POWERSHELL
PATTERN NAME: PS_STEALTH_FLAGS
DESCRIPTION: CLI flags to hide execution and bypass policy
OFFSET: anywhere
STRING PATTERN: -ExecutionPolicy Bypass, -WindowStyle Hidden
FILE TYPES: PS

PATTERN NAME: PS_DOWNLOAD_CRADLE
DESCRIPTION: Methods for downloading remote scripts into memory
OFFSET: anywhere
STRING PATTERN: DownloadString, Invoke-WebRequest, System.Net.WebClient
FILE TYPES: PS

PATTERN NAME: PS_ENCODED_CMD
DESCRIPTION: Base64 encoded command flag
OFFSET: anywhere
STRING PATTERN: -EncodedCommand, -enc
FILE TYPES: PS

6. BACKDOOR PERSISTENCE
PATTERN NAME: PERSISTENCE_RUN_KEY
DESCRIPTION: Windows Registry Run key for boot persistence
OFFSET: anywhere
STRING PATTERN: CurrentVersion\Run
FILE TYPES: Any

PATTERN NAME: PERSISTENCE_SCHTASKS

DESCRIPTION: Command to create scheduled tasks
OFFSET: anywhere
STRING PATTERN: schtasks /create
FILE TYPES: Any

PATTERN NAME: PERSISTENCE_STARTUP_DIR
DESCRIPTION: Path to the Windows Startup folder
OFFSET: anywhere
STRING PATTERN: Microsoft\Windows\Start Menu\Programs\Startup
FILE TYPES: Any

PATTERN NAME: NETWORK_C2_URL
DESCRIPTION: Plaintext network protocols indicative of C2 or exfiltration
OFFSET: anywhere
STRING PATTERN: http://, https://
FILE TYPES: AnyPATTERN NAME: PE_MZ_Header
DESCRIPTION: Detects Windows PE executables by MZ header
OFFSET: 0
HEX PATTERN: 4D 5A
STRING PATTERN: N/A
FILE TYPES: PE

PATTERN NAME: PE_DOS_Mode_String
DESCRIPTION: Detects DOS stub in PE files
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: This program cannot be run in DOS mode
FILE TYPES: PE

PATTERN NAME: PE_Malicious_APIs
DESCRIPTION: Detects suspicious API usage in PE files
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: VirtualAlloc|CreateRemoteThread|WriteProcessMemory
FILE TYPES: PE

PATTERN NAME: PE_UPX_Packed
DESCRIPTION: Detects UPX packed executables
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: UPX0|UPX1
FILE TYPES: PE

PATTERN NAME: PE_Suspicious_Sections
DESCRIPTION: Detects suspicious section names in PE files
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: .aspack|.adata|.vmp0

FILE TYPES: PE

PATTERN NAME: PDF_Header
DESCRIPTION: Detects PDF documents by header
OFFSET: 0
HEX PATTERN: 25 50 44 46
STRING PATTERN: N/A
FILE TYPES: PDF

PATTERN NAME: PDF_JavaScript_Tags
DESCRIPTION: Detects embedded JavaScript in PDFs
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: /JavaScript|/JS
FILE TYPES: PDF

PATTERN NAME: PDF_Actions
DESCRIPTION: Detects auto actions in PDFs
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: /OpenAction|/Launch
FILE TYPES: PDF

PATTERN NAME: PDF_EmbeddedFile
DESCRIPTION: Detects embedded files in PDFs
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: /EmbeddedFile
FILE TYPES: PDF

PATTERN NAME: PDF_Eval_String
DESCRIPTION: Detects suspicious eval usage in PDFs
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: eval(
FILE TYPES: PDF

PATTERN NAME: Office_OLE_Header
DESCRIPTION: Detects old Office documents by OLE header
OFFSET: 0
HEX PATTERN: D0 CF 11 E0
STRING PATTERN: N/A
FILE TYPES: Office

PATTERN NAME: Office_PK_Header
DESCRIPTION: Detects new Office documents by PK header
OFFSET: 0
HEX PATTERN: 50 4B 03 04

STRING PATTERN: N/A
FILE TYPES: Office

PATTERN NAME: Office_AutoMacros
DESCRIPTION: Detects auto-executing macros in Office docs
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: Auto_Open|Document_Open
FILE TYPES: Office

PATTERN NAME: Office_Shell_Objects
DESCRIPTION: Detects suspicious object creation in Office docs
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: WScript.Shell|CreateObject|Shell.Application
FILE TYPES: Office

PATTERN NAME: Office_Command_Exec
DESCRIPTION: Detects command execution in Office docs
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: powershell.exe|cmd.exe /c
FILE TYPES: Office

PATTERN NAME: JS_Eval_Unescape
DESCRIPTION: Detects obfuscation functions in JavaScript
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: eval(|unescape(|decodeURIComponent(
FILE TYPES: JS

PATTERN NAME: JS_ActiveX_WScript
DESCRIPTION: Detects ActiveX and WScript usage in JavaScript
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: ActiveXObject|WScript.Shell
FILE TYPES: JS

PATTERN NAME: JS_Network_IO
DESCRIPTION: Detects network and file IO in JavaScript
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: XMLHttpRequest|ADODB.Stream
FILE TYPES: JS

PATTERN NAME: PS_Execution_Bypass
DESCRIPTION: Detects execution policy bypass in PowerShell
OFFSET: anywhere

HEX PATTERN: N/A
STRING PATTERN: -ExecutionPolicy Bypass|-WindowStyle Hidden
FILE TYPES: PS

PATTERN NAME: PS_Download_Commands
DESCRIPTION: Detects download commands in PowerShell
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: DownloadString|Invoke-WebRequest
FILE TYPES: PS

PATTERN NAME: PS_EncodedCommand
DESCRIPTION: Detects encoded command usage in PowerShell
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: -EncodedCommand|-enc
FILE TYPES: PS

PATTERN NAME: PS_WebClient
DESCRIPTION: Detects WebClient usage in PowerShell
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: System.Net.WebClient
FILE TYPES: PS

PATTERN NAME: Persistence_Registry_Run
DESCRIPTION: Detects registry run key persistence
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: CurrentVersion\\Run
FILE TYPES: Any

PATTERN NAME: Persistence_Schtasks
DESCRIPTION: Detects scheduled task persistence
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: schtasks /create
FILE TYPES: Any

PATTERN NAME: Persistence_Startup_Folder
DESCRIPTION: Detects startup folder persistence
OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: \\Startup\\
FILE TYPES: Any

PATTERN NAME: Persistence_HTTP_URLs
DESCRIPTION: Detects hardcoded URLs in persistence mechanisms

OFFSET: anywhere
HEX PATTERN: N/A
STRING PATTERN: http://|https://
FILE TYPES: Any

Appendix G: Complete Scan Results (MP2)

The following table presents the complete results of the combined custom pattern scan performed on all 220 files located at F:\MP1_FILES\. The scan utilized the merged pattern set (ai_combined_patterns.txt) created from all four AI platforms (ChatGPT, Deepseek, Gemini, and Copilot). A total of **184 files (83.6%)** were identified as malicious and subsequently quarantined. Each malicious file includes the list of custom patterns that matched.

Filename	File Type	MD5	SHA1	SHA256	Malicious	Patterns Matched
File001	PE executable	7d0690a235298519fdc912928f0333ef	d2affe98fcb0a0c473ca5756c3687a246b642bb3	7a6f4e0442b3b4f6f936c00e7fd813d0fda011f10496481d94805003cf4f330b	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, IOC_HTTP, PE_MZ_HEADER, PE_DOS_MESSAGE, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File002	Binary data	e2142a946eca0ac74dd03fa1d9c67bd2	199a95428a251a9ffddfa209706016766b48087b	ed5de336ddc9ff34554219b5c125971399bbbbb71db8a465aa7d17bb8944eb9d	True	IOC_HTTPS, BACKDOOR_HTTPS_URL

File003	ASCII text	821094b7f221d9b915bbc55bc9530a1c	2368cdb34d96a5c5c38884ed8d125ca9832107fd	58d04b05a781cbfe76a36ce7a5e275b65972dacd5384b990217d271d090e8e0b	False	
File004	ASCII text	6d30132c09d8e7feb13cd0fc951b8c67	a2e190b5a8609eb9f75095806f73d00add3e20e7	621a179e360dde879b74b9be144b8ab6e630e58ea992971f3bca9146c9635c20	False	
File005	PE executable	8af314b532681aa2a272543e89bc1eeb	7e68dba9fa672d5ede7e787e5c568a4ff2a789c0	be03d7f716ccb1120fabf644f12d9bea11e0eeeb8b2c64686b842869cd4c50f0	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, PE_MZ_HEADER, PE_DOS_MESSAGE, PE_MZ_HEADER, PE_DOS_STUB, PE_MZ_Header
File006	Office Open XML	17bbcf874c7ef4315dc7114f020c46e6	7133fe4d816c890e44c4c78b58a6c8681ac874b0	fe5d03498443b9d9b248d83499f26e7c177ca6e41c4394ff010f77aa85344a91	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header

File007	Office Open XML	9184f89abe4a76d7a312312396473c2c	44e9c9a846f09707c247e4d4d4a3087c8574b4f1	593e2bfb37d712aa42b97ca849d23107456dcc69dc61408978c4ae4c5a9806e8	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File008	PNG image	75c7e160e5e35d778df5907c2c94dbd0	8a8b3c46ff4cb34085de75c055c43f3cc043cc1c	1e8804a55aa1931e0382776bad307c5b7401daed687b87bf2dc2ca4a4cd075f5	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File009	PNG image	dcff3b11da469dab3029d65b9a9a026d	08c81c41b9bba9ec78f908a770a2144ddca93ab3	caa4f0870e1edf0e723db15963b182576d841a4df2202ea7e36a0c19a4a3853f	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File010	Office Open XML	df23a5f65a651a5368a1f4c68b4537fc	fa74b7d0e1cec72539108a3cd9fc2bac6a7210e3	3b7abb872967e1db077436f9888bb0039ad4cc0ef0ac066a638a57d4bd26c775	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, IOC_HTTP, PE_MZ_HEADER, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, OFFICE_PK_HEAD

						ER, PE_MZ_Header, Office_PK_Header
File011	Office Open XML	aa4fe812dc7a13e684d33bb70f3e4265	69c37b69b78a41efd19eea6d67b4b9b3eea1aa38	7d0d7911b0d2f8edd31039124f32e2565cd65705d491344e7e51282da201de88	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File012	Office Open XML	cd7a8a6e473ca1693ae557da91650b8b	94158316e899691b587a346e5a5e5df2d6fcf437	071345a2709641e09bd87d46c65dd868e1000e1429f36051df2c9b479a9e8fd5	True	OFFICE_ZIP_HEADER, OFFICE_PK_HEADER, OFFICE_PK_HEADER, Office_PK_Header
File013	Binary data	2d9a7072177dd37a9fbacbdaf12b8b35	53b44947beaece17dc8d6c48ad2c8e5799a5ecb7	ad4a58c7b13e3bf2504d6e9a4f67c03bbdc3cc09a338c3b7fdc1e890e1c80e93	True	IOC_HTTPS, BACKDOOR_HTTPS_URL
File014	Office Open XML	79c0374c3dc8abfddac0c302d0330529	9ba252ab7303b9dbd6df8aab350e88da7f43fc1d	233c2b822e1930e61de96b91aa6051c327301c1f46a08	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER,

				87f43cc21534fd44de1		PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File015	ASCII text	e4507f2ec05eb69d935540e5b4d7ec5e	6079a4fc71df42383ada4142dffd97be27c483ea	324d140227ff55095c890442e288bd78259fb18008caac0bf611dfccad2d2bc2	False	
File016	ASCII text	e71bb959c29c164e77ae88bca85852d2	f29968c8621ec698f98f555172a0b0a2fadbcb53	a71945c90d5a1dd9b21256ad2f71d6b426aa5a51bcdc1692b128e6316771c890	False	
File017	Office Open XML	c374b2cac48b3b80144865fa7f181c2e	421b51dc3fd04041d3bac7434174e80f708b056d	6c2ddef8af98d34537d0b8cdb4251c2d6a55b09875305c82b39bdac20dea70ef	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File018	PDF document	5bd7a1b4904acc6280e2c1924a627860	2881a4e443a0aab7839bdf5647e9438afbe8bf24	304f551a0c08e27c12b749fcc8f213b97ba87c3f09d066	True	PE_MZ_HEADER_O0, PDF_HEADER, IOC_HTTP, IOC_HTTPS, PE_MZ_HEADER, PDF_HEADER,

				9f6655e1a5400c933f		OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, BACKDOOR_HTTPS_URL, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, PDF_Header
File019	PNG image	9d5089314e21bb5001c52cb6228d9e0a	ac7b220cbe0494f89ce685d7cbed849f2f992b73	2517ee94d08b1eb42f5599f45066d450c26782e1c6ffc0fc0225d89c62d2e2bf	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File020	ASCII text	2452330b9f8f2c566f6b7c54d53c711a	2fb25f4805f11d6f21f0d0b06f247ebab6799d59	9fd1640e94b12e51aa238c41e6306f9e0619f672f969ec4b2cb347732744188a	False	
File021	ASCII text	81936e46314e4a094681aa5a17eb0ed8	231b5648b1bc13c805553b5d644650f9d17422bd	13fb48a6c31a72e0e2b69650b1cebe8954f7b73449848492dcfd22b25bd91afa	True	IOC_HTTPS, BACKDOOR_HTTPS_URL

File022	PDF document	dc76bac39b0b05024c5dc4e167430170	994bddfd345637ae2abcdd587fe884ccee627104	5ba8b8c4481d78f5b39daffb796ff663aa50b8338edc61b0a6ee028ad0a1b2eb	True	PE_MZ_HEADER_O0, PDF_HEADER, IOC_HTTPS, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, BACKDOOR_HTTPS_URL, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, PDF_Header
File023	Binary data	9c301375efa5c4613b55b7ae9178d837	3c43b63251acb488c47854a2752328525fcb7259	172ceeeacd20f57c054f26ba9ba83d9ea971fcb3cca9a343135371cddcd77197	True	IOC_HTTP, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, OFFICE_PK_HEADER
File024	PE executable	371de9cf8b4b2520ab224038f0e6b9a5	6d2054702bdae5168522981080051e39da3291e1	7dc700da62ae9e18138d220ea344c5c104b8c4a32dc2af96076121a799185dc4	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, PE_MZ_HEADER, PE_DOS_MESSAGE, PE_MZ_HEADER, PE_DOS_STUB, PE_MZ_Header
File025	PE executable	0aca6a84b2b047abb36e3fb35b7938d3	d2dd2739ac3d2f496a7a5c849bcd2e12142d09ec	3df63fb05c080d1652305e4aa50bbcfba703b71f4f5b55b	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, IOC_HTTP, PE_MZ_HEADER, PE_DOS_MESSAG

				29762fdbb8eecf82b		E, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File026	ASCII text	4e5ec44fe5564450f9c1061ad18aeb38	dc447a64136642636d7aa32e50c76e2465801c5f	81cbae84a29ce7e770bf2bc7b178e50bda0ce8de6067aba661b0bc7b05b562f8	True	IOC_HTTPS, BACKDOOR_HTTPS_URL
File027	PE executable	899c8ae4b45a7d4957468e47833bc84d	13513de250639b432094dacbd14668339ca6b2a3	a82e984256abd016dfef2e0dd554bf7d36c3edffdcc516580df4d0527da63cf8	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, PE_MZ_HEADER, PE_DOS_MESSAGE, PE_MZ_HEADER, PE_DOS_STUB, PE_MZ_Header
File028	ASCII text	3d809cda8a3b43c52a692d24e21af70b	0b4c9a6830f3d0b511fbb5af8cc492bde2715f4d	0a7cc99c58d394b983cd6f0084acc9fe71b7cc6bcfc91c543bb8dc1a5174b63b	False	

File029	ASCII text	67571555f67b3a0b1cfb5b7efb8fa446	62c8a155d9a89b9442a85588bac0138b226500f2	f2d66c27e46fd953e90929cfa21390964036b8c913d99acacc4295f8daa049de	False	
File030	Office Open XML	da3fc8e4d769d1655552f9c000312c67	d4e97f94ed56361255d38a8559758e98b36324fb	570481ebf63d977e87206c3810f9ffd4d9357b37aae5767d86a061fc0135d767	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, PDF_JS_TAG, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File031	Office Open XML	18ae60e913174c448c66c4bb674936e7	10d5a709306eb512560ddbe30e601a9e9008cf9e	81082c4fcc0ec554191ffe3443dacf4d442337264060ee97d019c3c84c6e099c	True	OFFICE_ZIP_HEADER, OFFICE_PK_HEADER, OFFICE_PK_HEADER, Office_PK_Header
File032	Office Open XML	93048e6c41c1e52a2be85096b8df6614	5db12a5ec83b30e7b4d690f60159afd7e027fe04	89329961c24be7ab46447b5a0da0a3787e37f2d7d00c98aa3673c77c973e02a4	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header

File033	Binary data	8f037af809f60b33d6325be04e7e2747	a94507bb7a4c6b0ee11b752d8b385d215ce34473	52a382ff0005e95597de5539d5b9ab1208e5e943d19bb6a14537fec2d2a3d64d	True	OFFICE_ZIP_HEADER, OFFICE_PK_HEADER, OFFICE_PK_HEADER, Office_PK_Header
File034	ASCII text	98c35c46893f75f22975e6c1ab6131e0	f255dac4e27bb6793a05f093452b3c411a076f07	be063e3c106e6ecbcd6366742cfbacd3c6fdd5596665d2679de8cbfa5960d26f	False	
File035	Office Open XML	0c5393a755845905 13c78674fa3f0d6f	09671885a19250f7c5512043ea3816b174e150e5	155f0ed93182147375 8ba6f804d113c50afe262eb1c4c182e3a0f1a7a09970ec	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File036	PNG image	a272e6b746211b62152db107c265ec14	9720f66f73dcbe0ab836b55ed8e8ca6ff343744a	fc691fa2e54dafccb9744fe0249f946a87b4e69f2a49ace3c9cce22dd737b82e	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header

File037	Office Open XML	10c891b8ebc12cc4066af9a675d637e0	295cbc82d0451974ed79e2f19c79be35861d6fb0	7321f38d919d4fb46030f5692d2aaa8d74caee26fbbdf7e4b06ea71b68ed27fd	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File038	Binary data	ee65f859755a2a7aae846761cccad234	9cac09cf3f08cdfa6042d1bc8ef45cdbb1ad7f8e	131694dfcd8c271b37e54ca9e5c1abbb9ac88526e15cbb9356084667282d36eb	True	PE_MZ_HEADER_O0, IOC_HTTP, IOC_HTTPS, PE_MZ_HEADER, PDF_JS_TAG, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, BACKDOOR_HTTPS_URL, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File039	Binary data	8f179f9b48d6c13fa96e801c31c5120b	97a36bda7a3707c371a61ca1be5babd819f22dfc	abbc52db824b9a134dfd0ef0c646a8e2382a05e7af9a9f5865b77cc704844495	True	IOC_HTTPS, BACKDOOR_HTTPS_URL
File040	ASCII text	9ed8f42ccba544d90543e5f82e255fac	b8c466614bb3ec0e42d6c56c601dda2b44fc7107	0eefccda4460faab2f6a2243598f74a58c93c2ff01a60e4c	False	

				7b87ff2ff048677e		
File041	PNG image	6f821e51b6c6f33e1768187ddd1fba46	9d978cfbbe2db83aa8c852ec6955ddbf1969e504	d2c03ebbcb573163994d6eb247209a6cd9ffd5141b14511e4766a31b303c0b65	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File042	PE executable	74a4b714c851de93c773b1772a40807d	48506056f569a40ee28155c507e3578af5ea4579	8b9d7b249fd8b32d14465b6d038060da3545114bcf0041ada1f7205b81d6887d	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, IOC_HTTP, PE_MZ_HEADER, PE_DOS_MESSAGE, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File043	Office Open XML	43e1b1fb5bcd16c937b0da2c4a6e540c	661d2ddf9bd727de31c653aba3edd83c19e0959a	13e84bab911c370a9c80081d6a711f6f334296996cac27348aaad72aa59dc5b7	True	OFFICE_ZIP_HEADER, OFFICE_PK_HEADER, OFFICE_PK_HEADER, Office_PK_Header
File044	ASCII text	58e6cf4d000a0330	8318fd3c54c48db49737aba	d6a99d5fa30152aa09b95e81986	False	

		e09c022cf94f9be7	dab240afb2c7fa7e3	41cc9ee95c27051ceb5031b22bacb674a7073		
File045	JPEG image	6d8f6f9f0df51f804f0d9e706373d881	1a4ac39badd635bb1a9e764a70e72fa901b3c98d	e3a4563c0f11178ce2eb4623899a43c9221b68e6e0b8ecaeebca23c4277d9496	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File046	PE executable	f63d16a402673307cbd7e0a69205baa4	afb6c414197688a1bc6c113bb5add0f02f308a50	2300646f4daccc0773f9cdc078f603814c2025828cb978137e3b38fb1a2a64a3	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, PE_MZ_HEADER, PE_MZ_Header
File047	PE executable	130368dcac9cf8171b792bb164b1cd1c	a4609bedc4d885a3c0dd0f3fc9757af6a377dc8c	6a34ddbbc2a43d2c911fbf7b59824c4b7860448f7c1435e520b562af33614d50	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, IOC_HTTP, IOC_HTTPS, PE_MZ_HEADER, PE_DOS_MESSAGE, PDF_JS_TAG, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, BACKDOOR_HTTPS_URL, PE_MZ_HEADER, PE_DOS_STUB,

						OFFICE_PK_HEADER, PE_MZ_Header
File048	Binary data	e117cf3498fa380b6c06de68232ee65a	f6745cee730e1665db5932e0ab51d53a885a5ea8	5ac465d7d37bad28b424c139e142cea865d163495182eecdebf02ea91705d0a9	True	IOC_HTTPS, BACKDOOR_HTTPS_URL
File049	Binary data	428232826229b4484bd2a9de0091b60e	1819ba9f99f764843716e04ed1d150fbe9efa16d	9f85d5be7ccd66b89e730bb720e157bac05d165d38cb41c343eb21421c10e359	True	IOC_HTTPS, BACKDOOR_HTTPS_URL
File050	ASCII text	c9ee034bbc5f70cc0246bde27c1be76f	600cc227ee8d10820ed06a214b58ea151c213371	0bfdd7cb1d0cb8697316b798e83792d11634273aa4471e31409fe86c15e70104	False	
File051	PE executable	db97870e110d2e495f191c432797ad04	3a73e650e6386cfd23f3d726c347a9359a37478b	bb09f3b4e07672d3ac19638db0979ec4c5442492af49b14006c60c55a97f7d04	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, PE_API_VIRTUALALLOC, PE_MZ_HEADER, PE_DOS_MESSAGE, PE_API_VIRTUALA

						LLOC, PDF_JS_TAG, OFFICE_PK_HEADER, PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File052	JPEG image	a978b63e14d67b4d8a9625655adb3838	2aebfa2cce4c9a5498dc9541bee9cb27cce887ab	2853b3fa183165c4ec4fe29a88c722c5573c60c40ac92581fa2ee53f6af22a8c	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File053	JPEG image	b7bbbb113e2913b4d54d17b47ec11308	e7840553b64fad9bab39e0c5a138e21a18d313ee	cbc1858fa38513e9d63f90483cccc15745a5bfa58642b9fe8f60374f7d684308	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File054	Binary data	2c6444207e051dbb8f9f8ceb8f05bd06	7df60f39ece494a400b4afb33484d4652cd16003	b2234d575d7257d1280199c630c2a9bc15d637dd04e040cff0d42dd8001f9531	True	IOC_HTTPS, BACKDOOR_HTTPS_URL
File055	Binary data	386812055de7c60fd297046a661ffd76	7e5659b43870dffc2bc7e1cebaf5aab028085d4f	044dabbdcc1d8d6dcac29643daf6a30de44ba64d55136	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, IOC_HTTP, IOC_HTTPS,

				1019d0ac29f3d09c551		PE_MZ_HEADER, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, BACKDOOR_HTTPS_URL, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File056	Office Open XML	1c03344f83275ac70b5391df2e8e0876	bcc8198a1d3d16525e4bf56a0d42a078a8893424	eebca9f734909c752bd248639b0b387908cb6df0182ea97346857fc88affb08a	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File057	Binary data	7e40596c2d0399b64cc871ff4158ae57	f440d75b7b3c54939699edb42c48fed43ed131b1	9c81453c7de7f7b959263051a8107ddafaad8a53fad4125c6a72541eb06a06bb	True	IOC_HTTPS, BACKDOOR_HTTPS_URL
File058	ASCII text	18039dade136470d4112c839cc043efe	6cd50628e742766f69ce4dd62c8a351b4eefc8c5	0b37ab6ce65c46852c4201e7756c243a8a019ceba7b62558b2666a7eb24afb17	False	

File059	PNG image	ebc5f89869f7306fe420ca02a9677f9f	d8096c532ab21c705c08de6304672f2a0e6def72	5220da04b5ef146ca0230533d56c2085c170d9557c7333faad22f3bc766fb2b0	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File060	PE executable	8d404ac68cc517b456d0c46305caf6b4	da8c7f8945337943f5e1fe1d1513395144711333	35c1692f61a508740541e3ce33ebd664db88b7d78bbd99994b9082c42b99efdd	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, PE_MZ_HEADER, PE_DOS_MESSAGE, PE_MZ_HEADER, PE_DOS_STUB, PE_MZ_Header
File061	ASCII text	0a9bbaa0e74a6e389c73c1a18d9897ac	c4b960d9a12c103dcf41b68eedc34c1fb59e1c37	d3dd96a21af17539c75ef957bd5a90b745c590af2667c282b160bdb2f0732067	False	
File062	PE executable	f0c3b5b14d0b5c1420d1e22f3585f4c3	cde069a5cbc8bdefcff8a67dcf26538cf35eb99b	145cca35fc844f511e8af7756374d27125a440b088410ade80c15527fb39e224	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, IOC_HTTP, PE_MZ_HEADER, PE_DOS_MESSAGE, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL,

						PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File063	Binary data	2b2cf30e60c4c5c6a7bfaf7f05f559b3	82cf0d9d4f3ea20c6e2f8f55c883865d5f77b5ae	f1330cf05c37e77a9b29e7078d564282ca9c897ceebf50fa1e2078d21e48cffd	True	PE_MZ_HEADER_O0, PDF_OPENACTION, IOC_HTTP, PE_MZ_HEADER, PDF_OPENACTION_TAG, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File064	ASCII text	ca807e6ce43369833704d96678d19667	f53f88263eb2968f54c43bf34a4905d1c30ee469	06b41becd146579b61b1ff06b083406ec810d047b89adef3eaa0e50a3d6ade33	False	
File065	ASCII text	3ba7c71043ab870a87d59a781def60a5	baeb7379a48da8345479b809c9084eb419d38764	ae1723cc3fb17ca87d6be46de09162dcb52f4b5291c58a095e12cbe3bb5e2414	False	
File066	PDF document	597122e0437a3753	a99f8441c9dc40ce170c0b7e	dacc4061e08088885f6338bdee771df12475	True	PE_MZ_HEADER_O0, PDF_HEADER, IOC_HTTP, IOC_HTTPS,

		b00487694f30de80	129caff0ee4206ea	5e4b59c8abdd6bf96b9b81ddf538		PE_MZ_HEADER, PDF_HEADER, PDF_JS_TAG, OFFICE_PK_HEADER, PS_ENC_SHORT, BACKDOOR_HTTP_URL, BACKDOOR_HTTPS_URL, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, PDF_Header
File067	PE executable	ea15bc1156bf638002b99c63bd4abb1a	1256d16942369961683c7f21c7f77128ea1f73c9	78d9ccc3aea19b6656afb4cb7a7afddf0eb3fa2ee3c348a628b3beeacd0e98a2	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, PE_MZ_HEADER, PE_MZ_Header
File068	PNG image	4f81b658e6cc40fbf7c4664e14b24a29	6dc6d6c0568f695a24a42fcedfcb2409aa550654	e9099b712728530d628b1198561e6c2cac999f54e07a0a3923b9ad6f2d36004b	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File069	PNG image	314de58ba4408e243fc5e7f691014c26	96deb65cd007cbcf829b7ff81e5629b03cd87907	4597944501acf1ef4475162997e25e581a6f5d7d760f85826e6960ef5a49d1fe	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header

File070	PNG image	4effb217ca55d93bb55192693ef0ee9c	72acaa872934217c3fc370c361a21e22853c6565	c4cc527b4b4c5cab414a2ca920a3596df19ed8f5808509672b6b25b7feeca047	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, PDF_Header, Office_OLE_Header, Office_PK_Header
File071	PE executable	08422e540f71afbf8895440dbce80aa4	22f690f251032776e47565d46f3285c4011695bd	f650b7d8b20d0dc12ecdd0f683c3a1252d4fa0d9941378541738f38614af3c67	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, IOC_HTTP, PE_MZ_HEADER, PE_DOS_MESSAGE, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File072	PNG image	5956a153b3144e3256047ab8eead83e5	463c9c107e470c360e75f5e7e569bd6c3a4ae9e1	cfe80c6d2e2233791b84e68c40dbba5be93805c5f8ad829cf525b9dfd67c25a8	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File073	JPEG image	614cf260d4d13d5ff	8082d965f637e78065d417d	bd678df29699f8c9af45ec4bf10f8	True	OFFICE_PK_HEADER,

		b6f947cc806d671	cf7c42915e7cbb988	b7a197bb0d07850fa5c64fd013910bc5e5c		OFFICE_PK_HEADER
File074	Office Open XML	f9b2c6c68a3ddfc2678fcd9f0cf0464e	7178220d10182f9ab2c3efd1054a8872e24800e9	1a8bd54fb55f42a50ab36aae0ec2e366a7e16f2169086aa448d062be79b1ab1a	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, PDF_JS_TAG, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File075	PE executable	cbf27b79169a22e2ed9f2f6ded176cdb	8929c844ae8a6f80694d0e8de149adbae60fd78a	44703ef6d22f6859c137eb844f3939d0e7aa21fb7f6bed16861758bb9e5229c5	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, IOC_HTTP, PE_MZ_HEADER, PE_DOS_MESSAGE, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File076	Office Open XML	585ce4469a6adfebb17e5c7b1163f349	f90e097bb2880d250c2d1fe6d6d69f49a4c89ed3	f7bd687eddd64d561b50372ddaef38b90ae58a358099a06dbf33a8	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER,

				94856b9760		PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File077	Binary data	f5fa09dace753bebba5b6075f127dc26	6b628dfe25adc892b188fbf79dd627b55f0474cd	611d489613cbef351d56d9e5b5e51e44d95376d60b70cee8c1bf4fd9a3d128e0	True	IOC_HTTPS, BACKDOOR_HTTPS_URL
File078	JPEG image	2431f8e75577a0e57896d66782cb9890	11df096f561dcf067384125730ea9c354217abef	18bda15f6aed8448f4d714569c4ca343ad8e7dc0ebdd7016fe4ecf8ba51b2c2d	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File079	Binary data	b6167a060ce70bcc90de22e27a3fbb07	875417a1f75bd6b215b49df4ac0daf19697f412a	96cb4aaffd42b83c1f3325208d88d1d37bdc985d1e09bf3a01c42c3f4ce0a943	True	PE_DOS_STUB_CANNOT_RUN, IOC_HTTP, PE_DOS_MESSAGE, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_DOS_STUB, OFFICE_PK_HEADER
File080	PE executable	12961a8d12e77fc9b66345b80456f7fe	ec320d7a73dcf97b4b4fcb6e764a238a318e38c4	4879d504f9bfb07bea63f737a7d075d02bd3ec8fdbec4a	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, IOC_HTTP,

				cbcf13c22e2bece3b8		PE_MZ_HEADER, PE_DOS_MESSAGE, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File081	PE executable	52376b5fbaa7a368612ca909730cf9b3	b4e26ea63c2994e7d994c052c52d3f4e8da27c9d	9a253a8e0737cd0f157afcdcccc509e8da3e15aa072c5d1aeb3921aa9364216b	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, PE_MZ_HEADER, PE_DOS_MESSAGE, PE_MZ_HEADER, PE_DOS_STUB, PE_MZ_Header
File082	ASCII text	559dd7a83d0d7688e334978e75d45beb	9fa22c8cb4ee7c89f6787dc7c131758bf524e13f	a8e59a9df5508fc37a2bbbd42cb25427b0916d474b012ba2bf826190145594cb	False	
File083	ASCII text	d324ff068ad2ee0ce710cfe951697116	5355854158f0f85378e0e61a6176f12e805af563	07c2b8430a74a8ac1622b5223d7b11b1cdfbc24db0e0b34ef72697c35799a91f	False	

File084	PDF document	0003f23eb36cf0145bb82e108295b143	c514b8f17c465ef3d259d44ba6af4295b568f802	8d274a84770ef915b9a2470fa11e0127ca78cff6717552b57a8a155aa3381533	True	PE_MZ_HEADER_O0, PDF_HEADER, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, PDF_Header
File085	Office Open XML	5a237806a6afec0a40fb433530387001	3163223d1fc12c29ad6a01051f00054c5995f8c4	ef72773db6a85f9958aa9902903f0e901526100d826007770c095467ed72e9e3	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File086	ASCII text	6130cc3a7797bd20785db9a1e38246bc	c60b117808fc4e70da2fbc77f3a66da0ff9d655c	5c2ca1f15d4e9c3964785018c1f0a0309b2c64279c1b93cf3116fb71b60b9a0d	False	
File087	Office Open XML	b4ea3fa8c44c0e05e86e4b0d8859cf0f	ad9ada5e4004b1892ff0800575ffb3ba1ca2af0e	0b56a46148fe47c91eb72148db0d6948ba562aea7431e4b71eb5cd855ac0f1a3	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, IOC_HTTP, PE_MZ_HEADER, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL,

						PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File088	ASCII text	822123f73ba273c1a622f5d5cf32633b	3e089d46050c8b97e53a7892ff27fce5577062e9	5cdf52caf1a723074afda063ada5d669fa3035e4122618a1379530863b6ff471	False	
File089	JPEG image	905363c4abd5f39bc6f98ff3e25b0281	3eea1966f23ac6499d0cb9a013a1d14fa1919fca	34452eded8f8e27c105408e04149ed8de01b80fb4f05e8683ba24e831554734c	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File090	JPEG image	3e736f0149faed08b112e43245750aa8	20c0b1492e950b392bf8088f673a5bb9072983cc	bd72109e3820a742b32a683a315f5bc0512250da6dcb4c46744af3e85f22da12	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File091	ASCII text	00cf085bfe51c5de71ff776ac8257e34	5b487a6cd44e96db251408822b2f08c833f77152	22711f384d3443a30000d7829ff71f09745995c935bfd2e5f730884a4aa557a9	False	

File092	PE executable	cb22ec32b6b7d83582b7fda7d682ff89	47a126f450d91264f4a399c52d25efb622d2a169	4cd742b67eab294606c6f7813618af6d5824c2fefdc7469941a7f4394d8c3d12	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, IOC_HTTP, IOC_HTTPS, PE_MZ_HEADER, PE_DOS_MESSAGE, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, BACKDOOR_HTTPS_URL, PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File093	Office Open XML	8a0b5139a1e35f636b28893cb6436fb9	2cc913d580f766965d2a1c95cc60303f84f5dece	a3c1db4b5a48165f66b1c52610c8c3c4232417756cf6f638a273020cd7eb8707	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File094	Office Open XML	d46229b61ac5fa694f1e5547d7cf424e	6e3978345a445e7e659069466f1707e46f8c15b7	c5f3ac4a574e2bc4485f118c2861d93f6bdc8668cd28bb307fbd685fc98a26ed	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEAD

						ER, PE_MZ_Header, Office_PK_Header
File095	PE executable	bb24a4f283eeeb0a5b6a63256e197fad	e8039cd053f2974edaa188541159dfe12bea125f	bc00e73aa9064d4acca54d5b4d69bf2e7182f31169fd8ad0a214ed6b306a7b0c	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, PE_MZ_HEADER, PE_DOS_MESSAGE, PE_MZ_HEADER, PE_DOS_STUB, PE_MZ_Header
File096	Office Open XML	6b8f2e87dfddec9238fb7252e63ad558	f426e87d01f2d3d46e2217e053ffaf2b8f518511	188899ce3ec824a842b71508e6a2b8fe96bb96d4bd7be338042a88df447f11b2	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File097	Binary data	d9b96769f85876192c5efcfa556ab7f4	fe10fede7cc3507280bc6913686a0488224abf47	98758513eab9dec8874b5bbe1c651544721007d344f9d8945c95c76a70a20bf9	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, PE_MZ_HEADER, PE_MZ_Header
File098	PNG image	e23a82888ff953e592c9935f447348e7	78146598cdd205e741b122cf067352b1d7ec9cb7	a920e0ea11d2ba741547e289f88de354c5719366a8b3d	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER,

				32cf37db91cd68bd219		PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File099	ASCII text	492b46063a1fa97a738da1a411aad4d1	a85e740b01159f63bf255c5feaa89eeb7a917766	1458551d93f12aeada0bacd52b1e17f75167cc90fc09cf7504cce2b296352f65	True	IOC_HTTPS, BACKDOOR_HTTPS_URL
File100	Office Open XML	95b6f67c991027de4266a1e0e7e8d527	9be1f1347b642ed0becdc2fea3e4f272860a91d5	8d5335dba28ff705ead7a9f4c83872cdb0908e34f21421580a52c2ec8e72e446	True	OFFICE_ZIP_HEADER, OFFICE_PK_HEADER, OFFICE_PK_HEADER, Office_PK_Header
File101	PE executable	b022e52fae3ea7da7b43c8e80ca646a9	cc1fab36ee81c119aae9856ebea2e1735a309b30	655ce7fca0959a0eeb39b41dc7bdf3e094558875fb433067b6abce1be34c7c08	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, IOC_HTTP, PE_MZ_HEADER, PE_DOS_MESSAGE, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File102	Binary data	63acfbba8e6e6dfb	6bb6cc99155a64d8a0210ffe	6f07fef56af1ab119fa864ceb2d4e	True	PE_MZ_HEADER_O0, IOC_HTTP, PE_MZ_HEADER,

		a235018deaad7ab6	b2d455c4f67b32fa	47a1877573ad12d4adae77d44e854c19552		OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, PDF_Header, Office_OLE_Header, Office_PK_Header
File103	PDF document	1f6d1bba0572b67079ce8b19f66377f4	310d4f9892781915a9b1a06be70a145e13089b8a	fc04c312670430be2afbd2119e448d49c95d53351777cf308b0f95c94ce254ff	True	PE_MZ_HEADER_O0, PDF_HEADER, IOC_HTTP, IOC_HTTPS, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, BACKDOOR_HTTPS_URL, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, PDF_Header
File104	Office Open XML	315c12e381824fa8187634f37e4d12da	ceb6330fa3b9c7221d97447fff5201796eedc3d0	b8c95670d177067441851cf45d2da12f39ec89bf3c9e85dea17a531ffadddfee	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header

File105	PNG image	2e8f9dd9eacb56032b79a06250e949a6	1f7d9c86df7456e49ea0bc11e8a6a86c4c07923c	67cc5157e5d6ac139820be90071de05cf2d90400bb01f87610ec0b9f1aea9dc2	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File106	ASCII text	fe975f7601a2126ba0431d2e84cb71d8	9b6443c7c129555a34070863f8112406beefea355	f14a00c695926cd69546d07703f863828470524760afbce999abb84a6c13d900	False	
File107	JPEG image	62b5f962ab419a0e7d39693a47f6f8cf	85f3663b76b7a5cf7a6a7509c92bfbffc574a15f	9ae9a368cd561264a95abb5e5198f0b449c17afb9eb4eb93352f399978eac44a	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File108	PNG image	2083b7845475c50f4d3d0ce43abddf47	90f76c4eefb5140ed52e3cb09aa9dafe54e90501	c4652a51f423257ee8f535c8ebab08230171ab681f25bfaed0caeaa7f4d91ecf	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File109	Binary data	a8db5b9ff83ebe7e8	3ece35df69d47b45cf2f3d6cf	03ef8efddfe82b94dd5ab780706f	True	PE_MZ_HEADER_O0, PE_MZ_HEADER,

		7a3971576158977	825def15af9f531	928bfb3c614e21fae4686e9812f06fc000b4		OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File110	ASCII text	8b17d3c1a890583d649554e66df1c9e6	8e542857954afc39520828bcdd48bf58cd3f08eb	e64f9accbe14dc70a30ded34c08ff2a374e1b69a9ba777693eee04f83986d5af	False	
File111	Office Open XML	bbfd94f750618d4eb4fc8c3d6a9ff11d	d21850fb8f7ff21a8d48aaa9bb928043416924eb	be4813b07294520099ee561ef594396d61726eb042568cdca2bd27af6582e2d5	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File112	ASCII text	c75dbb43150fd2ce4732c77ef9997def	82c0591ef8737c0e3a74a0b98e4ba4f9996f1189	c46cfe45b8b38d43397873bcb7fa2e4d17a4d20727e77ab8d86a66bc8acd9a32	False	
File113	JPEG image	8019c9631f622de19c81a3e19df8b4ed	eb7d1a333bc894fb8c1a9afa2f70768246f9fa0c	c556339e370a7f8bc3fd9c1966adad947090eb47a158f9	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER,

				06f0a21e292cdc54ee		OFFICE_PK_HEADER, PE_MZ_Header
File114	PDF document	9b702a3a329410a22d2877816ece84c5	83e2b181ad279115050e23bce261ddd082f1b650	7662e8701c4b0dd703a57a78f4be877a743315267c28fb78ced2ba2f286bb8dc	True	PE_MZ_HEADER_O0, PDF_HEADER, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, PDF_Header
File115	PE executable	9e25484b1907186fa37395a4c3b79940	65e3ec37850f9d2d5ef82f264171611be5414d5c	ad1d9a68a9007982adbf3af743d79b5e21bd8048c8ebf513cbed290406c26a4b	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, IOC_HTTP, IOC_HTTPS, PE_MZ_HEADER, PE_DOS_MESSAGE, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, BACKDOOR_HTTPS_URL, PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File116	Office Open XML	b153dc51097eaf6bc297727b41d16fa9	73c3036755de9c36bd22f36f0238d56dfd094240	358f93a1d4b7abbfbb30e5e1cf307101626b20898ae1979c4cc532	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER,

				38f24ea25e		PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File117	Office Open XML	0d7783aa3a521e6d64aa1a829c5ea90a	6a6a963ea8245d4034c2ebcc01e98e1c6feb3440	9d21d85165b09487ce74a8b955757454616cda16525b02d2e2043f21de7c0bff	True	OFFICE_ZIP_HEADER, OFFICE_PK_HEADER, OFFICE_PK_HEADER, Office_PK_Header
File118	PE executable	e729b5cc0ca22d973378a595f0c02cf8	c278fc9f054943a8bc9fb22e322e588d30494a4b	624ee8195382d2c555046293546c65c215e25751b75c0aedaa8a842cc22aca11	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, PE_MZ_HEADER, PE_DOS_MESSAGE, OFFICE_PK_HEADER, PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File119	Binary data	fdea5ae8a8ede952bc58bdb26b47d57a	8960ef02e567df940ebd6bd1f46c06a3565fbcf9	1b94ffee23b76220566a8507d7e1b0b1940a971e1a700d4496658fcd3b5f5f8c	True	IOC_HTTPS, BACKDOOR_HTTPS_URL
File120	Binary data	0f3ee7c9eb9340ae44a93bfcd27523ba	6d9e489b5b0f096484935299db389aa2fc42332c	13a6ac51b47813e8e2bc1fea590c05cc6bc8e3ad964e35	True	IOC_HTTPS, BACKDOOR_HTTPS_URL

				18de976d3ebb66f8f9		
File121	Office Open XML	78db5fb8d3b5f11d349384c5c808d94c	65b6579a5c5dbe6d42f49031bb4c416a0d36f254	efbbc257b678eba3fd5a0d5ed8d5490a4a55fad8e4cd1e491a6b8cfcb3ac6945	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File122	Office Open XML	423e0563fe719395891fa51a000a41ad	83e586e9a4ccd04f8c77cf11bca3d1339e891340	af914f86a773bb8a035937c1603ae4cc1576b83dc6e4149bbf3d5f1219500015	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File123	PDF document	bd0ea60d14eea5566361109dc08af4ae	e84d7eedce3c34193739ab5f41af545a56467971	2c174db14e47a3582b5949c040ee19766c0a1e79810b924609c737ef14a77de6	True	PE_MZ_HEADER_O0, PDF_HEADER, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, PDF_Header

File124	JPEG image	d2183bc6bc61647833e480df21079df5	eeb2d7a4366130e5523204e5e85832c7a0b2d5be	115d22a6395054520a5e9e5e3e3d802ed1e2152455daea71b1bbb3fb3818cfcd	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File125	PNG image	b4eb8e22a873e2bff9a2c056c20da979	e182a3a956d31cf17eae094508ecda51bdf0f9b3	314241eeef941f55c2be2c9da7d161d5c55e91c020f2fb5e8af91e418a695f4e	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File126	PNG image	eff3a759ab08c2c95fdba8c3cde7c1f8	2515298c9ac3310c0edada1f005041b1dacc8750	ec201bdcb6f90ca2416ded4104e63c1a1f08ea14f4c22a2cbda99ee6c8259b6f	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, PDF_Header, Office_OLE_Header, Office_PK_Header
File127	JPEG image	07167ed9288a6fb574bbb0ad229bf1b0	ce30a350e536ad0ae9af5f625a0213d6fbdd8e6b	e188f2817b33216c311c017d1b92e112abcb5bee1a2d4437efc58ab7490cc273	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header

File128	ASCII text	c79e9121e51d9373a410b53ae8a3d964	52bf71c606ab98d918cda1fae17ecbdaf58fe34d	aecfaf78992755f73ee068196d43a6775e2ece96c2770171529e2517633396b6	False	
File129	PDF document	321ae2c8ccc6a16073c16910ec598891	c0ec1fdf59c40d422a2fc1d63bc7fb23c845db7a	34768eb405fa77fd6375a3d87e40ad1df08c268171dfaf20230774e6c7612a8f	True	PE_MZ_HEADER_O0, PDF_HEADER, IOC_HTTP, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, PDF_Header
File130	Office Open XML	1e40fe728f54ab560be84048b37e2136	5be30928b8af0b3e684761d3e192bc7d52692a0e	1e6534a07ccd6b616c9a80228698c73903dce2aad3053db598e9ae8ab4788c1e	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File131	Office Open XML	ec026d8ed4a1aeaeca8eeb4e7bd17715	25d67d32197d0c32dab7c19404d7d33eb6c3e89b	ca081c2d441688ece3475cdbd3136d3249fe993605e1053eeafbbb	True	OFFICE_ZIP_HEADER, OFFICE_PK_HEADER, OFFICE_PK_HEAD

				441f9486d8		ER, Office_PK_Header
File132	PDF document	d0787c3ca7aab644472e1a015c23597c	7c0efa4720977994df80da127d1f3fd7f278d197	d795e506a98b382038c9b29117996c95f2edf9388105fc61347539c8312fbc83	True	PE_MZ_HEADER_O0, PDF_HEADER, IOC_HTTP, IOC_HTTPS, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, BACKDOOR_HTTPS_URL, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, PDF_Header
File133	JPEG image	89e432c568ecd36d5136ce58c923eb9a	0d7e846937431043925526e882e17ca758adb83d	0dcaf05f6c9f659cb8e342464316504b7ee03cb0aaae1731307206bb8130d74c	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File134	Office Open XML	d6eadec77ea7a7436fb9e5f39bdc9b36	711fdd86b47733ffdeeec89f21a52c540ee7c23a	338d8b3851a2bed5444ff6364c5b70d6e5e1901195611e4a0a7592423375cab9	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header

File135	PDF document	14ca718fd2c6718e4a31472da950627a	363e3fb566ee15d4f4a5c35ceefbd5d1a5343f03	dbcc6774709f846bfd632ba7f29bc152b24a5afcd431e1c875a7e67e97c9f6b3	True	PE_MZ_HEADER_O0, PDF_HEADER, IOC_HTTP, IOC_HTTPS, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, BACKDOOR_HTTPS_URL, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, PDF_Header, Office_OLE_Header, Office_PK_Header
File136	Binary data	31bbd83292460bdfb30a7bd9ff9aaf4b	369cc313e0ff83e79e030eb69673e2287386f3d8	e9e9b42ee499b4d0e717bbfece1011af2127c9bc70dd9d1e8feb0a59e654c2f7	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File137	ASCII text	81543b22c36f10d20ac9712f8d80ef8d	892b34f7865d90a6f949f50d95e49625a10bc7f0	36266a8fd073568394cb81cdb2b124f7fdae2c64c1a7ed09db34b4d22efa2951	False	
File138	PDF document	302b502d4e62ce93	418baca3e862279dd832d4	41074035c291360d30b057eab0418219e3c5	True	PE_MZ_HEADER_O0, PDF_HEADER, IOC_HTTP, IOC_HTTPS,

		64d06014e544c095	489d6a82d8665168ba	31bb3186be50bb15bffbb8d7fb64		PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, BACKDOOR_HTTPS_URL, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, PDF_Header
File139	ASCII text	0618ec59c2401e95bc66f5f48bb46bdc	de10d7b09c2b49ae3b8beda9ee949f1cf084a519	2c214b7b450719ebdfde3e695e521d5b68592086983ee6bec6b9cc8a9983fab2	False	
File140	JPEG image	f83e3fd9b34363578c2128dc3eac6f01	64d091ad38f3289dcdaac720635577f1fabbcea6	2616a07362b644bcaf0bdb9f045cf066d1dc270dc3cccce937d4922d73954007b	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File141	PE executable	ac1b94145d5bf11f34eba1137d8b39e2	c33584ab2f523cbb8556599ccbefc67129602668	e9b15661934525f6e87edabbc0ada0a8604398b2f1f47435803a9cec645a3881	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, IOC_HTTP, IOC_HTTPS, PE_MZ_HEADER, PE_DOS_MESSAGE, OFFICE_PK_HEAD

						ER, BACKDOOR_HTTP_URL, BACKDOOR_HTTPS_URL, PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File142	Office Open XML	ecf92b48eaa6ff7d658170aa49a6e660	ab486d834224d33b59515b7d1c738911b949c126	c6d39b4da918770a0a8ba7af099d3fe93723b0fa335814e073d2aa75d1c18bd9	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, IOC_HTTP, IOC_HTTPS, PE_MZ_HEADER, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, BACKDOOR_HTTPS_URL, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File143	JPEG image	cef114c5875ba465fbdcb7dc7b297a42	b0e30d82001184e7373b2576fc229609046f104f	81759f96075607fb7a984d8e9a6471e85a637146fec753e48eefe4d05a37e626	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File144	Office Open XML	d19b96636f75b86f0f2740e6434e7b6f	e601679953b0423457f636adb46fd7b3192cf6e1	fdf18d8256d5950e63a8ae81982706e49c206760e03896	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEAD

				555156f2d935718b9c		ER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File145	Office Open XML	d688f330a67743da9edf360bd87a4757	50314186f6005a2f6383fbfb9f112588096a9323	dff503b08e4f8818a56b64390d279d741919f743891329a4280bbb0061f6b2c0	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File146	PE executable	290a44f94f15d6ae35915320294947f6	dbba2d945d3a10d313896d32d90af9278559bba6	31096b3f8270edc5ddcb3dbd462c41afb39b2caabb928aa6938010a7e24578f2	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, PE_MZ_HEADER, PE_DOS_MESSAGE, PE_MZ_HEADER, PE_DOS_STUB, PE_MZ_Header
File147	ASCII text	a4da80722c9b88f6fd9261637af70098	15d07a19e6cdfeca5bec0472779a3a4258e759e2	2ddbd5f4ce4eccba52f3cc7ecf4929dfb571361d97a33ab3246617f70c9d482f	False	
File148	Binary data	ef063d467563865a	c87af5eee0cfaf88c5176d1ba	9e345ad76741371fbba34da23c3	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CA

		a7a97b4dae9924ce	7591bee24a32673	9ccc3897ae35846ee44ab79fbfe18d3a9fd4e		NNOT_RUN, PE_API_VIRTUALALLOC, PE_MZ_HEADER, PE_DOS_MESSAGE, PE_API_VIRTUALALLOC, OFFICE_PK_HEADER, PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File149	Office Open XML	0a35b9503060597bb052f21b4b4290d6	98629156401fa0e895aed31c838fd1519d2674ec	ca651505e86f7be138e5fc8a11b450a175064952e4bf882c332aba1831497212	True	OFFICE_ZIP_HEADER, OFFICE_PK_HEADER, OFFICE_PK_HEADER, Office_PK_Header
File150	Office Open XML	4cc3cee04f7e695a8de8591e15f19fc1	06b1ffb9db80cdce1e1623453882ed96b9602335	9db6a42860fa24a4f1b731f6ec736bd1235f95806c66b6d27064dde5917a974e	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File151	Office Open XML	c0e566ec23e885ddeb11e5a4070fb841	4247d02a033cb899485941a63f2f36e76c76c65f	861b99687b257f226a124b0bf7d028d46bef5cfb31cccf	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, IOC_HTTP, IOC_HTTPS, PE_MZ_HEADER,

				4e68bd448af06fc2d4		OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, BACKDOOR_HTTPS_URL, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File152	JPEG image	d647f67138863d8bf85b246774392709	c5eec9ec4683f0eafa40897a37d7d88b52fc21a6	6329dda11c90cc6461e58fbebfb0ed1ae744d5177b647748826eb22472b128c4	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File153	Office Open XML	07b039a576b247f2719d7c1372e7d1c7	8ade27f7dcc94cc0537bfa99754daaf97b8adbff	2d722a116f991e38f3f64fd486109be6280d91c1804682b4e40f67a7682d899f	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File154	ASCII text	a1c1ccba318f1630f5ffbdca6cdd12b2	926b7d6b492c6f9c315cfbfbec82a7bbcad7d30b	42f2787a27c9889b60b1ef6da602c57c9a4fe0b6a945c7cc0fda60a8980d7e35	False	

File155	PE executable	774faa35314ccf72d1b9d841b4532b03	ef57d337b2480bf4981091d9ec3072494f3c76f1	3e30293590a4f124e5ad3a783f312f0a4607331799d31e103011a90af5aa8e92	True	PE_MZ_HEADER_O0, PE_API_VIRTUALALLOC, PE_MZ_HEADER, PE_API_VIRTUALALLOC, PE_MZ_HEADER, PE_MZ_Header
File156	P DF document	8e984f349c1855d2ac331a822f548ddd	a8a0da335bc884d7c7314719036f326c92233995	2dc04d6ab4d83b63c572f6a2aa0817c70394b8cb7342ea603a04e8cee59d764e	True	PE_MZ_HEADER_O0, PDF_HEADER, IOC_HTTP, IOC_HTTPS, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, BACKDOOR_HTTPS_URL, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, PDF_Header, Office_OLE_Header, Office_PK_Header
File157	ASCII text	362889c21e1a9f7404c13205ea1b9fde	5c86fc5e9473205eba78ad2f46718d01c588b502	320bee996580871e3863b5d8bcb333a83519bff327e2764bb82f84065f44d25d	False	

File158	Binary data	5eba08d45abbd9fdbf82cc748a2b9774	1e768d7f6c7229c0db5e17f636f203ee2e084f72	164de5ced33b91f414e4837d123d2d77b44a1abc6bd23ccb8183f0e1355d0d9b	True	IOC_HTTPS, BACKDOOR_HTTPS_URL
File159	PE executable	2fb6c16c0f1c39d9ec32dbb311990b9c	dd56bde584ef1c333304c30f2cfa7311f576139a	c50e84b679470efe6494352ebef66cda2b5bda1dd30981036ff4faaca28d9544	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, PE_MZ_HEADER, PE_MZ_Header
File160	PE executable	5b88c51ea10f9db362369f3d71bea569	bee2ed1bea54e7d00b1bd0d44d817406d74a2af3	d0e714e047c50e9635d0eb408a19a5487f64950d40aa7e3f5fb03fd80550d9ed	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, PE_MZ_HEADER, PE_DOS_MESSAGE, PE_MZ_HEADER, PE_DOS_STUB, PE_MZ_Header
File161	Office Open XML	c875c6cc8e2086011748fd6f9de83ff9	b75a7ca6961ffdcafd25f6b3b5a523a71b0fefc1	b424fe8fe48d94e754bf63fc1d31eca150177edbd63065873931b833b1d92c8d	True	OFFICE_ZIP_HEADER, OFFICE_PK_HEADER, OFFICE_PK_HEADER, Office_PK_Header
File162	Office Open XML	174c53b13bb9ddbd	7be93d31bbb834fd3e1a073	19cfcbd8a84eafdc7d99627b9a8e6a3b38d24	True	OFFICE_ZIP_HEADER, OFFICE_PK_HEADER,

		b1984108cf13e210	3d8d507ee168852a3	14e124a412cd928ff0c3107467e		OFFICE_PK_HEADER, Office_PK_Header
File163	JPEG image	0691b1534a7b7dfa1f311471d386d105	68623a4e28d08d0de4e03b9089981c77563d58ba	a318143f9603d13e7c8f2afbcb27d8dd4045676773ea4170e60543e39f9d5aef	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File164	PE executable	f3dec47bdc290fb01d5d908775321ea7	f0eefa4f62179cf8ed63de2d287512089e95a9be	2d6f72967598597380 48cf02b07f381cab62045037950e590f419df824adfc36	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, IOC_HTTP, PE_MZ_HEADER, PE_DOS_MESSAGE, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File165	Binary data	1d786070021811ce3d962a420d868155	6e09b863e9d2f5d7bdff1e2b2bd4e4afef7876dc	1adecb3b5a3882cedce9fd64cddafebdcb045fe7035aac8f02ffe64dc827593c	True	IOC_HTTPS, BACKDOOR_HTTPS_URL

File166	PE executable	1af712a59ab34e0c131b40018d31c0f2	2c8951f8b0528ea9db00688cd76e14383ed0ed21	565d4e064b9757bf3e30ff5f229f401c113aa6f073eb823f6d8994be5b189f0b	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, PE_MZ_HEADER, PE_DOS_MESSAGE, PE_MZ_HEADER, PE_DOS_STUB, PE_MZ_Header
File167	ASCII text	dfb1b490080d1aae0067e35550910c4d	d88c7c5224c113911860215 8ad1d0217210780b0	147a3761456127bcb8ada2c34728a301d01acd316946aad7d605c3a9ce37e6f2	True	IOC_HTTPS, BACKDOOR_HTTPS_URL
File168	ASCII text	cc46e3e9ef97cbdc56318ee7ff23d73e	288f748f941f87169209eb1a058034c9a83e9b48	f831e7eed577481687a9bc0b48024e5e40b6f655fcde073ede964b50be5d55d9	True	IOC_HTTPS, BACKDOOR_HTTPS_URL
File169	Binary data	d44a541590b9d5973136d0e595da9ae2	35c5bf4c4b20ff967dc35d6dd1ed79037f3189b8	5ca9c2b5b336be53bfe6310c3ea4a51f85d003898e267d9871803d05b1c1214f	True	IOC_HTTPS, BACKDOOR_HTTPS_URL

File170	Office Open XML	fb9de5ec633240495f1ee57735839420	18771e2a544fdaffb6403aedf84b539f9d1fdb22	bf2180708e187ea7b3118b65d83767054c8fbdfc258c3fc7d0980f4d936de028	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File171	Office Open XML	19af6dc29171e27844e34c3c807231d9	976a891eb695b834305d28d287311793cf803222	021ea4dc427c81adebedb8152f67e67de7dc74417b65c2e276c58ca7d27112fc	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, IOC_HTTP, PE_MZ_HEADER, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File172	ASCII text	e322949af834af9a143a29625e33a4e4	018baebf31f8635703aa99d8e66ee097760a46f8	f98118e352c0c17b1bd04afee6f258d5c29116bf5e6b74fa0a28148ab372519a	True	IOC_HTTPS, BACKDOOR_HTTPS_URL
File173	PE executable	3c555bd977fd395f0e3e0b3758071d52	06849f3bfb38b3b4aa5e2a0c0ac88734478bb9f7	d042f2a62c3169aae27e079f596effacf5c7d98e3777f23	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, IOC_HTTP, PE_MZ_HEADER, PE_DOS_MESSAG

				0908e1a27f97d4aeb		E, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File174	ASCII text	a8dfc4d3598873ff3fd86e28be66108f	474981497504ac58781039c01b9d3b32b18ed191	40e4e9e028ff2c341f0f10c7e39bf16bb5328d3faa71e87b4e9bdb117101f0da	False	
File175	Binary data	ec7522bc9873d8fb52d2dba4f19161ea	989ae95e6645c4f35a6326c1e69973954b2d75c8	e1be23b940aa48907364cf126cd3ba656d13f6e9db0317f5072e22952e3c36f2	True	IOC_HTTPS, BACKDOOR_HTTPS_URL
File176	PE executable	6d8a087c87e8c522ae0b7948357ee226	3001282ecd140d7120b94da8defcb02dbb26a8b0	827be39a735f8c49e3fc483b5a57401c463c0aa306ab14f277f20453f45833a1	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, PE_MZ_HEADER, PE_DOS_MESSAGE, PE_MZ_HEADER, PE_DOS_STUB, PE_MZ_Header

File177	PE executable	894e538fbd29d9af2dac82abbb798aa8	3c28b3063ce80b3fd61e0afc6934e3180f5bef12	b12679d33126d2dcb0cd3625fccf5c3afc40d95c1be36dc55f7471de94929d23	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, IOC_HTTP, PE_MZ_HEADER, PE_DOS_MESSAGE, BACKDOOR_HTTP_URL, PE_MZ_HEADER, PE_DOS_STUB, PE_MZ_Header
File178	Binary data	955ddefc4f016045fb167016e499dd15	ecf760f19d07c912466db0f04b5bebb91cb07a22	8f10f3044eedf59e35d2a9b06eafeb6c73acfb25cb2cbac88b55e57e49034e9d	True	IOC_HTTPS, BACKDOOR_HTTPS_URL
File179	ASCII text	130668abb2367024e10447831c36f700	6b0e6307e68f873cc4c01e3c9d02b61269784b85	2d0f446840b79edc6392af5a19603d1092bf816a0853aa7e4017029ba05c7616	False	
File180	PE executable	6a9a905f16f64919e6892a2e8d37a5b7	faef2d937021c3ca367f275f0128b5ebaf50dbd8	93ad8b3ae0098063a59ecb9d4784d6222778e52c7c5f1d18238992842ba02ed6	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, PE_API_VIRTUALALLOC, IOC_HTTP, PE_MZ_HEADER, PE_DOS_MESSAGE, PE_API_VIRTUALA

						LLOC, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File181	Office Open XML	cdc2e1b7b617de1db19b5820fc9380a0	c19d042042cfe77f58ce28d55eca3f9741ba942c	8a44bdbeaa77f132076aba2f6e3f59738ea5f75cc17717536c65f862aaa3131c	True	OFFICE_ZIP_HEADER, OFFICE_PK_HEADER, OFFICE_PK_HEADER, Office_PK_Header
File182	PDF document	adc7a2cddbfa9080df9a9bce33153410	7774fd6a224c351c149d5db71d5c40fcfa762bf1	2e4e83d7d42bbd305f4bb20a15f0f4b3346b7b335cb50d8c3a546f681ca1b832	True	PE_MZ_HEADER_O0, PE_UPX0_SECTION, PDF_HEADER, IOC_HTTP, PE_MZ_HEADER, PE_UPX0_SECTION, PDF_HEADER, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, PDF_Header
File183	Office Open XML	5c4c9c4f71b373d50967fea0d9293b80	ed4ac6cf06a6828888f24b58416542dacf6e9960	42efce0f18b4f04638c0f8804ac4b80a69992259e8e4cd	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEAD

				842fe6b105eea3bfb2		ER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File184	PNG image	edf9fd64840aca234a2cf40acd08026d	6faea83b19c6cbd392abe6df9c6fc0d10570d67b	ef084f96946a33800a10c8cc5baea73a48025bcb100e1ab2c7f9c68fdd5755c3	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File185	Office Open XML	5b1392a9f190a431ac6bac3b2f5be47c	b8fc3a4d1bb60155967dbf863b71ddd42148b1e0	5a61ca0690a051e9dace2cacd65a9aa956278cbef643e4a259d37d8b1da2055a	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File186	ASCII text	cd58b4860366a300cfb76a652d6e1dbf	1376127733d1bfb923dafc1aa0d47dcd078cd97e	c665a33a2462010c88cc488ac00654401470 0c2d19b931e528cf40041cc5e9b0	False	
File187	Office Open XML	77d26a68e9264e9f	028eee19afce5d914baec76	d0a731438645729fef6185853644eb033f089	True	OFFICE_ZIP_HEADER, OFFICE_PK_HEADER,

		acac4242de2e740f	9287298cb9a3f8b8e	44318d6085f46c9561a1a7dc074		OFFICE_PK_HEADER, Office_PK_Header
File188	ASCII text	ce1742175ff199f4d207ac4edc013419	c7324e8cca9b6ac3a39e2baaa564d52e13966a9f	915c0560471eece77640068019d519f6f5cb345877fc4b7b2006a2cc58277ddf	False	
File189	Office Open XML	38c425bff814be81cacd15d2b4bcfeae	431f43d1284050f1a979a7260e72910213395704	61907f67de8563ebbc76740b23bd97e332cb78a052b4491e9d7f740946cc692f	True	OFFICE_ZIP_HEADER, OFFICE_PK_HEADER, OFFICE_PK_HEADER, Office_PK_Header
File190	PNG image	2ebe5a3b54f71e08d247aaacd9536fc7	aef536ba6dd1c9f499815a869340642f8a1a7d2a	4ed04cdb00ed9809033efbf8f9ad3debee924ff4564f200c96d88a9dc508d7dd	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, PDF_JS_TAG, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, PDF_Header, Office_OLE_Header, Office_PK_Header
File191	Office Open XML	7c3c2be5d0f5bd562546cc8d03829aa4	ae984a2eebfad940599f7bc75bd60838d6d4eb2d	19df1c62878cf4ef4634f87f35ad6b66a1df5433e934177	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, IOC_HTTP, PE_MZ_HEADER, OFFICE_PK_HEADER,

				2efa0ab2449962e92		BACKDOOR_HTTP_URL, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File192	Binary data	eb0293aa9d3cd061798d7fe931dc5fa6	78642c9626ac9ca7e1812f397923b81a0adfabf2	105e7abe3ee6468f590368d72d02b63ff6eafe147a0cfdd994f952e2b7c9acbd	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, IOC_HTTP, PE_MZ_HEADER, PE_DOS_MESSAGE, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File193	Office Open XML	a7f70b3ec9911480be7dacbd1e37820d	420cc47965de6c73fe3065931d8d6fef01792ea2	d3f23962bd711525f1e2797f0ec17f15d2a0b73665065bd20cb45d23fa6e3c99	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File194	Office Open XML	44661e9d8e0b19cc4cdc71c58dac4595	355ed21d8da04a0ea27205b807979240fbefb356	9c78b4fcd0a2b8c46af7574db506f6e5c1ae85250034be	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEAD

				f6e52c26a a88424b59		ER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File195	Binary data	8581f7c7 288375f1 4b8f164c 83ea0ac9	3e38c02ce6df 71a9aba792e 89ea20beeb8 032819	c0f86d100 3cd3c7117 dc032b943 928910944 d00a61de2 b3aef60f51 c902c896f	True	PE_MZ_HEADER_O 0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File196	Office Open XML	3acb832c 0b5bcd7d f8cc140a 771c612e	8e63f741347a 04d664a023c 6ec09c6c8ac9 9d327	9e22d6eed ee0a3399d c254fdfa16 95b4a33e3 7782c2fe7 dddca72f1 37e831c3c	True	PE_MZ_HEADER_O 0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File197	Office Open XML	abfdce43 32a47d00 56f54ecd 5ff64e41	8ff3d262e9c8 3355e5c7ca32 55c0688b53c6 c983	f5afab8876 440ad6cb9 b95a3988d 0d97bc9ea ffef86a3d6 68e74237d 7013bd0b	True	PE_MZ_HEADER_O 0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header

File198	Office Open XML	dd461e6bc31f224519587dffd93048f8	93c63eb338cddb1cf2e23374e2bdebdf79302f4c	ada5a897f6c0a25fd1f71df864553464356390 00cd10f1dae3d32675ecc75660	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, IOC_HTTP, PE_MZ_HEADER, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File199	PDF document	fd3900a6c106515c2db20b09720c7b3d	cf4cfbf3c7f8db99bedb4b58242b5a93f2e24408	be349d9ae09d8cf0f9065dd7a790a41416d3ebf93eb75015bd852266d3b223f9	True	PE_MZ_HEADER_O0, PDF_HEADER, IOC_HTTP, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, PDF_Header
File200	PE executable	cb98f5f2982feefd5e6d9fca98bb7e39	c5c554b41d7005dafdbfc89c4f4b776331cabc48	e8f2c7602555751cf8fe40de5e51d1d51c3abcaf617af949cae601c7f21cb4fb	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header

File201	PE executable	4e9a6f255bff5ab93b6a7a51ebf62e99	f4baeceff5c949c66a85edcbf3ac40a774e3b887	cca30ca51762ebc3627fb2bf0ad625f04e52ef1af1b5070132279f5f40015d7f	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, PE_MZ_HEADER, PE_DOS_MESSAGE, OFFICE_PK_HEADER, PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File202	Binary data	2497589c198a5c7d7c48f56bd5c2a083	f69b900367fa62e24947333fad2e4d8094002990	4fb8184bee8c99d59555e7e387a0ebec6df0e3e1801aa640133374ead7c56e78	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File203	Binary data	80d158a5fce462fb2d54c71adc8e0da6	f0957d0ec699d079d4ad5adc73525925ce7ff829	24795cf6cd080bb4897f9e532b34c1e2ebd3d841aa5bfcd30d8c75b833d69518	True	IOC_HTTPS, BACKDOOR_HTTPS_URL
File204	Binary data	d601a536054abe2a84612a08b5b749f5	902f98265eb67fcd8d7da5eb0b8198071d887c85	a3a56b01c83255a27ffc21329316ee7bbc2fe6a0e0dc4449d1e8d154312b7ce4	True	IOC_HTTPS, BACKDOOR_HTTPS_URL

File205	JPEG image	eae001e9a70c97bb5a2e6f17849c1d12	72cc668b05bac25159ee0198ce375b4358ab375c	d0aa4a315d66e1197e18c51b0924b09e5c716b1204fafb8ac43d101a9da06a89	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File206	Office Open XML	16aa2764840ee306b4d60ed118ed642b	3b02a535fa7a2ccbe80d975f2b468de92dd3b8d5	f435ebe4e5aef3479aedf5b2128f6111635cce44f4a7f19cb3ea8d92d21b216d	True	OFFICE_ZIP_HEADER, OFFICE_PK_HEADER, OFFICE_PK_HEADER, Office_PK_Header
File207	PE executable	8c5c9499e5bf2515b46ad201e8340748	df382e06b0b50420b6d63240efe0681998fa8dfb	101bcf563ebe0e8ef4f0a63513614791b3c306c10f8382416276992a85dd63b1	True	PE_MZ_HEADER_O0, PE_DOS_STUB_CANNOT_RUN, IOC_HTTP, PE_MZ_HEADER, PE_DOS_MESSAGE, OFFICE_PK_HEADER, BACKDOOR_HTTP_URL, PE_MZ_HEADER, PE_DOS_STUB, OFFICE_PK_HEADER, PE_MZ_Header
File208	Office Open XML	cb5e8db8123b89323a4822992b5ddd34	b7d12ca8a56e690c38b3d2c82a3b52e138176b4f	24a50953cdcb5923deb5c91fc03c89144d6b48ce288457	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER,

				a85f380f511cf6f0ff		PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File209	PDF document	76909cea13b5097b22a7faa800652758	fc407e7e35ab2470cc9228bacc41394a169a6fd5	19de82512ea67a8ca22985d597dc2190f33d23a410315fde887e60bd525a268b	True	PE_MZ_HEADER_O0, PDF_HEADER, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, PDF_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, PDF_Header
File210	Office Open XML	d23ddb02acd5a876e306962bd71f1631	43db097d4df5012de9903c2de395a68147071827	ff59a2acd6b318ffc75e2ed3c5936c69d56eda24b6a57049a5d75f14308c00e7	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header
File211	Office Open XML	c70ec8c66f59ecd212b7d08a4357b5d5	50b27cdf5ab5e9b9cc52a102fafae2c112403a1e	dcfff3750dfaabd52f4c62a35a98f1d0df5227aec8ebfcc2e8167bc71c4b05cc	True	PE_MZ_HEADER_O0, OFFICE_ZIP_HEADER, PE_MZ_HEADER, PDF_JS_TAG, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header, Office_PK_Header

File212	ASCII text	4a08c4bd4d73b050f07c51d9d7192e57	df37ae666a00d846792533782a95b28b66d1320a	5949cf679127b5a6fdb19a4200f72a8bb2b6095bd61e64692cb6b39a06fa96f9	False	
File213	PE executable	2a951f81a38691f52fb9985513e46946	c79bf016e59f5a63697184e470304af54d1689da	47914ae30332a9b287cd49cf1848a923790dec9693742ea370ee5ae9d5f4cd4f	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, PE_MZ_HEADER, PE_MZ_Header
File214	JPEG image	3938baaa0bb6e35e2420178db164cb3d	03ad96e61cf716ff65919902d92c5cff197eee2f	568ea3e5a0735f8eb6cde01fe02dd588f2474ecb959188c4fe55a4252cf1d429	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File215	Binary data	ebc84c0bae4a06d2db24c264d52dbb1f	d9c43a3c4df555360bf5e0eace9d2c04a5df74fd	39f623db0d8f79359e3f585fec270bf9a189c19392e4a69d111ecc71415bad32	True	IOC_HTTPS, BACKDOOR_HTTPS_URL
File216	Binary data	825341f997632ff776e477883fe6493c	e9bfe5ea0889af5ed63d66a63b7b5b0861212ebb	44e76ab7e070b8e79e26f13b31c2c574ac6194364864ed485961ec	True	IOC_HTTPS, BACKDOOR_HTTPS_URL

				bce0a19dc7		
File217	Binary data	f1e7f2ec19d0fd37d892e992a71fcbd6	5aa2bda8638878126e463b0f935a50d693bdbcee	0d359811829860ca2834f57db70b89d41ec044ab22f85f1a87f5c22228d5d899	False	
File218	PNG image	bb84661af617931ba5b294a8737c421d	76a256f4ddd4584f562af45a06ba9952434d1671	0037cf511e2b34db246d16b964f9373c21b2dc7b268b9cfcbe89620b16fc4c74	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File219	JPEG image	e87decf883153221f84d73f70bb2f53b	043296334c60c14c8be4fce8b1a4bb962847581a	a119016709f406bf1550822b2bdc3807c1a2fdb953e387e9257e08d1a81549f9	True	PE_MZ_HEADER_O0, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_HEADER, OFFICE_PK_HEADER, PE_MZ_Header
File220	ASCII text	db631805541e0958fd9e6e3616a42c98	28b9d9f7b21ca28bf2b7e01f1da1f5a2080fe674	a02767a52d8c1fcde77ed6d297d3ac17029275c6125af1c6ac5a618904f71c6e	False	